

**RANCANG BANGUN TONGKAT PINTAR DETEKSI
OBJEK DENGAN *CNN MOBILENET* DAN NAVIGASI
BAGI TUNANETRA**

SKRIPSI

**Diajukan Sebagai Persyaratan Untuk Memperoleh
Gelar Sarjana Terapan Teknik**



**Disusun Oleh
RAHMAWATI DIVANA PUTRI
2241160078**

**PROGRAM STUDI JARINGAN TELEKOMUNIKASI DIGITAL
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI MALANG**

2026

**RANCANG BANGUN TONGKAT PINTAR DETEKSI
OBJEK DENGAN *CNN MOBILENET* DAN NAVIGASI
BAGI TUNANETRA**

SKRIPSI

**Diajukan Sebagai Persyaratan Untuk Memperoleh
Gelar Sarjana Terapan Teknik**



**Disusun Oleh
RAHMAWATI DIVANA PUTRI
2241160078**

**PROGRAM STUDI JARINGAN TELEKOMUNIKASI DIGITAL
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI MALANG**

2026

HALAMAN PENGESAHAN

RANCANG BANGUN TONGKAT PINTAR DETEKSI OBJEK DENGAN CNN MOBILENET DAN NAVIGASI BAGI TUNANETRA

Oleh:

Rahmawati Divana Putri

NIM. 2241160078

Skripsi ini telah dipertahankan di depan dewan penguji pada 14 Juli 2026,
dan disahkan oleh:

Pembimbing I:

Lis Diana Mustafa, S.T., M.T.

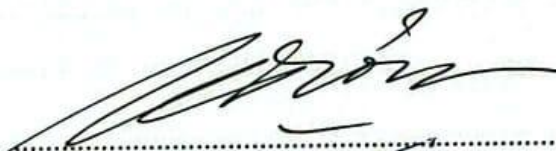
NIP. 197805052001122003



Pembimbing II:

Ir. Hudiono, M.T.

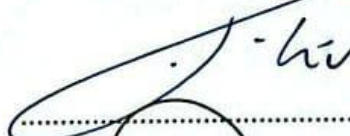
NIP. 196301151988111001



Penguji I:

Aad Hariyadi, S.ST., M.T.

NIP. 196301041988031002



Penguji II:

Muhammad Syirajuddin S., S.T., M.T

NIP. 198510172019031009



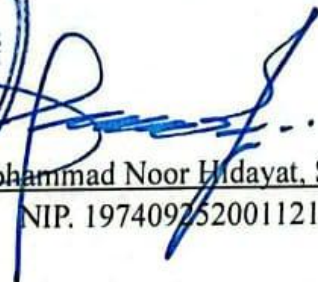
Malang, 07 Agustus 2026



Mengetahui,
Ketua Jurusan Teknik Elektro

Prof. Dr. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D.

NIP. 197409252001121003



Menyetujui,
Koordinator Program Studi
Jaringan Telekomunikasi Digital

Mila Kusumawardani, S.T., M.T.

NIP. 197503012000032002



PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan dibawah ini:

Nama : Rahmawati Divana Putri

NIM : 2241160078

Judul Skripsi : Rancang Bangun Tongkat Pintar Deteksi Objek Dengan
CNN MobileNet Dan Navigasi Bagi Tunanetra

Saya menyatakan dengan sebenar-benarnya bahwa skripsi ini merupakan hasil penelitian, pemikiran, dan pemaparan asli saya sendiri. Saya tidak mencantumkan tanpa pengakuan bahan - bahan yang telah dipublikasikan sebelumnya atau ditulis oleh orang lain, atau sebagai bahan yang pernah diajukan untuk gelar atau ijazah pada Politeknik Negeri Malang atau perguruan tinggi lainnya.

Apabila dikemudian hari terdapat penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik sesuai dengan peraturan yang berlaku di Politeknik Negeri Malang.

Demikian pernyataan ini saya buat.

Malang, 07 Agustus 2026



Rahmawati Divana Putri

NIM. 2241160078

ABSTRAK

Rahmawati Divana Putri, 2026 “Rancang Bangun Tongkat Pintar Deteksi Objek Dengan CNN MobileNet Dan Navigasi Bagi Tunanetra” Program Studi Jaringan Telekomunikasi Digital, Jurusan Teknik Elektro, Politeknik Negeri Malang. Dosen Pembimbing I: Lis Diana Mustafa, ST, MT Dosen Pembimbing II: Ir.Hudiono, M.T.

Tongkat pintar merupakan perangkat bantu yang penting bagi penyandang tunanetra untuk mendukung mobilitas dan menghindari rintangan selama beraktivitas. Namun, sebagian besar sistem tongkat pintar yang telah dikembangkan masih mengandalkan satu jenis sensor sehingga informasi lingkungan dan kemampuan navigasinya menjadi terbatas. Oleh karena itu, penelitian ini mengembangkan sistem tongkat pintar yang mengintegrasikan deteksi objek berbasis Artificial Intelligence (AI), pengukuran jarak menggunakan sensor LiDAR VL53L1X, serta layanan navigasi berbasis OpenStreetMap dan *Open Source Routing Machine* (OSRM) dalam aplikasi Android. Sistem dikendalikan oleh mikrokontroler ESP32 dan dilengkapi dengan fitur perintah suara, *Text-to-Speech* (TTS), umpan balik getaran, serta pemantauan lokasi pengguna secara *real-time* melalui Firebase Realtime Database. Model deteksi objek dikembangkan menggunakan MobileNet SSD dan dilatih menggunakan dataset khusus, yang menghasilkan nilai mean *Average Precision* (mAP) sebesar 31,0% dan mAP@50 sebesar 52,5% pada 150.000 langkah pelatihan dengan nilai total *loss* sebesar 0,803. Selain itu, sensor LiDAR VL53L1X mampu mengukur jarak rintangan hingga 400 cm dan memberikan umpan balik getaran berdasarkan jarak objek yang terdeteksi. Integrasi fitur deteksi objek, pengukuran jarak, navigasi, dan pemantauan lokasi memungkinkan sistem memberikan informasi lingkungan serta arahan perjalanan melalui umpan balik suara dan getaran. Hasil penelitian menunjukkan bahwa sistem tongkat pintar yang dikembangkan mampu mendukung deteksi rintangan dan navigasi bagi penyandang tunanetra melalui pemanfaatan teknologi Artificial Intelligence (AI), Internet of Things (IoT), dan aplikasi mobile.

Kata Kunci: Tongkat Pintar, Deteksi Objek, MobileNet SSD, LiDAR VL53L1X, Navigasi, Android.

ABSTRACT

Rahmawati Divana Putri, 2026 “Design and Implementation of a Smart Cane for Object Detection and Navigation Assistance for Visually Impaired People Based on CNN MobileNet” Digital Telecommunications Network Study Program, Electrical Engineering Department, Malang State Polytechnic. Advisor I : Lis Diana Mustafa, ST, MT Advisor II: Ir.Hudiono, M.T.

Smart canes are important assistive devices for visually impaired individuals to support mobility and obstacle avoidance during daily activities. However, most existing smart cane systems rely on a single sensing technology, resulting in limited environmental information and navigation capabilities. Therefore, this study develops a smart cane system that integrates Artificial Intelligence (AI)-based object detection, distance measurement using a VL53L1X LiDAR sensor, and navigation services based on OpenStreetMap and *Open Source Routing Machine* (OSRM) within an Android application. The system is controlled by an ESP32 microcontroller and equipped with *Voice Commands*, *Text-to-Speech* (TTS), vibration feedback, and *real-time* location monitoring through Firebase Realtime Database. The object detection model was developed using MobileNet SSD and *trained* on a custom dataset, achieving a mean *Average Precision* (mAP) of 31.0% and mAP@50 of 52.5% at 150,000 *training steps* with a total *loss* value of 0.803. In addition, the VL53L1X LiDAR sensor successfully measured obstacle distances up to 400 cm and provided vibration feedback based on the detected distance. The integration of object detection, distance measurement, navigation, and location monitoring enables the system to provide environmental information and navigation guidance through audio and tactile feedback. The results indicate that the proposed smart cane system can effectively support obstacle detection and navigation assistance for visually impaired individuals by combining AI, Internet of Things (IoT), and mobile application technologies.

Keywords: Smart Cane, Object Detection, MobileNet SSD, VL53L1X LiDAR, Navigation, Android.

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas segala rahmat, karunia, dan hidayah-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul **“Rancang Bangun Tongkat Pintar Deteksi Objek dengan CNN MobileNet dan Navigasi bagi Tunanetra”** dengan baik. Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Terapan pada Program Studi D-IV Jaringan Telekomunikasi Digital, Jurusan Teknik Elektro, Politeknik Negeri Malang.

Dalam proses penyusunan skripsi ini, penulis memperoleh banyak bantuan, bimbingan, dukungan, serta motivasi dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih kepada:

1. Allah SWT yang telah memberikan rahmat, kesehatan, kemudahan, dan kesempatan sehingga penulis dapat menyelesaikan skripsi ini.
2. Bapak Supriatna Adhisuwignjo, S.T., M.T., selaku Direktur Politeknik Negeri Malang.
3. Bapak Prof. Ir. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D., selaku Ketua Jurusan Teknik Elektro Politeknik Negeri Malang.
4. Ibu Mila Kusumawardani, S.T., M.T., selaku Koordinator Program Studi D-IV Jaringan Telekomunikasi Digital.
5. Ibu Lis Diana Mustafa, S.T., M.T., selaku Dosen Pembimbing I yang telah meluangkan waktu, tenaga, dan pikiran dalam memberikan bimbingan, arahan, serta masukan selama proses penyusunan skripsi.
6. Bapak Ir. Hudiono, M.T., selaku Dosen Pembimbing II yang telah memberikan bimbingan, arahan, dan dukungan selama penyusunan skripsi.
7. Kedua orang tua yaitu Bapak Wanto dan Ibu Riani Setyowati tercinta, yang senantiasa memberikan kasih sayang, doa, dukungan, motivasi, serta pengorbanan yang tidak ternilai sehingga penulis dapat menyelesaikan pendidikan hingga tahap ini.

8. Kakak sekaligus sahabat, Aldi Aprilla Nur Rizqi, yang senantiasa memberikan doa, dukungan, motivasi, serta semangat kepada penulis selama proses perkuliahan hingga penyusunan skripsi ini, sehingga penulis dapat menyelesaikan penelitian ini dengan baik.
9. Seluruh keluarga yang selalu memberikan dukungan dan doa kepada penulis.
10. Teman-teman penulis, Laila, Naurotur, Nisa, Raudhatul, Ayu Juliana, Agustin yang telah banyak memberikan semangat dan membantu dalam pengerjaan, pengangambilan data serta pengujian hingga penulis dapat menyelesaikan skripsi ini.
11. Teman-teman Program Studi D-IV Jaringan Telekomunikasi Digital angkatan 2022 yang telah memberikan bantuan, semangat, dan kebersamaan selama masa perkuliahan.
12. Semua pihak yang telah membantu dan mendukung penulis selama proses penyusunan skripsi yang tidak dapat disebutkan satu per satu.
13. Terakhir, kepada diri penulis sendiri yang telah berusaha, bertahan, dan terus berjuang dalam menghadapi berbagai tantangan selama proses perkuliahan hingga penyusunan skripsi ini.

Penulis menyadari bahwa skripsi ini masih memiliki keterbatasan dan jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Semoga skripsi ini dapat memberikan manfaat bagi pembaca serta menjadi salah satu kontribusi dalam pengembangan teknologi asistif bagi penyandang tunanetra.

Malang, 15 Juni 2026



Rahmawati Divana Putri

DAFTAR ISI

HALAMAN PENGESAHAN	i
PERNYATAAN KEASLIAN SKRIPSI.....	ii
<i>ABSTRAK</i>.....	iii
<i>ABSTRACT</i>	iv
KATA PENGANTAR.....	v
DAFTAR ISI	vii
DAFTAR GAMBAR	x
DAFTAR TABEL	xii
BAB 1	1
PENDAHULUAN.....	1
1.1 Latar Belakang Masalah	1
1.2. Rumusan Masalah	3
1.3 Batasan Masalah.....	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian.....	4
1.6 Luaran Penelitian.....	5
BAB II	6
TINJAUAN PUSTAKA	6
2.1 Penelitian Terdahulu	6
2.2 Kajian Teori	10
2.2.1 Tunanetra dan Teknologi Bantu Navigasi	10
2.2.2 Mikrokontroler ESP32	11
2.2.3 Sensor LiDAR VL53L1X	12
2.2.4 Motor Getar	13
2.2.5 18650 <i>Battery Shield</i> V3	14
2.2.6 Notifikasi Audio	15
2.2.7 Aplikasi Mobile.....	16

2.2.8	<i>Open Source Routing Machine (OSRM)</i>	16
2.2.9	<i>Convolutional Neural Network (CNN)</i>	17
2.2.10	Mobile Net SSD	23
2.2.11	Evaluasi Performa Model Deteksi Objek	25
2.2.12	DataSheet COCO	27
2.2.13	Android Studio	27
2.2.14	Firebase	28
2.2.15	Roboflow	28
2.2.16	Visual Studio Code (VS Code)	29
BAB III		30
METODOLOGI PENELITIAN		30
3.1.	Jenis Penelitian	30
3.2.	Tahapan Penelitian	30
3.3.	Perancangan Alat dan Aplikasi	32
3.3.1	Perancangan Alat	32
3.3.2	Blok Diagram Sistem	33
3.3.3	Diagram Alir Sistem	36
3.3.4	Diagram Alir Sistem Deteksi MobileNetSSD	38
3.3.5	Skema Perancangan Rangkaian Alat Pada Tingkat	41
3.3.6	Desain Alat Pada Tingkat	43
3.3.7	Desain Aplikasi	44
3.4.	Alat dan Bahan	46
3.5.	Alur Penggunaan Sistem	48
3.6.	Parameter Pengujian	50
3.7	Perancangan Program ESP32	51
3.8	Perancangan Database Firebase	54
3.9	Perancangan AI Deteksi Objek	54
3.10	Perancangan <i>Training</i>	56
BAB IV		58
HASIL DAN PEMBAHASAN		58
4.1	Tempat dan Waktu Penelitian	58
4.2	Hasil Penelitian	58

4.2.1 Hasil Perencanaan Desain Alat	59
4.2.2 Hasil Perencanaan Aplikasi	60
4.2.3 Hasil Pengujian Sensor LiDAR VL53L1X	65
4.2.4 Hasil Pengujian Modul Getar	69
4.2.5 Hasil Pengujian Daya Tahan Batrai Hardware.....	70
4.2.6 Hasil Pengujian Daya Tahan Batrai Sistem Aplikasi	71
4.2.7 Hasil Persiapan Dataset	72
4.2.8 Hasil <i>Training</i> MobileNet SSD	74
4.2.9 Hasil Evaluasi Model.....	80
4.2.10 Hasil Pengujian Parameter Deteksi Objek.....	96
4.2.11 Nilai <i>Confidence Threshold</i> yang Digunakan pada Sistem	97
4.2.12 Hasil Pengujian <i>Voice Command</i>	98
4.2.13 Hasil Pengujian <i>Text-to-Speech</i> (TTS).....	101
4.2.14 Hasil Pengujian Navigasi OSRM	102
4.2.15 Hasil Pengujian Database Firebase.....	104
4.2.16 Pengujian Deteksi Objek dan LiDAR Secara Bersamaan	107
4.2.17 Hasil Pengujian Sistem pada Lingkungan Indoor dan Outdoor	114
4.2.18 Dokumentasi pengujian sistem oleh penyandang tunanetra.	115
BAB V.....	118
PENUTUP.....	118
5.1 Kesimpulan	118
5.2 Saran.....	118
DAFTAR PUSTAKA.....	120
LAMPIRAN.....	123
DAFTAR RIWAYAT HIDUP	139

DAFTAR GAMBAR

Gambar 2. 1 Ilustasi Tunanetra	10
Gambar 2. 2 ESP32	11
Gambar 2. 3 Sensor LiDAR VL53L1X.....	12
Gambar 2. 4 Vibration Motor Module.....	13
Gambar 2. 5 18650 Battery Shield V3.....	14
Gambar 2. 6 Notifikasi Suara dari Aplikasi Android.....	15
Gambar 2. 7 Aplikasi Mobile	16
Gambar 2. 8 API OSRM.....	16
Gambar 2. 9 Arsitektur CNN	18
Gambar 2. 10 Convolutional Layer	19
Gambar 2. 11 Pooling Layer	20
Gambar 2. 12 Fully Connected Layer	21
Gambar 2. 13 Macam-macam CNN	22
Gambar 2. 14 arsitektur mobilenetv2	24
Gambar 2. 15 Contoh Dataset COCO	27
Gambar 2. 16 Logo Android Studio	27
Gambar 2. 17 Logo Firebase	28
Gambar 2. 18 Logo Roboflow	28
Gambar 2. 19 Logo Visual Studio Code.....	29
Gambar 3. 1 Flowchart Alur Tahapan Penelitian.....	30
Gambar 3. 2 Blok Diagram Sistem.....	33
Gambar 3. 3 Diagram Alir Sistem	36
Gambar 3. 4 Diagram Alir Sistem Deteksi MobileNetSSD	38
Gambar 3. 5 Skema Perancangan Rangkaian Alat pada Tongkat.....	41
Gambar 3. 6 Desain Alat.....	43
Gambar 3. 7 Desain Aplikasi Pengguna Tunanetra	44
Gambar 3. 8 Desain Aplikasi Monitoring Tunanetra.....	45
Gambar 3. 9 Gambar Potongan Program Pembacaan Sensor LiDAR	52
Gambar 3. 10 Program Motor Getar.....	53
Gambar 3. 11 Struktur penyimpanan data pada Firebase.	54
Gambar 3. 12 Library yang harus diinstal	55
Gambar 3. 13 Folder Datasets	56
Gambar 3. 14 Pembagian Dataset.....	57
Gambar 3. 15 Program Ekspor ke Format TensorFlow Lite	57
Gambar 4. 1 Pengujian Sensor Lidar.....	65
Gambar 4. 2 Hasil Pengujian Modul Getar Pada Serial Monitor	69
Gambar 4. 3 Lama Daya Tahan Batrai	70
Gambar 4. 4 Lama Daya Tahan Batrai Aplikasi	71
Gambar 4. 5 Konfigurasi Model Mobilenet SSD V2	74
Gambar 4. 6 Konfigurasi Training.....	75
Gambar 4. 7 Konfigurasi Optimizer dengan Cosine Decay	75
Gambar 4. 8 Augmentasi yang Digunakan	76
Gambar 4. 9 Konfigurasi train_input_reader pada file pipeline.config.....	76
Gambar 4. 10 Grafik Loss Evaluation Menggunakan Platform TensorBoard pada Proses Training 150k Steps.	79

Gambar 4. 11 Confusion Matrix	82
Gambar 4. 12 Classthreshold Pada File Objectdetector.Kt	97
Gambar 4. 13 Pengujian Pembaruan Data Real-Time Database Firebase.....	105
Gambar 4. 14 Penyimpanan Log Data Pada Firebase	105
Gambar 4. 15 Isi pada Log Data.....	106
Gambar 4. 16 Dokumentasi Pengujian Tongkat Pintar Oleh Penyandang Tunanetra.....	115
Gambar 4. 17 Dokumentasi Pengujian Tongkat Pintar Oleh Penyandang Tunanetra.....	116
Gambar 4. 18 Hasil Deteksi Objek Pada Aplikasi Android.....	116

DAFTAR TABEL

Tabel 2. 1 Penelitian Terdahulu	7
Tabel 3. 1 Daftar Pin Rangkaian Sistem.....	42
Tabel 3. 2 Alat dan Bahan.....	46
Tabel 3. 3 Konfigurasi Model SSD MobileNetV2	55
Tabel 4. 1 Hasil Perencanaan Desain Alat	59
Tabel 4. 2 Hasil Perancangan Aplikasi untuk Tunanetra	61
Tabel 4. 3 Hasil Perancangan Aplikasi untuk Monitoring Tunanetra.....	63
Tabel 4. 4 Hasil pengujian sensor LiDAR VL53L1X	66
Tabel 4. 5 Jumlah Dataset.....	73
Tabel 4. 6 Distribusi Jumlah Anotasi pada Setiap Kelas Objek	73
Tabel 4. 7 Hasil Loss Pelatihan Model MobileNet SSD	77
Tabel 4. 8 Hasil Evaluasi mAP Selama Pelatihan	78
Tabel 4. 9 Hasil Average Precision (AP) Model MobileNet SSD per Kelas.....	80
Tabel 4. 10 Hasil Average Recall (AR) Model MobileNet SSD per Kelas	81
Tabel 4. 11 Metrik Precision dan Recall Berdasarkan Confusion matrix.....	83
Tabel 4. 12 Hasil Validasi Visual Evaluasi Model Train 150.000 steps	93
Tabel 4. 13 Hasil Pengujian Confidence Threshold	96
Tabel 4. 14 Nilai Confidence Threshold yang Digunakan pada Sistem.....	97
Tabel 4. 15 Hasil Pengujian Voice Command	99
Tabel 4. 16 Hasil Pengujian Navigasi OSRM	102
Tabel 4. 17 Pengujian Deteksi Objek dan LiDAR Secara Bersamaan	107
Tabel 4. 18 Hasil Pengujian Deteksi Kamera dengan Objek.....	111
Tabel 4. 19 Tabel Perhitungan Hasil Pengujian Jarak Lidar.....	113
Tabel 4. 20 Hasil Pengujian Sistem pada Lingkungan Indoor dan Outdoor	114

BAB 1

PENDAHULUAN

1.1 Latar Belakang Masalah

Mobilitas merupakan kebutuhan mendasar bagi setiap individu, termasuk penyandang tunanetra. Ketidakmampuan untuk mengenali lingkungan sekitar secara visual menjadikan aktivitas sehari-hari seperti berjalan kaki, menyeberang jalan, atau mencari arah menjadi tantangan serius[1]. Skala permasalahan ini sangat signifikan secara global. *World Health Organization* (WHO) melaporkan bahwa gangguan penglihatan di seluruh dunia telah mencapai 2,2 miliar orang, dengan 1,1 miliar di antaranya mengalami kebutaan atau disabilitas netra total. Secara khusus, Indonesia menempati peringkat ketiga di dunia dengan jumlah penyandang tunanetra terbanyak setelah India dan China[2][3]. Data Kementerian Kesehatan RI menunjukkan bahwa angka ini setara dengan 1,5 persen dari total populasi, atau diperkirakan mencapai 4 juta jiwa dari lebih dari 270 juta penduduk Indonesia[4]. Besarnya jumlah ini menegaskan urgensi pengembangan teknologi bantu untuk meningkatkan kemandirian dan keselamatan para penyandang tunanetra dalam beraktivitas[5].

Selama ini, penyandang tunanetra mengandalkan tongkat putih konvensional sebagai alat bantu navigasi. Meskipun sederhana dan ringan, tongkat ini hanya mampu mendeteksi objek yang bersentuhan langsung dan tidak memberikan informasi spasial atau arah yang lebih luas[6]. Untuk menjawab keterbatasan tersebut, beberapa penelitian telah mengembangkan tongkat pintar dengan sensor ultrasonik dan GPS untuk mendeteksi hambatan dan mengetahui posisi pengguna secara *real-time* [7], [8]. Namun, teknologi tersebut masih memiliki keterbatasan dalam mengenali jenis objek secara spesifik.

Perkembangan teknologi *Artificial Intelligence* (AI), khususnya dalam bidang visi komputer, membuka peluang baru dalam deteksi dan klasifikasi objek. Model *MobileNet Single Shot Detector* (SSD) merupakan arsitektur *Convolutional Neural Network* (CNN) yang ringan dan dirancang untuk perangkat mobile. Jika

diimplementasikan pada aplikasi Android, model ini mampu mengenali objek seperti manusia, motor, mobil, lubang gorong-gorong, tembok, bangku taman, pohon dan sepeda secara *real-time* [9], [10]. Di sisi lain, sensor LiDAR VL53L1X mampu memberikan deteksi halangan hingga 4 meter, sehingga bisa digunakan untuk mendeteksi rintangan fisik di lingkungan sekitar pengguna [11], [12].

Dalam penelitian ini, dikembangkan sistem tongkat pintar yang menggabungkan kamera pada perangkat Android dengan model MobileNet SSD untuk mendeteksi objek di sekitar pengguna serta sensor LiDAR VL53L1X untuk mengukur jarak halangan. Sistem dikendalikan oleh mikrokontroler ESP32 dan dilengkapi motor getar pada tongkat sebagai umpan balik fisik. Selain itu, sistem terhubung dengan aplikasi Android melalui Bluetooth dan mendukung fitur navigasi berbasis OpenStreetMap menggunakan API OSRM, perintah suara menggunakan *Speech Recognizer*, serta notifikasi audio berbasis *Text-to-Speech* (TTS).

Pengembangan penelitian ini berfokus pada integrasi antara deteksi objek berbasis AI, pengukuran jarak menggunakan sensor LiDAR VL53L1X, dan aplikasi Android yang bekerja secara terhubung. Sistem memanfaatkan jaringan internet untuk mendukung navigasi dan sinkronisasi data ke Firebase Realtime Database. Melalui kombinasi kamera Android, ESP32, dan sensor LiDAR, sistem dapat memberikan informasi mengenai objek dan jarak halangan melalui umpan balik berupa getaran dan suara. Selain itu, sistem juga menyediakan fitur pemantauan lokasi pengguna yang dapat diakses melalui aplikasi. Dengan pendekatan tersebut, penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan teknologi bantu navigasi bagi penyandang tunanetra.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang sistem tongkat pintar yang mampu mendeteksi hambatan secara *real-time* menggunakan sensor LiDAR VL53L1X dan menyampaikan informasi tersebut kepada pengguna tunanetra melalui notifikasi suara dan getaran?
2. Bagaimana mengintegrasikan kamera *smartphone* dengan model MobileNet SSD untuk mendeteksi objek penting di lingkungan sekitar seperti manusia, kendaraan, dan sepeda?
3. Bagaimana menyinkronkan data sensor dengan sistem navigasi berbasis aplikasi Android agar mampu memberikan arah tujuan menggunakan API OSRM ?
4. Bagaimana mengimplementasikan perintah suara (*Voice Command*) melalui microphone untuk mengendalikan fungsi sistem?

1.3 Batasan Masalah

Agar penelitian lebih terfokus dan terarah, maka batasan masalah ditetapkan sebagai berikut:

1. Sistem hanya mengukur jarak objek menggunakan sensor LiDAR VL53L1X tanpa mendeteksi kecepatan dan arah pergerakan objek.
2. Sensor LiDAR VL53L1X digunakan untuk mendeteksi rintangan di depan pengguna dengan batas deteksi hingga ± 4 meter. Sehingga sistem hanya mendeteksi objek yang berada di area depan pengguna, dan tidak mencakup objek di sisi belakang pengguna.
3. Deteksi visual menggunakan model MobileNet SSD pada aplikasi Android, terbatas pada 8 kelas objek, yaitu orang, mobil, motor, sepeda, bangku, lubang, tembok, dan pohon.
4. Pengujian deteksi objek dengan kamera dilakukan pada kondisi pencahayaan normal, sedangkan pengujian pada kondisi ekstrem seperti hujan deras, kabut tebal, atau lingkungan tanpa pencahayaan tidak termasuk dalam ruang lingkup penelitian.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Merancang tongkat pintar berbasis sensor LiDAR VL53L1X yang mampu mendeteksi hambatan di depan pengguna secara *real-time* dan memberikan umpan balik dalam bentuk getaran.
2. Mengembangkan aplikasi Android yang terintegrasi dengan kamera *smartphone* dan algoritma MobileNet SSD untuk mendeteksi objek visual secara *real-time*.
3. Mengimplementasikan modul navigasi berbasis API OSRM untuk membimbing pengguna menuju tujuan tertentu menggunakan notifikasi suara.
4. Mengintegrasikan sistem perintah suara (*Voice Command*) yang didukung tombol volume *smartphone* sebagai alternatif kendali agar pengguna tunanetra dapat mengoperasikan aplikasi tanpa bergantung pada tampilan visual.

1.5 Manfaat Penelitian

Adapun manfaat yang diharapkan dari penelitian ini adalah:

1. Manfaat akademis: Menjadi referensi ilmiah dalam pengembangan sistem bantu navigasi untuk penyandang disabilitas, khususnya tunanetra, dengan pendekatan berbasis *Internet of Things* (IoT), visi komputer, dan pemrosesan suara.
2. Manfaat praktis: Memberikan solusi mobilitas yang efektif, adaptif, dan terjangkau bagi penyandang tunanetra untuk menjalani aktivitas harian secara lebih aman, mandiri, dan efisien.
3. Manfaat sosial: Berkontribusi dalam peningkatan kualitas hidup penyandang tunanetra melalui pemanfaatan teknologi asistif yang responsif dan user-friendly.
4. Manfaat teknologi: Mendorong inovasi integratif antara teknologi sensor, sistem navigasi, dan komunikasi berbasis suara dalam satu sistem bantu yang portabel dan hemat energi.

1.6 Luaran Penelitian

Luaran dari penelitian ini adalah:

1. Sebuah prototipe tongkat pintar berbasis sensor LiDAR VL53L1X yang mampu mendeteksi hambatan di lingkungan sekitar pengguna.
2. Aplikasi Android terintegrasi yang dapat mendeteksi objek, memberikan arah navigasi berbasis API OSRM, memberikan notifikasi suara *real-time* , serta terhubung ke tongkat melalui Bluetooth.
3. Laporan skripsi yang dapat digunakan sebagai dasar pengembangan lebih lanjut dalam bidang teknologi untuk disabilitas.
4. Hasil penelitian ini akan dipublikasikan pada Jurnal Jartel Politeknik Negeri Malang dengan ISSN (print): 2407-0807,ISSN (online):2654-653.

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terdahulu

Solikh, Sairi (2023) mengembangkan sistem tongkat pintar yang memanfaatkan sensor LiDAR untuk mengukur jarak objek dan menerapkan logika fuzzy guna menentukan tingkat getaran berdasarkan kedekatan hambatan. Serta menggunakan modul gps pemantauan lokasi pengguna . Namun masih terbatas untuk deteksi objek nya tanpa menggunakan kamera[13].

Putra et al. (2023) mengembangkan tongkat pintar berbasis Artificial Intelligence menggunakan YOLOv7 untuk deteksi objek visual. Sistem ini mampu mengenali objek seperti mobil atau tiang, tetapi sangat bergantung pada kamera dan pencahayaan yang baik serta tidak efisien dalam konsumsi daya[6]. Sistem usulan menggunakan model algoritma AI model MobileNet SSD dalam format TensorFlow Lite[6].

Utomo et al. (2023) merancang tongkat pintar untuk anak tunanetra dengan sensor ultrasonik dan GPS, serta fitur pelacakan melalui pesan SMS. Meskipun mampu melacak lokasi, sistem ini tidak memiliki antarmuka visual dan audio navigasi yang intuitif [7]. Sistem yang dikembangkan akan menyertakan Maps dan notifikasi audio untuk mendukung navigasi [7].

Lubis et al. (2025) mengembangkan tongkat dengan sensor ultrasonik dan GPS, serta notifikasi suara sederhana. Sistem ini sudah terhubung ke aplikasi Android, namun deteksi objek masih terbatas secara visual [9]. Sistem yang diusulkan dalam penelitian ini mengintegrasikan deteksi objek melalui kamera Android menggunakan MobileNet SSD, serta menambahkan fitur kendali melalui tombol *smartphone* dan perintah suara untuk memudahkan pengguna tunanetra[9].

Tabel 2. 1 Penelitian Terdahulu

Penelitian Terdahulu	GPS	Sensor Jarak	Deteksi Objek dengan Kamera	Output Suara	Notifikasi Getar	Algoritma AI
<i>Empowering Blind People Mobility: A Millimeter- Wave Radar Cane (2020)</i>		Radar mmWave		✓	✓	
<i>Android-based Object Recognition Application for the Visually Impaired (2021)</i>			✓	✓		SSD (TFLite)
<i>Real-time Object Detection With Speech Output (2022)</i>			✓	✓		MobileNe SSD
<i>Alat Bantu Jalan Tunanetra Menggunakan Sensor Lidar(2023)</i>		Lidar		✓	✓	
<i>Smart Stick Tunanetra Terintegrasi</i>	✓	Ultrasonik		✓	✓	

<i>Smartphone (2023)</i>						
<i>Alat Bantu Tuna Netra Berbasis Arduino Uno dan Artificial Intelligence (2023)</i>	✓		✓	✓		YOLO v7
<i>Tongkat Perekam Jejak untuk Tunanetra Berbasis Mikrokontroler (2023)</i>	✓			✓	✓	
<i>Alat Bantu Navigasi Tunanetra Menggunakan Esp Cam Dan Sensor Tof Dengan Metode Image Processing (2024)</i>		ToF	✓	✓		SSD
<i>Tongkat Tunanetra Menggunakan Sensor</i>	✓	Ultrasonik		✓	✓	

<i>Ultrasonik dan GPS (2025)</i>						
<i>Sistem Usulan (2025)</i>	✓	LiDAR VL53L1X	✓	✓	✓	MobileNet SSD

Seperti pada tabel 2.1 dapat dilihat pengembangan sistem tongkat pintar untuk tunanetra yang diusulkan dalam penelitian ini memiliki beberapa pengembangan yang membedakannya dari penelitian-penelitian terdahulu. Sistem ini mengintegrasikan sensor LiDAR VL53L1X sebagai deteksi jarak hambatan, serta memanfaatkan kamera *smartphone* dengan algoritma MobileNet SSD untuk mengenali objek visual secara *real-time*, yang belum diimplementasikan secara komprehensif pada penelitian sebelumnya. Selain itu, sistem ini didukung oleh fitur perintah suara (*Voice Command*), notifikasi audio berbasis *Text-to-Speech*, serta sinkronisasi data ke Firebase yang memungkinkan keluarga pengguna memantau posisi dan riwayat log peristiwa secara langsung. Selain membantu proses navigasi, sistem juga dilengkapi fitur pemantauan lokasi pengguna melalui aplikasi sehingga informasi posisi pengguna dapat diakses oleh pihak keluarga. Fitur ini menjadi pengembangan dari penelitian sebelumnya yang umumnya hanya berfokus pada deteksi rintangan atau navigasi lokal.

2.2 Kajian Teori

2.2.1 Tunanetra dan Teknologi Bantu Navigasi

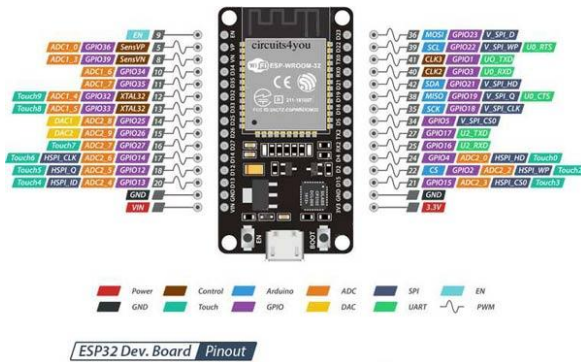


Gambar 2. 1 Ilustasi Tunanetra

Menurut data terbaru dari *World Health Organization* (WHO) dalam *World Report on Vision* tahun 2019, lebih dari 2,2 miliar orang di seluruh dunia mengalami gangguan penglihatan atau kebutaan, dan sekitar 1 miliar kasus di antaranya dapat dicegah atau belum ditangani[2]. Secara khusus, Indonesia menempati peringkat ketiga di dunia dengan jumlah penyandang tunanetra terbanyak setelah India dan China[3]. Data Kementerian Kesehatan Republik Indonesia menunjukkan bahwa angka ini setara dengan sekitar 1,5 persen dari total populasi, atau diperkirakan mencapai 4 juta jiwa dari lebih dari 270 juta penduduk Indonesia.

Penyandang tunanetra merupakan individu yang mengalami gangguan penglihatan, baik secara total maupun sebagian, sehingga kemampuan mobilitas dan orientasi ruangnya menjadi terbatas[4]. Untuk membantu aktivitas sehari-hari, penyandang tunanetra umumnya menggunakan tongkat putih konvensional sebagai alat bantu utama dalam bernavigasi. Namun, alat ini memiliki keterbatasan karena hanya mampu mendeteksi hambatan yang bersentuhan langsung dan belum dapat memberikan peringatan dini terhadap objek di kejauhan maupun informasi arah yang akurat [1], [6]. Seiring dengan perkembangan teknologi, berbagai alat bantu navigasi modern berbasis sensor dan kecerdasan buatan (AI) mulai dikembangkan untuk meningkatkan kemandirian serta keselamatan penyandang tunanetra dalam beraktivitas[1], [6].

2.2.2 Mikrokontroler ESP32

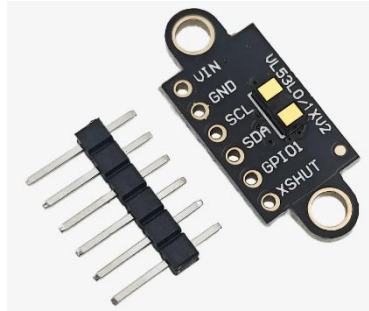


Gambar 2. 2 ESP32

ESP32 DevKit V1 merupakan papan pengembangan (development board) yang menggunakan modul ESP32-WROOM-32 berbasis mikrokontroler ESP32 System-on-Chip (SoC) yang dikembangkan oleh Espressif Systems. Board ini mendukung komunikasi Wi-Fi IEEE 802.11 b/g/n dan Bluetooth, sehingga mampu melakukan pertukaran data secara nirkabel dengan perangkat lain tanpa memerlukan modul komunikasi tambahan. ESP32 DevKit V1 menggunakan prosesor Tensilica Xtensa LX6 dengan kecepatan hingga 240 MHz, memiliki memori Flash 4 MB dan SRAM 520 KB, serta bekerja pada tegangan operasi 3,3 V. Selain itu, board ini menyediakan 25 pin digital (GPIO), 6 kanal Analog-to-Digital Converter (ADC), 2 kanal Digital-to-Analog Converter (DAC), 3 antarmuka UART, 2 antarmuka SPI, dan 3 antarmuka I2C, sehingga dapat dihubungkan dengan berbagai sensor, aktuator, maupun perangkat eksternal lainnya sesuai kebutuhan aplikasi.[10]

Pada penelitian ini, ESP32 DevKit V1 digunakan sebagai pengendali utama pada perangkat keras tongkat pintar. ESP32 menerima data hasil pengukuran dari sensor LiDAR VL53L1X melalui komunikasi I2C, kemudian mengolah data tersebut untuk menentukan pola peringatan yang diberikan kepada pengguna melalui motor getar. Selanjutnya, data hasil pengukuran jarak dikirimkan ke aplikasi Android menggunakan komunikasi Bluetooth, sehingga dapat dipadukan dengan hasil deteksi objek berbasis SSD MobileNet V2, informasi navigasi, dan keluaran suara (Text-to-Speech). Dengan demikian, ESP32 berperan sebagai pusat kendali yang menghubungkan sensor, aktuator, dan aplikasi Android agar sistem tongkat pintar dapat bekerja secara terintegrasi.

2.2.3 Sensor LiDAR VL53L1X



Gambar 2. 3 Sensor LiDAR VL53L1X

VL53L1X merupakan sensor pengukur jarak berbasis teknologi *Time of Flight* (ToF) yang dikembangkan oleh STMicroelectronics. Sensor ini bekerja dengan memancarkan cahaya inframerah berpanjang gelombang 940 nm ke arah objek, kemudian mengukur waktu yang dibutuhkan cahaya untuk dipantulkan kembali ke sensor. Berdasarkan waktu tempuh tersebut, sensor dapat menghitung jarak objek dengan resolusi pengukuran hingga 1 mm. VL53L1X memiliki kemampuan pengukuran hingga sekitar 4 meter pada mode *Long Distance* dan menggunakan antarmuka komunikasi I²C melalui pin SDA dan SCL. Sensor ini juga dilengkapi dengan pin GPIO1 sebagai keluaran interupsi dan pin XSHUT untuk fungsi shutdown atau reset. Sensor ini mempunyai sudut deteksi (*Field of View*) sekitar 27°, sehingga sesuai digunakan pada aplikasi deteksi rintangan, robotika, sistem navigasi, dan perangkat *Internet of Things* (IoT).[14]

Pada penelitian ini, sensor LiDAR VL53L1X digunakan untuk mengukur jarak antara tongkat pintar dengan objek atau hambatan yang berada di depan pengguna bukan objek di kiri dan kanan. Informasi jarak yang diperoleh digunakan sebagai dasar pemberian peringatan melalui motor getar pada tongkat serta dikirimkan ke aplikasi Android melalui mikrokontroler ESP32 menggunakan komunikasi I²C. Selanjutnya, aplikasi menyampaikan informasi tersebut kepada pengguna melalui fitur *Text-to-Speech* (TTS) sehingga pengguna dapat mengetahui keberadaan dan jarak hambatan secara *real-time*.

Prinsip pengukuran jarak pada sensor *Time of Flight* (ToF) dinyatakan menggunakan Persamaan (2.1).

$$d = \frac{c \times t}{2}$$

Keterangan :

- d = jarak objek (m)
- c = kecepatan cahaya
- t = waktu tempuh sinar laser (s)

Pembagian dengan dua dilakukan karena waktu yang diukur merupakan waktu perjalanan sinar laser dari sensor menuju objek dan kembali lagi ke sensor. Semakin singkat waktu pantulan yang diterima, maka semakin dekat posisi objek terhadap sensor, sedangkan semakin lama waktu pantulan menunjukkan objek berada pada jarak yang lebih jauh. Berdasarkan prinsip tersebut, sensor VL53L1X mampu menghasilkan pengukuran jarak hingga sekitar 4 meter, sehingga sesuai digunakan sebagai sensor pendeteksi hambatan pada sistem tongkat pintar bagi penyandang tunanetra.[14]

2.2.4 Motor Getar



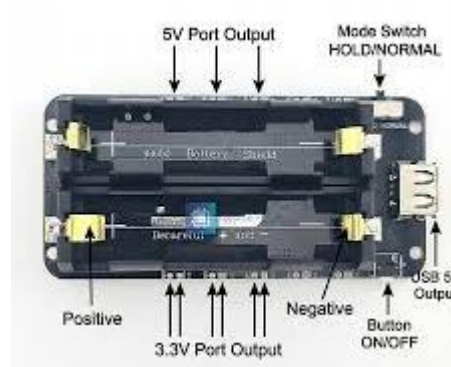
Gambar 2. 4 Vibration Motor Module

Vibration Motor Module merupakan modul aktuator yang menggunakan motor getar tipe coin (*coin Vibration Motor*) sebagai sumber getaran. Motor ini bekerja berdasarkan prinsip *Eccentric Rotating Mass* (ERM), yaitu menghasilkan getaran melalui putaran massa yang tidak seimbang pada rotor. Modul ini umumnya beroperasi pada tegangan 3–5 V dan telah dilengkapi rangkaian driver sehingga dapat dihubungkan langsung dengan mikrokontroler seperti ESP32 melalui pin kendali digital. Bentuknya yang ringkas dan konsumsi dayanya yang relatif rendah membuat modul ini sesuai digunakan pada perangkat portabel.[15]

Pada penelitian ini, *Vibration Motor Module* digunakan sebagai media umpan balik taktil (*haptic feedback*) untuk memberikan peringatan kepada pengguna mengenai keberadaan rintangan di sekitar. Ketika sistem mendeteksi

objek pada jarak tertentu, ESP32 akan mengaktifkan motor getar sehingga pengguna dapat menerima informasi secara langsung melalui getaran. Penggunaan umpan balik taktil ini membantu pengguna tunanetra memperoleh informasi lingkungan tanpa bergantung sepenuhnya pada notifikasi suara.

2.2.5 18650 Battery Shield V3



Gambar 2. 5 18650 Battery Shield V3

18650 *Battery Shield V3* merupakan modul manajemen daya yang dirancang untuk menggunakan baterai lithium-ion tipe 18650 sebagai sumber energi. Modul ini memiliki variasi satu sampai empat slot baterai yang dapat digunakan untuk meningkatkan kapasitas penyimpanan energi sehingga waktu operasional perangkat menjadi lebih lama. Selain berfungsi sebagai tempat baterai, modul ini juga dilengkapi dengan rangkaian pengisian daya (*charging circuit*), proteksi baterai, dan konverter tegangan (*boost converter*) yang mampu menghasilkan tegangan keluaran stabil sebesar 5 V dan 3.3 V. Fitur tersebut memungkinkan modul digunakan sebagai sumber daya portabel untuk berbagai aplikasi elektronika berbasis mikrokontroler.[16]

Pada penelitian ini, 18650 Lithium *Battery Shield V3* digunakan sebagai sumber daya utama untuk mensuplai kebutuhan daya sistem tongkat pintar yang terdiri dari ESP32, sensor LiDAR VL53L1X, motor getar, dan komponen pendukung lainnya. Modul ini dipilih karena memiliki kapasitas energi yang cukup besar, dapat diisi ulang (*rechargeable*), serta menyediakan keluaran tegangan yang stabil sehingga mampu mendukung kinerja sistem selama proses pengoperasian. Selain itu, adanya fitur proteksi terhadap *overcharge*, *overdischarge*, dan *short circuit* dapat meningkatkan keamanan penggunaan baterai serta memperpanjang umur pakai sistem.

2.2.6 Notifikasi Audio



Gambar 2. 6 Notifikasi Suara dari Aplikasi Android

Notifikasi suara merupakan metode interaksi utama dalam sistem bantu tunanetra karena memungkinkan penyampaian informasi secara langsung tanpa ketergantungan pada tampilan visual. Bagi penyandang tunanetra, indera pendengaran menjadi jalur sensorik utama untuk memahami kondisi lingkungan. Oleh karena itu, sistem tongkat pintar yang dikembangkan memanfaatkan teknologi *Text-to-Speech* (TTS) yang tergabung dengan aplikasi pada Android sebagai sarana untuk menyampaikan informasi secara *real-time*. TTS berfungsi mengubah teks menjadi suara secara dinamis, sehingga sistem dapat menginformasikan arah navigasi, peringatan rintangan, maupun status sistem dengan fleksibel dan jelas, misalnya dengan mengucapkan “Belok kanan dalam 100 meter” atau “Ada penghalang di depan sejauh 50 cm.”[9].

Secara teknis, integrasi TTS di aplikasi Android melibatkan inisialisasi *engine* TTS bawaan *TextToSpeech* dan mengaturnya ke Bahasa Indonesia. Di sisi lain, aplikasi secara konstan mendengarkan data peringatan yang dikirimkan oleh mikrokontroler ESP32 melalui *Bluetooth Low Energy* (BLE). Ketika ESP32 mengirimkan data mentah misalnya, "D:50" untuk jarak 50 cm, aplikasi Android akan menerima data tersebut, mengubahnya menjadi kalimat lengkap seperti "Ada penghalang di depan sejauh 50 cm", dan kemudian memanggil fungsi *.speak()* untuk memerintahkan *speaker* ponsel mengucapkannya.

2.2.7 Aplikasi Mobile



Gambar 2. 7 Aplikasi Mobile

Aplikasi *mobile* digunakan sebagai pusat pengolahan data dan tampilan utama bagi pengguna. Aplikasi ini dibuat menggunakan Android Studio dan berfungsi untuk mengambil serta mengolah data secara langsung dari beberapa sumber data yang diterima berasal dari ESP32 yang mengirimkan informasi jarak dan lokasi melalui Bluetooth, serta dari kamera ponsel untuk mendeteksi objek. Data visual dari kamera ini diproses menggunakan metode AI MobileNet SSD dan TensorFlow Lite agar bisa mengenali nama objek, seperti orang, mobil, sepeda motor, pohon, bangku, sepeda. Setelah semua data jarak, lokasi, dan nama objek terkumpul, aplikasi ini menggabungkannya. Data GPS dipakai untuk Maps sebagai navigasi dan pelacakan, sementara semua informasi peringatan seperti "Awas, ada 'mobil' 3 meter di depan" akan diubah menjadi suara menggunakan *Text-to-Speech* (TTS).[17], [18].

2.2.8 Open Source Routing Machine (OSRM)



Gambar 2. 8 API OSRM

Open Source Routing Machine (OSRM) merupakan layanan *routing engine* berbasis data *OpenStreetMap* (OSM) yang digunakan untuk menghitung rute dan memberikan informasi navigasi pada berbagai aplikasi berbasis lokasi. OSRM menyediakan *Application Programming Interface*

(API) yang memungkinkan pengembang memperoleh rute perjalanan berdasarkan koordinat lokasi awal dan tujuan. Dalam penelitian ini, OSRM digunakan untuk mendukung fitur navigasi pejalan kaki bagi penyandang tunanetra serta membantu memberikan arahan perjalanan melalui aplikasi Android.[19]

Untuk fungsi navigasi, aplikasi mengirimkan permintaan (*request*) ke API OSRM yang berisi koordinat lokasi pengguna sebagai titik awal dan koordinat tujuan yang dipilih. Selanjutnya, server OSRM memproses data tersebut dan menghasilkan rute perjalanan berdasarkan jaringan jalan yang tersedia pada *OpenStreetMap* dengan mode perjalanan *walking*. Hasil yang diterima aplikasi berupa data berformat JSON yang berisi informasi rute, jarak tempuh, estimasi waktu perjalanan, serta instruksi navigasi. Data tersebut kemudian diolah oleh aplikasi dan disampaikan kepada pengguna dalam bentuk suara menggunakan fitur *Text-to-Speech* (TTS).[19]

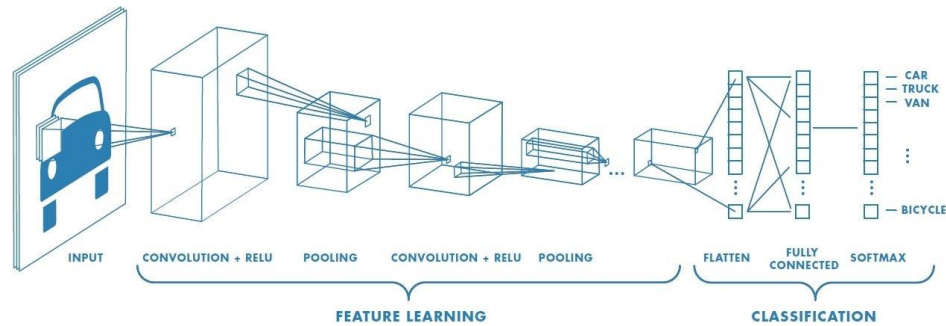
Selain digunakan untuk navigasi, aplikasi juga memanfaatkan koneksi internet untuk mengirimkan data lokasi pengguna ke Firebase Realtime Database. Data lokasi yang diperoleh dari *smartphone* kemudian disinkronkan secara berkala sehingga dapat diakses oleh keluarga atau pengawas melalui aplikasi pemantauan. Dengan demikian, sistem tidak hanya membantu pengguna dalam menentukan rute perjalanan, tetapi juga menyediakan informasi lokasi yang dapat dipantau dari jarak jauh.

2.2.9 *Convolutional Neural Network (CNN)*

Convolutional Neural Network (CNN) adalah salah satu arsitektur *Deep learning* yang paling sering digunakan dalam domain *Computer Vision*. CNN merupakan pengembangan dari Jaringan Saraf Tiruan yang arsitekturnya terinspirasi oleh cara korteks visual manusia memproses gambar. CNN memiliki kemampuan untuk mempelajari dan mengekstraksi fitur (fitur) dari gambar secara otomatis dan hierarkis. CNN tidak memerlukan ekstraksi fitur manual sebaliknya, model ini belajar sendiri fitur-fitur apa yang penting mulai

dari yang sederhana seperti garis atau tepi hingga yang kompleks seperti bentuk, tekstur, atau bagian objek[17]

Arsitektur CNN pada dasarnya terdiri dari beberapa jenis lapisan (layer) yang bekerja secara berurutan untuk mengubah gambar input menjadi output (prediksi). Berikut merupakan tahap deteksi pada arsitektur CNN:



Gambar 2. 9 Arsitektur CNN

1. Tahap Pembelajaran Fitur (Feature Learning)

Tahap pertama ini adalah inti dari CNN, yang bertugas mengekstrak fitur atau pola penting dari gambar secara otomatis. Proses ini dimulai dengan Lapisan Konvolusi (Convolution + ReLU), di mana serangkaian "filter" digital memindai gambar untuk mendeteksi fitur-fitur dasar seperti garis tepi, sudut, dan tekstur. Hasilnya, yang disebut "Peta Fitur", kemudian dimasukkan ke Lapisan Pooling (biasanya Max Pooling). Lapisan *pooling* ini berfungsi untuk meringkas dan memperkecil ukuran peta fitur, yang bertujuan mengurangi beban komputasi dan membuat deteksi fitur lebih tangguh terhadap variasi posisi. Seperti yang terlihat di diagram, blok proses Konvolusi dan Pooling ini diulang beberapa kali, memungkinkan model untuk mempelajari fitur yang semakin kompleks di setiap lapisan.

2. Tahap Klasifikasi (*Classification*)

Setelah semua fitur penting diekstraksi, tahap kedua mengambil alih untuk membuat keputusan akhir. Pertama, Peta Fitur terakhir dari tahap sebelumnya akan "diratakan" oleh lapisan Flatten, mengubah data matriks 2D/3D menjadi satu vektor 1D yang panjang. Vektor fitur ini kemudian dimasukkan ke Lapisan

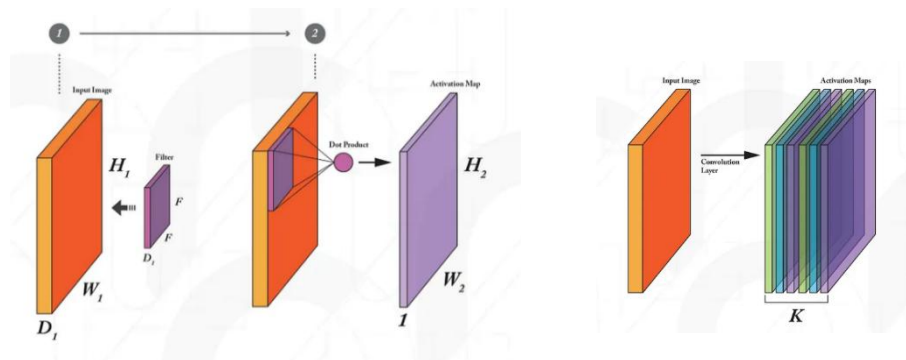
Fully Connected, yang merupakan jaringan saraf tiruan standar. Lapisan ini menganalisis kombinasi fitur-fitur tingkat tinggi tersebut (misalnya, "roda" + "jendela" = "mobil") untuk menghitung skor probabilitas. Terakhir, lapisan Softmax mengambil skor tersebut dan mengubahnya menjadi persentase yang mudah dipahami (total 100%), yang menunjukkan seberapa yakin model tersebut bahwa gambar input termasuk dalam kelas tertentu, seperti "CAR", "TRUCK", atau "BICYCLE".

Secara umum, arsitektur CNN modern terdiri dari tujuh macam lapisan (layer) utama yang memiliki fungsi berbeda-beda.

1. Lapisan Input (Input Layer)

Lapisan Input adalah "pintu masuk" pertama tempat data gambar mentah dimasukkan ke dalam jaringan. Lapisan ini tidak melakukan pemrosesan, melainkan hanya mendefinisikan format dan ukuran data yang akan diterima oleh model. Data ini direpresentasikan sebagai matriks angka (piksel) dengan dimensi spesifik: [Tinggi] x [Lebar] x [Kanal Warna].

2. Lapisan Konvolusi (Convolutional Layer)



Gambar 2. 10 Convolutional Layer

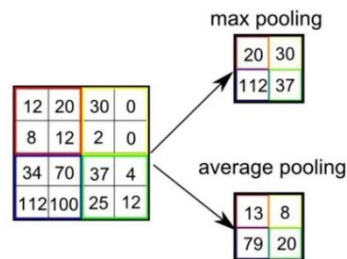
Ini adalah lapisan inti dan paling fundamental dalam CNN, yang bertugas sebagai pengekstrak fitur (feature extractor). Lapisan ini bekerja dengan menggeser (mengkonvolusi) satu atau beberapa "filter" (atau "kernel") matriks kecil berisi angka ke seluruh permukaan gambar input. Setiap filter dirancang untuk mendeteksi fitur spesifik, seperti garis tepi vertikal, tekstur, atau lengkungan. Saat filter bergeser, ia melakukan operasi matematis (dot product)

dengan piksel di bawahnya untuk menghasilkan sebuah "Peta Fitur" (Feature Map), yang merupakan "peta" yang menyoroti di mana saja fitur tersebut terdeteksi di dalam gambar.

3. Lapisan Aktivasi (Activation Layer - ReLU)

Lapisan Aktivasi diterapkan segera setelah Lapisan Konvolusi untuk mengenalkan non-linearitas ke dalam model. Fungsi yang paling umum digunakan adalah Rectified Linear Unit (ReLU). Cara kerjanya sangat sederhana: ia akan mengubah semua nilai negatif yang diterimanya menjadi nol, sementara semua nilai positif akan dibiarkan apa adanya. Penambahan non-linearitas ini sangat krusial, karena tanpanya, seluruh jaringan CNN yang hanya terdiri dari operasi linier seperti konvolusi hanya akan bisa mempelajari pola-pola yang sangat sederhana (linier). ReLU memungkinkan model untuk mempelajari dan merepresentasikan pola yang jauh lebih kompleks dan rumit, seperti bentuk bodi mobil atau wajah manusia.

5. Lapisan Pooling (Pooling Layer)



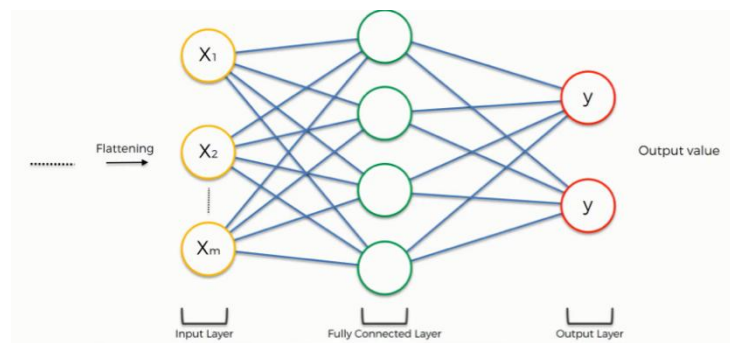
Gambar 2. 11 Pooling Layer

Setelah fitur diekstraksi oleh lapisan konvolusi, Lapisan Pooling bertugas untuk meringkas dan mengurangi ukuran (dimensi) dari Peta Fitur. Metode yang paling umum adalah Max Pooling. Lapisan ini bekerja dengan mengambil sebuah jendela kecil dan menggesernya melintasi Peta Fitur, di mana ia hanya akan mengambil nilai piksel tertinggi dari setiap jendela tersebut. Proses perampingan (down-sampling) ini memiliki dua manfaat utama pertama, mengurangi beban komputasi secara signifikan untuk lapisan berikutnya, dan kedua, membuat model lebih tangguh (robust) terhadap variasi posisi fitur (dikenal sebagai *spatial invariance*).

5. Lapisan Flatten (Flatten Layer)

Lapisan Flatten adalah lapisan transisi yang sederhana namun penting, yang berfungsi untuk mengubah format data. Lapisan ini mengambil output dari lapisan pooling terakhir (yang masih berupa matriks 2D atau 3D) dan "meratakannya" atau "memipihkannya" menjadi satu vektor 1D (satu larik angka yang sangat panjang). Hal ini mutlak diperlukan karena data fitur yang telah diekstraksi dari tahap pembelajaran fitur harus diubah formatnya agar kompatibel dengan input yang dibutuhkan oleh Lapisan *Fully Connected*, yang hanya dapat menerima data dalam bentuk vektor 1D.

6. Lapisan Fully Connected (Fully Connected Layer)



Gambar 2. 12 Fully Connected Layer

Lapisan ini adalah otak pembuat keputusan dari CNN. Ini adalah Jaringan Saraf Tiruan standar di mana setiap neuron di lapisan ini terhubung ke setiap nilai neuron dari lapisan Flatten sebelumnya. Setelah menerima vektor fitur yang panjang, lapisan inilah yang bertugas menganalisis kombinasi dari fitur-fitur tingkat tinggi tersebut. Misalnya, lapisan ini belajar bahwa jika ada fitur roda dan fitur jendela dan fitur pintu yang terdeteksi, maka probabilitasnya tinggi bahwa objek tersebut adalah mobil. Mungkin ada beberapa lapisan *Fully Connected* yang ditumpuk untuk analisis yang lebih mendalam.

7. Lapisan Output (Output Layer)

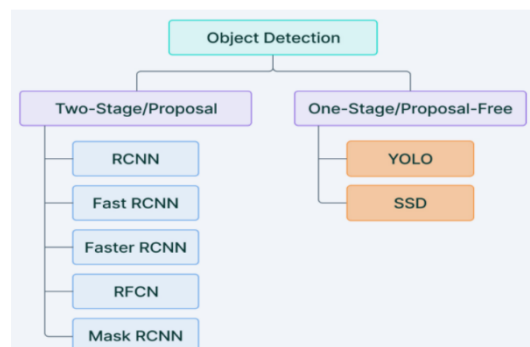
Lapisan Output adalah lapisan *Fully Connected* terakhir yang memberikan prediksi akhir dari model. Jumlah neuron di lapisan ini sama persis dengan

jumlah kelas yang ingin dideteksi oleh model (misalnya, 6 neuron untuk 6 kelas objek Anda). Lapisan ini sering menggunakan fungsi aktivasi Softmax, yang mengambil skor mentah dari lapisan sebelumnya dan mengubahnya menjadi distribusi probabilitas (persentase). Outputnya adalah daftar probabilitas di mana totalnya adalah 100%, misalnya: {Orang: 5%, Mobil: 90%, Sepeda: 3%, ...}. Model kemudian akan memilih kelas dengan probabilitas tertinggi ("Mobil") sebagai hasil prediksinya.

Dalam penelitian ini, arsitektur CNN tidak hanya digunakan untuk *klasifikasi*, tetapi untuk Deteksi Objek. Ini adalah tugas yang lebih kompleks yang bertujuan untuk mengidentifikasi dan menemukan lokasi satu atau lebih objek dalam sebuah gambar. Hasil dari deteksi objek bukanlah satu label, melainkan serangkaian parameter keluaran yang meliputi:

- Kotak Batas (*Bounding box*): Koordinat (misal: $y_{\min}$, $x_{\min}$, $y_{\max}$, $x_{\max}$) yang menentukan lokasi objek.
- Kelas Objek (*Class*): Label atau nama dari objek yang terdeteksi.
- Skor Kepercayaan (*Confidence Score*): Nilai probabilitas (0-100%) yang menunjukkan seberapa yakin model terhadap deteksinya.

Arsitektur CNN untuk deteksi objek sangat beragam, namun dapat dikategorikan berdasarkan keseimbangan antara akurasi dan kecepatan. Kebutuhan utama proyek ini adalah sistem yang dapat berjalan secara *real-time* pada perangkat *mobile* (Android) dengan sumber daya terbatas[11]



Gambar 2. 13 Macam-macam CNN

1. Detektor Dua Tahap (Two-Stage Detectors)

- Contoh: Faster R-CNN.
- Cara Kerja: Arsitektur ini bekerja dalam dua tahap: pertama mengusulkan wilayah yang mungkin berisi objek, lalu mengklasifikasikan setiap wilayah tersebut.
- Kelemahan: Meskipun sangat akurat, proses dua tahap ini sangat lambat dan berat secara komputasi, sehingga tidak cocok untuk inferensi *real-time*.

2. Detektor Satu Tahap (One-Stage Detectors)

- Contoh: SSD (Single Shot Detector), YOLO.
- Cara Kerja: Arsitektur ini jauh lebih efisien. Model ini membuang tahap proposal dan mampu memprediksi lokasi (*bounding box*) dan kelas (*class*) secara bersamaan hanya dalam satu kali proses (single shot).
- Kelebihan: Sangat cepat dan ideal untuk aplikasi *real-time*.

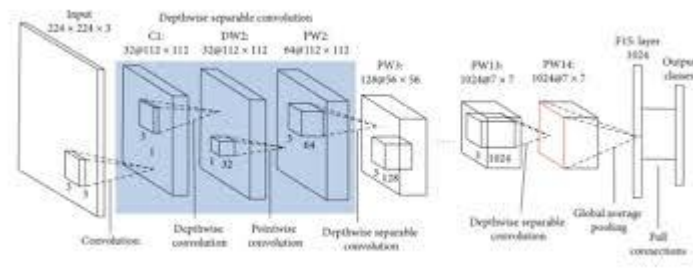
Selain *detector*, arsitektur *backbone* (pengekstrak fitur) juga penting. Arsitektur berat seperti ResNet atau VGG16 memiliki akurasi tinggi namun lambat. Untuk aplikasi *mobile*, diperlukan *backbone* yang ringan.

MobileNet adalah arsitektur *backbone* CNN yang dirancang khusus untuk perangkat *mobile*. Ia menggunakan teknik inovatif (*depthwise separable convolutions*) untuk mengurangi jumlah parameter dan komputasi secara drastis, sehingga menghasilkan model yang sangat cepat dan ringan. Oleh karena itu, model MobileNet SSD dipilih untuk penelitian ini.

2.2.10 Mobile Net SSD

MobileNet adalah arsitektur CNN yang ringan dan dioptimalkan untuk dijalankan pada perangkat *mobile* atau *embedded*. Model ini menggunakan teknik *depthwise separable convolution* untuk mengurangi jumlah parameter dan komputasi, tanpa kehilangan akurasi secara signifikan. *Single Shot MultiBox Detector* (SSD) adalah algoritma deteksi objek yang bekerja dengan mendeteksi banyak *Bounding box* dan label sekaligus dalam satu inferensi.

Kombinasi MobileNet dan SSD menghasilkan model deteksi objek *real-time* yang efisien dan cocok untuk aplikasi seperti sistem bantu tunanetra berbasis Android.[11], [20]



Gambar 2. 14 arsitektur mobilenetv2

Arsitektur ini mengambil input gambar standar (misalnya, 224 x 224 x 3).

Setelah lapisan konvolusi standar (C1) di awal, arsitektur ini menggunakan blok *depthwise separable convolution* (ditandai pada area biru) secara berulang. Berbeda dengan konvolusi standar yang memproses semua *channel* input sekaligus, teknik ini memecah operasi menjadi dua langkah:

1. **Depthwise Convolution (DW2)**

Lapisan ini menerapkan satu filter unik untuk setiap *channel* (saluran) input secara terpisah. Jika input memiliki 32 *channel* (seperti pada DW2), lapisan ini hanya menggunakan 32 filter untuk memproses informasi spasial (lebar dan tinggi) dari masing-masing *channel* tanpa menggabungkannya. Proses ini sangat ringan secara komputasi.

2. **Pointwise Convolution (PW2)**

Lapisan ini menggunakan filter berukuran 1 x1 untuk mencampur atau menggabungkan informasi dari 32 *channel* yang dihasilkan oleh langkah *depthwise*. Lapisan inilah yang bertugas untuk mengubah kedalaman (jumlah *channel*), misalnya dari 32 menjadi 64.

Dengan memisahkan proses ekstraksi fitur spasial (langkah 1) dan kombinasi fitur *channel* (langkah 2), jumlah total parameter dan perhitungan dapat ditekan secara dramatis. Seperti terlihat pada diagram, arsitektur ini mengulangi blok tersebut (PW3...PW13) sambil mengurangi ukuran spasial (dari 112 x112 ke 56 x 56, dst.). Arsitektur ini kemudian diakhiri dengan

lapisan Global Average Pooling metode efisien untuk merangkum fitur dan Full Connections untuk klasifikasi akhir.[17]

Dalam penelitian ini, arsitektur MobileNet yang efisien tersebut digunakan sebagai backbone (tulang punggung)—yaitu sebagai pengekstrak fitur—untuk model Deteksi Objek (Object Detection). *Backbone* MobileNet ini kemudian digabungkan dengan Single Shot MultiBox Detector (SSD). SSD adalah algoritma deteksi objek *satu tahap (one-stage)* yang sangat cepat. Disebut "Single Shot" karena ia mampu mendeteksi lokasi (*bounding box*) dan label (nama kelas) dari semua objek dalam gambar hanya dalam satu kali inferensi (satu *pass*).

2.2.11 Evaluasi Performa Model Deteksi Objek

Evaluasi performa model deteksi objek dilakukan untuk mengetahui kemampuan model dalam mengenali objek sesuai dengan kelas yang telah ditentukan. Pada penelitian ini, evaluasi model dilakukan menggunakan beberapa metrik, yaitu *Accuracy*, *Precision*, *Recall*, *Average Precision* (AP), dan *Mean Average Precision* (mAP). Metrik-metrik tersebut digunakan untuk menilai ketepatan prediksi model serta kualitas hasil deteksi objek.

1. *Accuracy*

Accuracy merupakan metrik yang digunakan untuk mengetahui tingkat ketepatan prediksi model secara keseluruhan. Nilai *Accuracy* menunjukkan perbandingan antara jumlah prediksi yang benar terhadap seluruh data yang diuji. Semakin tinggi nilai *Accuracy*, maka semakin baik kemampuan model dalam melakukan prediksi.[21]

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \times 100\% \quad (2.2)$$

Keterangan:

- TP (True Positive) = jumlah objek yang berhasil dideteksi dengan benar.
- TN (True Negative) = jumlah data yang benar-benar bukan objek dan diprediksi bukan objek.
- FP (*False positive*) = jumlah objek yang terdeteksi tetapi sebenarnya bukan objek tersebut.

- FN (*False negative*) = jumlah objek yang seharusnya terdeteksi tetapi tidak berhasil dideteksi.

2. Precision

Precision merupakan metrik yang digunakan untuk mengukur tingkat ketepatan hasil prediksi positif yang dihasilkan oleh model. Nilai *Precision* menunjukkan seberapa banyak prediksi positif yang benar dibandingkan seluruh prediksi positif yang diberikan model. Semakin tinggi nilai *Precision*, semakin kecil kemungkinan terjadinya kesalahan deteksi (*False positive*).[21]

$$Precision = \frac{TP}{TP+FP} \times 100\% \quad (2.3)$$

3. Recall

Recall merupakan metrik yang digunakan untuk mengukur kemampuan model dalam menemukan seluruh objek yang sebenarnya ada pada data. Nilai *Recall* menunjukkan perbandingan antara jumlah objek yang berhasil dideteksi dengan seluruh objek yang seharusnya dapat dideteksi. Semakin tinggi nilai *Recall*, maka semakin sedikit objek yang gagal dideteksi (*False negative*).[21]

$$Recall = \frac{TP}{TP+FN} \times 100\% \quad (2.4)$$

4. F1-Score

F1-Score merupakan metrik evaluasi yang digunakan untuk mengukur keseimbangan antara nilai *Precision* dan *Recall*. Metrik ini digunakan ketika kedua nilai tersebut sama-sama penting dalam mengevaluasi performa model deteksi objek. Nilai *F1-Score* yang tinggi menunjukkan bahwa model memiliki kemampuan yang baik dalam memberikan prediksi yang tepat sekaligus mampu mendeteksi sebagian besar objek yang sebenarnya. Nilai *F1-Score* berada pada rentang 0 hingga 1, di mana nilai yang semakin mendekati 1 menunjukkan performa model yang semakin baik.[21]

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (2.5)$$

2.2.12 DataSheet COCO



Gambar 2. 15 Contoh Dataset COCO

Common Objects in Context (COCO) adalah dataset besar yang digunakan dalam pelatihan model deteksi objek dan segmentasi gambar. Dataset ini menyediakan lebih dari 200.000 gambar dan 80 kategori objek umum, seperti *person*, *car*, *bicycle*, *bus*, *truck*, dan lainnya. COCO dirancang untuk mendukung model deteksi objek dalam konteks dunia nyata, sehingga cocok untuk melatih dan mengevaluasi performa model seperti MobileNet SSD dalam mengenali objek relevan untuk sistem bantu visual.[22]

2.2.13 Android Studio



Gambar 2. 16 Logo Android Studio

Android Studio adalah lingkungan pengembangan terintegrasi (IDE) resmi yang direkomendasikan Google untuk membangun aplikasi Android. Android Studio menyediakan fitur lengkap seperti editor kode, emulator, manajemen proyek, dan integrasi dengan pustaka Android Jetpack serta TensorFlow Lite. Dalam penelitian ini, Android Studio digunakan untuk membangun aplikasi yang mengintegrasikan kamera, deteksi objek dengan TFLite, sistem *Text-to-Speech*, serta navigasi API OSRM.[18]

2.2.14 Firebase



Gambar 2. 17 Logo Firebase

Firebase adalah platform pengembangan aplikasi berbasis cloud yang disediakan oleh Google. Firebase menyediakan berbagai layanan seperti *Realtime Database*, *Authentication*, *Cloud Storage*, dan *Cloud Messaging* yang dapat digunakan secara langsung dalam aplikasi Android . Firebase sangat cocok digunakan untuk aplikasi yang memerlukan sinkronisasi data *real-time* , seperti pencatatan histori lokasi pengguna tunanetra, serta akses pihak keluarga terhadap status dan data terakhir pengguna.[23]

2.2.15 Roboflow



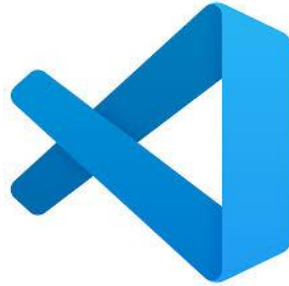
Gambar 2. 18 Logo Roboflow

Roboflow merupakan platform berbasis web yang digunakan untuk mempermudah pengelolaan dataset pada pengembangan aplikasi *computer vision*. Platform ini menyediakan berbagai fitur seperti anotasi gambar, pengaturan kelas objek, augmentasi data, serta ekspor dataset ke berbagai format yang kompatibel dengan *Framework machine learning* dan *deep learning*. Dengan fitur-fitur tersebut, Roboflow membantu mempercepat proses persiapan data sebelum digunakan untuk pelatihan model deteksi objek.[24]

Pada penelitian ini, Roboflow digunakan untuk melakukan anotasi objek pada dataset gambar serta melakukan augmentasi data guna meningkatkan variasi data pelatihan. Setelah proses anotasi dan pengolahan data selesai, dataset diekspor ke format yang sesuai untuk digunakan pada pelatihan model

MobileNet SSD. Penggunaan Roboflow membantu menghasilkan dataset yang lebih terstruktur sehingga mendukung proses pengembangan sistem deteksi objek pada tingkat pintar bagi penyandang tunanetra.

2.2.16 Visual Studio Code (VS Code)



Gambar 2. 19 Logo Visual Studio Code

Visual Studio Code (VS Code) merupakan perangkat lunak source code editor yang dikembangkan oleh Microsoft dan mendukung berbagai bahasa pemrograman, termasuk Python. VS Code menyediakan berbagai fitur seperti *syntax highlighting*, *debugging*, *extension support*, serta integrasi dengan terminal yang memudahkan proses pengembangan dan pengujian perangkat lunak. Karena bersifat ringan dan memiliki dukungan ekstensi yang lengkap, VS Code banyak digunakan dalam pengembangan aplikasi berbasis *machine learning*, *deep learning*, dan *computer vision*. [25]

Pada penelitian ini, VS Code digunakan sebagai lingkungan pengembangan untuk proses pelatihan (*training*) model MobileNet SSD menggunakan bahasa pemrograman Python versi 3.9. Melalui VS Code, proses pengolahan dataset, pelatihan model, evaluasi performa, serta konversi model dilakukan dengan bantuan berbagai pustaka Python seperti TensorFlow, OpenCV, dan NumPy. Penggunaan VS Code mempermudah pengelolaan kode program dan proses pengembangan sistem deteksi objek yang diterapkan pada aplikasi tingkat pintar bagi penyandang tunanetra.

BAB III

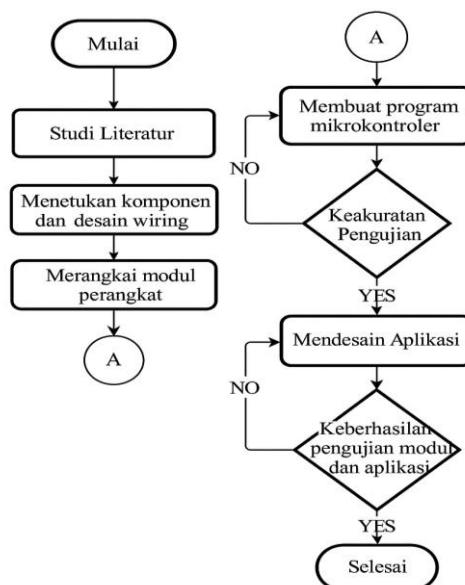
METODOLOGI PENELITIAN

3.1. Jenis Penelitian

Jenis penelitian yang digunakan dalam pengembangan sistem tongkat pintar ini adalah penelitian pengembangan (development research) dengan pendekatan eksperimen dan prototipe. Penelitian ini berfokus pada perancangan, implementasi, dan pengujian sistem perangkat keras dan perangkat lunak yang dikembangkan untuk membantu mobilitas penyandang tunanetra.

Tujuan utama dari penelitian ini adalah menghasilkan produk teknologi asistif berupa tongkat pintar yang mampu mendeteksi hambatan menggunakan sensor LiDAR VL53L1X, melakukan deteksi objek melalui kamera *smartphone* dengan algoritma MobileNet SSD, serta memberikan notifikasi suara melalui aplikasi Android secara *real-time*. Dalam pelaksanaannya, penelitian ini juga melibatkan proses iteratif mulai dari analisis kebutuhan, perancangan sistem, pengembangan prototipe, hingga uji fungsional untuk mengevaluasi kinerja sistem secara keseluruhan.

3.2. Tahapan Penelitian



Gambar 3. 1 Flowchart Alur Tahapan Penelitian

Penjelasan Tahapan Penelitian Berdasarkan

1. Mulai

Tahapan awal dimulai dengan merumuskan tujuan dan ruang lingkup penelitian, serta mengidentifikasi masalah yang ingin diselesaikan, yaitu keterbatasan alat bantu navigasi bagi penyandang tunanetra.

2. Studi Literatur

Peneliti melakukan kajian terhadap jurnal, buku, dan dokumentasi terkait teknologi *assistive* seperti sensor LiDAR VL53L1X, ESP32, *Text-to-Speech* (TTS), MobileNet SSD, dan sistem navigasi berbasis Android. Tahap ini menjadi dasar dalam merancang sistem yang sesuai kebutuhan pengguna tunanetra.

3. Menentukan Komponen dan Desain Wiring

Berdasarkan hasil literatur, peneliti menentukan komponen utama yang akan digunakan seperti sensor VL53L1X, mikrokontroler ESP32, modul getar, serta perancangan komunikasi BLE ke aplikasi Android. Desain wiring disusun untuk memastikan integrasi antar komponen berjalan dengan baik.

4. Merangkai Modul Perangkat

Proses perakitan modul dimulai, termasuk menghubungkan sensor ke mikrokontroler, konfigurasi Bluetooth, dan pengujian awal konektivitas dengan perangkat Android.

5. Membuat Program Mikrokontroler

Mikrokontroler ESP32 diprogram untuk membaca data sensor jarak dari LiDAR VL53L1X dan mengirimkan data ke aplikasi Android melalui *Bluetooth Low Energy* (BLE). Sistem juga memicu motor getar saat hambatan terdeteksi dalam jarak tertentu.

6. Keakuratan Pengujian

Setelah sistem berjalan, dilakukan pengujian keakuratan pembacaan jarak oleh sensor. Jika pembacaan tidak sesuai, dilakukan perbaikan. Jika hasil sudah akurat, maka lanjut ke tahap berikutnya.

7. Mendesain Aplikasi

Aplikasi Android dikembangkan menggunakan Android Studio, dengan integrasi kamera untuk deteksi objek menggunakan MobileNet SSD, serta sistem navigasi menggunakan API OSRM, *Text-to-Speech* (TTS) untuk notifikasi suara.

8. Keberhasilan Pengujian Modul dan Aplikasi

Seluruh sistem baik modul tongkat dan aplikasi diuji dalam kondisi sebenarnya. Jika integrasi data jarak, deteksi objek visual, dan notifikasi berjalan sesuai harapan, maka proses penelitian dinyatakan selesai. Jika tidak, dilakukan perbaikan hingga berhasil.

9. Selesai

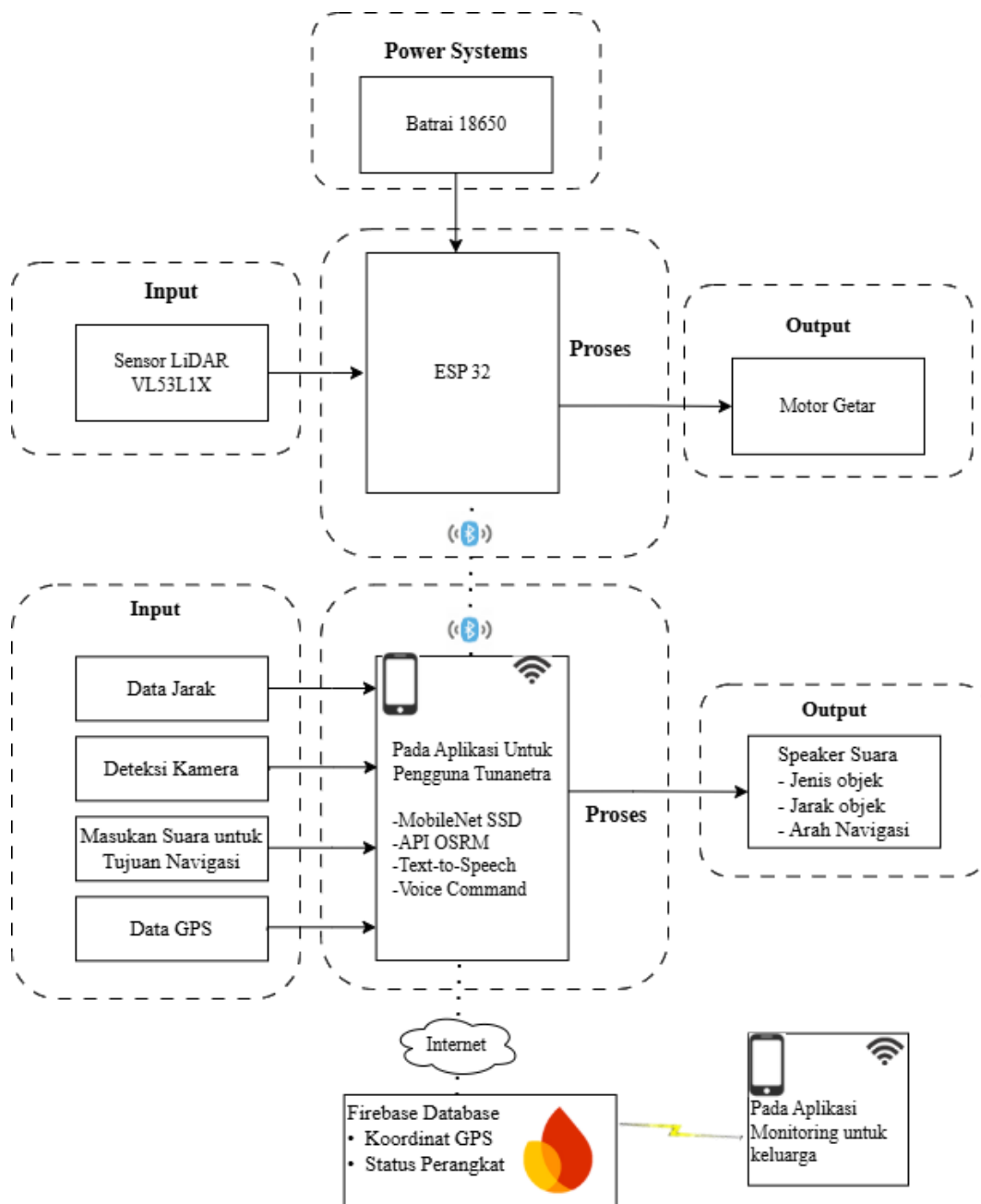
Penelitian selesai dengan menghasilkan prototipe tongkat pintar yang dapat digunakan oleh penyandang tunanetra secara efektif di lingkungan luar ruangan.

3.3. Perancangan Alat dan Aplikasi

3.3.1 Perancangan Alat

Perancangan alat pada penelitian ini dilakukan untuk menghasilkan prototipe tongkat pintar yang dapat membantu tunanetra dalam mendeteksi hambatan dan mendapatkan arahan navigasi secara *real-time*. Sistem ini dirancang dengan mengintegrasikan beberapa komponen utama, yaitu sensor jarak, mikrokontroler, motor getar, serta aplikasi mobile berbasis Android yang menjalankan algoritma kecerdasan buatan untuk deteksi objek visual dan navigasi arah. Dan data yang dapat diakses pengasuh.

3.3.2 Blok Diagram Sistem



Gambar 3. 2 Blok Diagram Sistem

Sistem tongkat pintar yang dirancang terdiri dari beberapa subsistem, yaitu input, catu daya (*power system*), proses, output, serta subsistem monitoring berbasis *cloud*. Setiap subsistem saling terhubung untuk mendukung proses deteksi objek, navigasi, dan pemantauan lokasi pengguna secara *real-time*.

1. Input

Pada sistem terdapat dua jenis input, yaitu input pada perangkat keras (*hardware*) dan input pada *smartphone Android*.

a. Sensor LiDAR VL53L1X (*Hardware/Tongkat*)

Sensor LiDAR VL53L1X yang terpasang pada tongkat pintar berfungsi mengukur jarak antara tongkat dengan objek atau hambatan di depan pengguna. Sensor memiliki jangkauan pengukuran hingga sekitar 4 meter dengan *Field of View* (FoV) sebesar 27°. Data hasil pengukuran dikirimkan ke ESP32 melalui komunikasi I²C untuk diproses lebih lanjut.

b. Data Jarak (ESP32 → *Smartphone Tunanetra*)

Data jarak merupakan hasil pembacaan sensor LiDAR yang telah diterima oleh ESP32, kemudian dikirimkan ke aplikasi Android melalui komunikasi *Bluetooth Low Energy* (BLE). Data ini digunakan sebagai informasi jarak hambatan.

c. Deteksi Kamera (*Smartphone Tunanetra*)

Deteksi kamera berasal dari kamera *smartphone* Android yang menangkap citra secara *real-time*. Citra tersebut menjadi masukan bagi model MobileNet SSD untuk mengenali objek di sekitar pengguna.

d. Masukan Suara (*Smartphone Tunanetra*)

Masukan suara diterima melalui *mikrofon smartphone* Android. Suara pengguna diproses menggunakan fitur *Speech Recognizer (Voice Command)* untuk menentukan tujuan navigasi.

e. Data GPS (*Smartphone Tunanetra*)

Data GPS diperoleh dari modul GPS bawaan *smartphone* Android. Informasi koordinat digunakan sebagai posisi pengguna pada proses navigasi dan monitoring.

2. Catu Daya (*Power System*)

Subsistem catu daya berada pada perangkat keras tongkat pintar. Dua baterai Li-Ion 18650 yang disusun paralel digunakan sebagai sumber energi utama untuk menyuplai ESP32, sensor LiDAR VL53L1X, dan motor getar.

3. Proses

a. ESP32 (*Hardware/Tongkat*)

ESP32 merupakan *mikrokontroler* yang bekerja pada perangkat keras tongkat pintar. ESP32 menerima data jarak dari sensor LiDAR melalui komunikasi I²C, mengolah data tersebut untuk menentukan pola getaran motor, kemudian mengirimkan data jarak ke *smartphone* menggunakan *Bluetooth Low Energy* (BLE).

b. Aplikasi Android (*Smartphone Tunanetra*)

Seluruh proses deteksi objek dan navigasi dijalankan pada *smartphone* Android. Aplikasi menerima data jarak dari ESP32, citra dari kamera *smartphone*, data GPS, serta masukan suara pengguna. Selanjutnya aplikasi menjalankan proses MobileNet SSD, API OSRM, *Voice Command*, dan *Text-to-Speech* (TTS) untuk menghasilkan informasi yang akan disampaikan kepada pengguna.

4. Output

a. Motor Getar (*Hardware/Tongkat*)

Motor getar yang dipasang pada pegangan tongkat menghasilkan umpan balik berupa getaran berdasarkan hasil pengukuran jarak oleh sensor LiDAR yang telah diproses oleh ESP32.

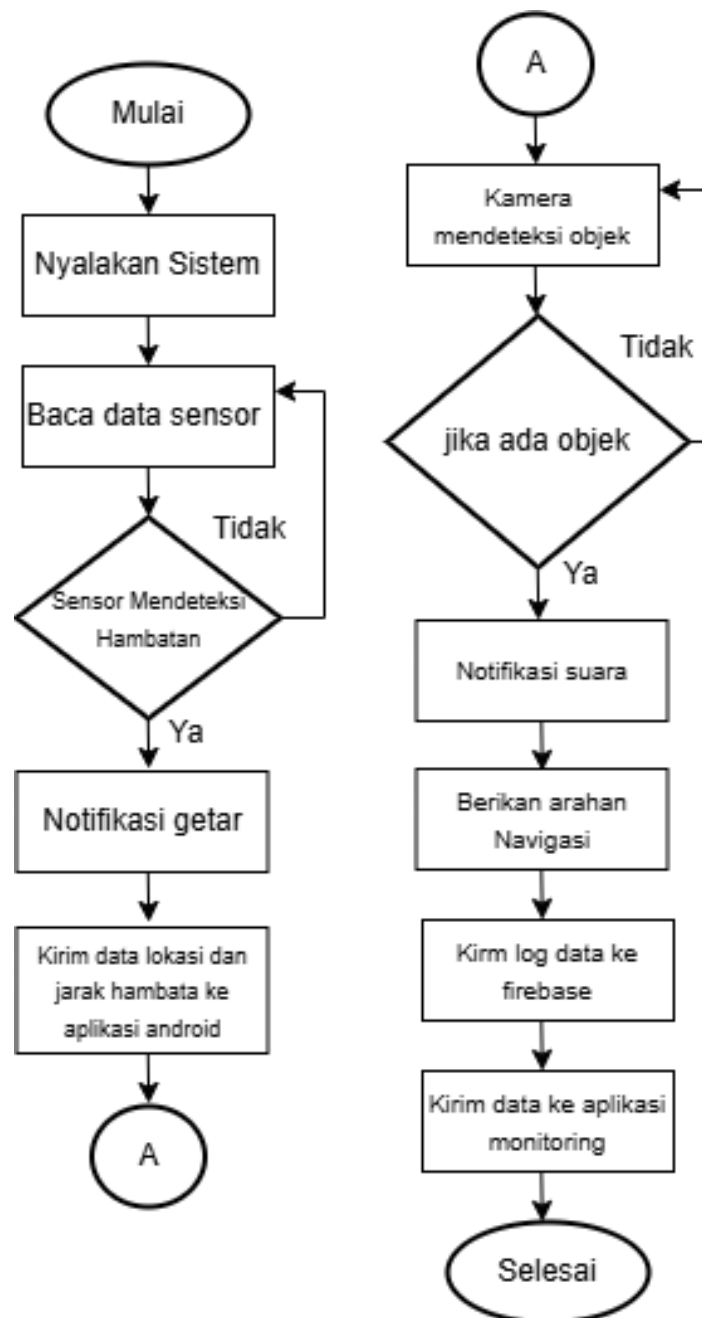
b. *Speaker Smartphone* (*Smartphone Tunanetra*)

Speaker bawaan *smartphone* menghasilkan keluaran suara dari fitur *Text-to-Speech* (TTS) berupa informasi jenis objek, jarak hambatan, serta petunjuk arah navigasi.

5. Subsistem Monitoring Berbasis Cloud

Subsistem monitoring bekerja menggunakan Firebase *Realtime Database* sebagai media penyimpanan berbasis *cloud*. Aplikasi Android mengirimkan koordinat GPS dan status perangkat melalui internet ke Firebase. Selanjutnya data tersebut diakses oleh aplikasi monitoring keluarga sehingga lokasi pengguna dapat dipantau secara *real-time*.

3.3.3 Diagram Alir Sistem



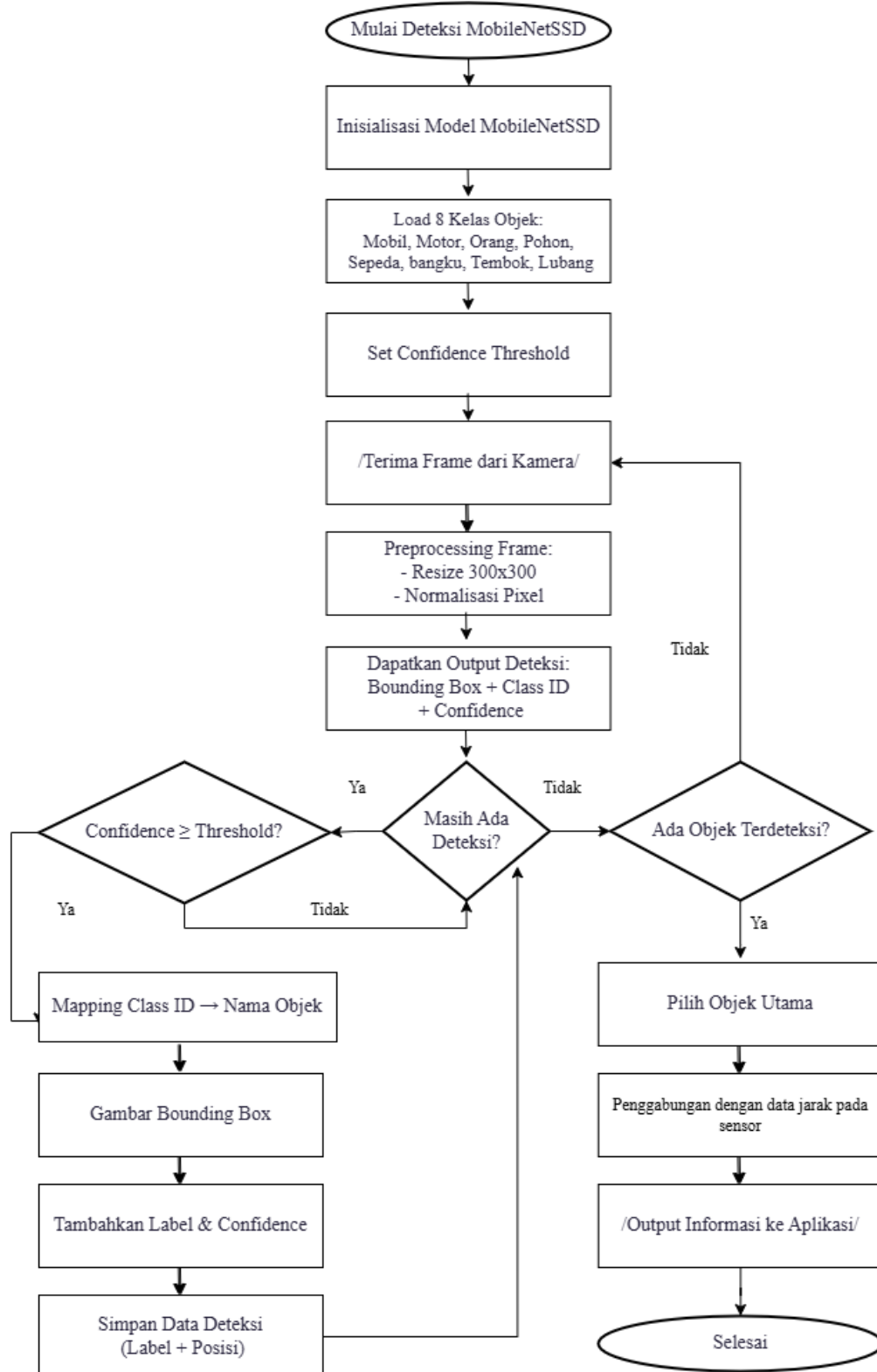
Gambar 3. 3 Diagram Alir Sistem

Sistem tongkat pintar untuk tunanetra mulai bekerja ketika perangkat keras dan aplikasi Android diaktifkan serta terhubung melalui Bluetooth. ESP32 membaca data jarak dari sensor LiDAR VL53L1X untuk mendeteksi hambatan di depan pengguna. Apabila terdeteksi objek pada jarak 1–2 meter, motor getar memberikan satu getaran secara periodik, sedangkan pada jarak kurang dari 50 cm motor getar memberikan dua getaran berturut-turut sebagai peringatan bahaya. Data jarak tersebut juga dikirimkan ke aplikasi Android melalui Bluetooth.

Fitur deteksi objek diaktifkan menggunakan tombol volume atas. Saat aktif, kamera *smartphone* menjalankan model MobileNet SSD untuk mendeteksi objek seperti orang, mobil, motor, sepeda, bangku, pohon, dan lubang. Hasil deteksi kemudian disampaikan kepada pengguna melalui fitur *Text-to-Speech* (TTS). Sementara itu, fitur navigasi diaktifkan menggunakan tombol volume bawah. Pengguna dapat mengucapkan tujuan perjalanan melalui mikrofon *smartphone*, kemudian perintah suara diproses menggunakan *Speech Recognizer* dan diteruskan ke layanan OSRM untuk menghasilkan rute navigasi yang akan disampaikan melalui TTS.

Selain itu, aplikasi mengambil data lokasi pengguna dari GPS *smartphone* secara *real-time* dan mengirimkannya ke Firebase Realtime Database melalui internet. Data tersebut dapat diakses oleh aplikasi monitoring keluarga sehingga posisi pengguna dapat dipantau secara *real-time*.

3.3.4 Diagram Alir Sistem Deteksi MobileNetSSD



Gambar 3. 4 Diagram Alir Sistem Deteksi MobileNetSSD

Diagram alir di atas menjelaskan proses pendeteksian objek menggunakan model *Convolutional Neural Network MobileNet Single Shot Detector (SSD)* yang dijalankan pada aplikasi Android. Sistem ini berfungsi untuk mengidentifikasi objek yang berada di sekitar pengguna tunanetra berdasarkan input berupa citra kamera.

Penjelasan alur prosesnya adalah sebagai berikut:

1. Mulai Sistem Deteksi

Proses dimulai ketika aplikasi pendeteksi objek dijalankan pada perangkat Android.

2. Inisialisasi Model MobileNet SSD

Sistem memuat arsitektur model MobileNet SSD yang ringan dan dioptimalkan untuk perangkat bergerak (*mobile device*).

3. Load Kelas Objek

Model memuat daftar kelas objek yang digunakan dalam penelitian. Pada sistem ini, terdapat 8 kelas objek yaitu: Mobil, Motor, Orang, Pohon, Sepeda, bangku, Tembok, Lubang.

4. Penentuan Nilai *Confidence Threshold*

Sistem menetapkan batas nilai kepercayaan (misalnya 0.5 atau 50%) sebagai syarat minimum agar suatu deteksi dianggap valid. Deteksi dengan nilai *Confidence* di bawah ambang batas akan diabaikan agar mengurangi kesalahan identifikasi.

5. Penerimaan Input *Frame* Kamera

Kamera perangkat menangkap gambar secara kontinu (*real-time*) sebagai masukan ke sistem deteksi.

6. *Preprocessing Frame*

Frame yang diterima diproses terlebih dahulu, meliputi:

- Mengubah ukuran citra menjadi 300×300 piksel, sesuai standar input MobileNet SSD
- Normalisasi nilai piksel agar sesuai dengan format model

7. *Forward Pass* ke *Neural Network*

Citra hasil *Preprocessing* dimasukkan ke model CNN untuk dilakukan proses ekstraksi fitur dan identifikasi objek.

8. Output Deteksi

Model menghasilkan keluaran berupa:

- *Bounding box* (koordinat lokasi objek)
- *Class ID* (jenis objek)
- *Confidence Score* (tingkat keyakinan deteksi)

9. Pemeriksaan *Confidence Threshold*

Jika *Confidence score* lebih besar atau sama dengan ambang batas, deteksi diterima. Jika tidak, sistem mengabaikan deteksi tersebut.

10. *Mapping Class ID* ke Nama Objek

Class ID kemudian diubah menjadi label teks yang mudah dipahami, misalnya “orang”, “mobil”, atau “motor”.

11. Pengiriman Informasi ke Sistem Output

Hasil deteksi objek akan diteruskan ke modul output dan pengabungan dengan pembacaan data jarak pada sensor, modul output seperti:

- *Text-to-Speech* (TTS) untuk memberikan peringatan suara
- Motor getar untuk memberi isyarat fisik

13. Proses Dilanjutkan ke *Frame* Berikutnya

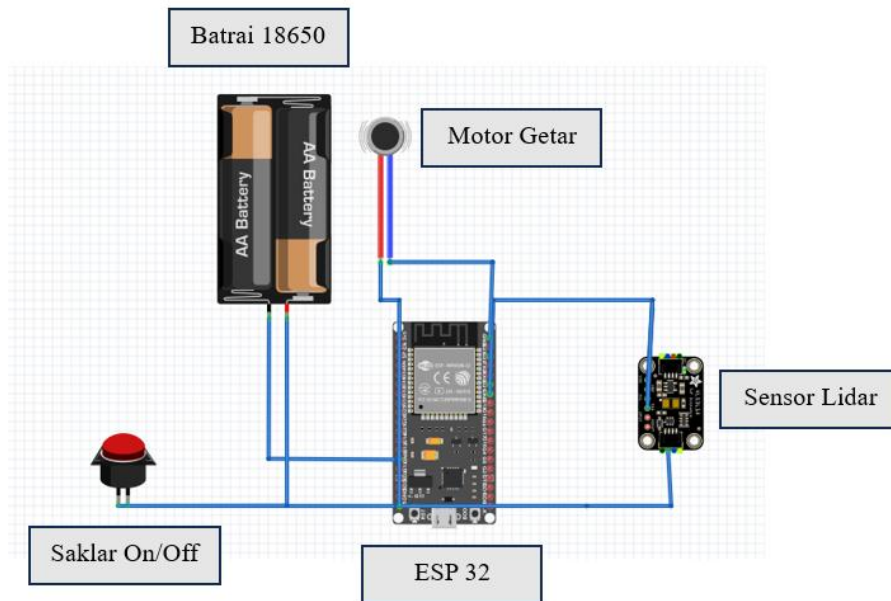
Sistem kembali ke proses penerimaan *Frame* kamera dan terus berjalan secara *real-time* selama aplikasi aktif.

14. Selesai

Proses berakhir ketika aplikasi dihentikan oleh pengguna.

MobileNet SSD berperan sebagai komponen utama dalam deteksi objek berbasis citra, yang berfungsi mengidentifikasi jenis dan posisi objek di sekitar pengguna. Informasi hasil deteksi kemudian diolah dan disampaikan kepada pengguna melalui output TTS ataupun umpan balik getaran sehingga sistem dapat membantu navigasi tunanetra secara *real-time*.

3.3.5 Skema Perancangan Rangkaian Alat Pada Tongkat



Gambar 3. 5 Skema Perancangan Rangkaian Alat pada Tongkat

Penjelasan dari perencanaan rangkaian sistem yang ditunjukkan pada Gambar 3.6 adalah sebagai berikut:

1. ESP32 DevKit berfungsi sebagai pusat pengendali perangkat keras yang menerima data jarak dari sensor LiDAR VL53L1X. Data tersebut kemudian diolah untuk menentukan pola getaran motor sebagai peringatan hambatan. Selain itu, ESP32 juga mengirimkan data jarak ke aplikasi Android melalui komunikasi Bluetooth.
2. Sensor LiDAR VL53L1X digunakan untuk mengukur jarak rintangan yang berada di depan pengguna. Sensor bekerja menggunakan teknologi *Time of Flight (ToF)* dan mampu mendeteksi objek hingga sekitar 4 meter. Data hasil pengukuran dikirimkan ke ESP32 melalui komunikasi I2C menggunakan pin SDA dan SCL.
3. Modul Motor Getar terhubung ke pin GPIO 25 pada ESP32 dan berfungsi sebagai media umpan balik taktil (*haptic feedback*). Frekuensi getaran akan berubah sesuai dengan jarak rintangan yang terdeteksi, sehingga pengguna dapat mengetahui tingkat kedekatan hambatan tanpa harus mendengarkan instruksi suara.
4. Saklar *ON/OFF* digunakan untuk menghubungkan dan memutus aliran daya dari baterai ke sistem. Komponen ini berfungsi untuk memudahkan

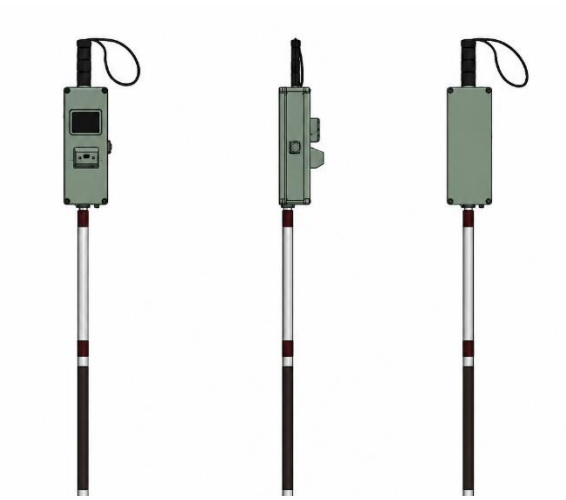
pengguna dalam menghidupkan maupun mematikan perangkat secara keseluruhan.

5. *Battery Shield V3* dengan dua baterai Li-ion 18650 berfungsi sebagai sumber daya utama sistem. Modul ini menyediakan tegangan keluaran yang stabil untuk menyuplai kebutuhan daya ESP32, sensor LiDAR, dan motor getar sehingga perangkat dapat digunakan secara portabel tanpa bergantung pada sumber listrik eksternal.

Tabel 3. 1 Daftar Pin Rangkaian Sistem

No	Komponen	Pin Komponen	Pin ESP32	Keterangan
1	LiDAR VL53L1X	VIN	VIN	Sumber tegangan sensor
		GND	GND	Ground bersama
		SDA	GPIO 21	Jalur data I2C
		SCL	GPIO 22	Jalur clock I2C
2	Motor Getar Coin 3V	In	GPIO 25	Sinyal kontrol getaran
		VCC (+)	VIN	Sumber tegangan
		GND (-)	GND	Ground motor
3	Saklar ON/OFF	Input	Positif baterai	Memutus dan menyambung catu daya
		Output	VIN ESP32	Menyalurkan daya ke sistem
4	<i>Battery Shield V3</i> (2 Slot 18650)	OUT+	VIN ESP32	Tegangan keluaran shield
		OUT-	GND ESP32	Ground sistem

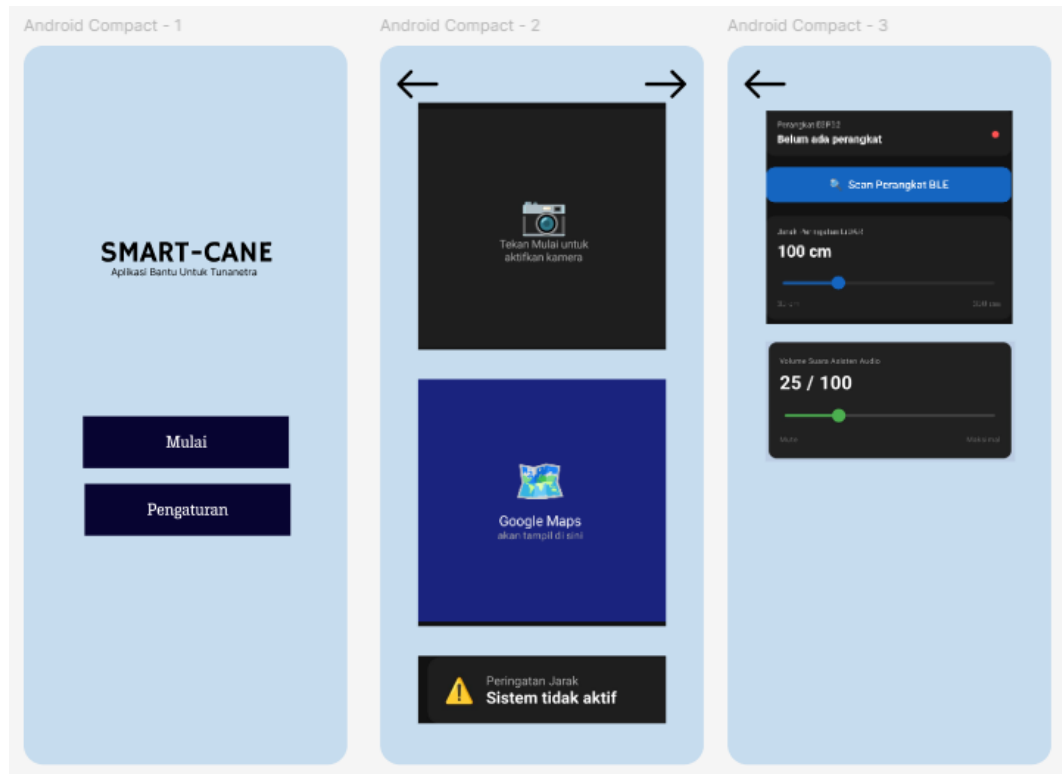
3.3.6 Desain Alat Pada Tongkat



Gambar 3. 6 Desain Alat

Gambar tersebut menunjukkan desain 3D dari sebuah tongkat pintar yang dirancang untuk membantu pengguna, khususnya penyandang tunanetra, dalam mendeteksi rintangan saat berjalan. Tongkat ini memiliki panjang total 120 cm, yang merupakan ukuran standar tongkat tunanetra pada umumnya. Di bagian atas tongkat terdapat kotak komponen berukuran 17 cm × 6 cm × 3,5 cm dengan berat sekitar 300 gram. Kotak ini berfungsi sebagai wadah bagi perangkat utama seperti ESP32, sensor LiDAR VL53L1X, baterai, dan motor getar. Sensor LiDAR VL53L1X dipasang pada bagian depan kotak dengan kemiringan sekitar 30° ke bawah terhadap arah horizontal tongkat. Kemiringan tersebut dipilih agar area deteksi mengarah ke jalur yang akan dilalui pengguna, sehingga hambatan di permukaan jalan maupun objek di depan dapat terdeteksi. Selain itu, posisi tersebut mengurangi kemungkinan sensor mendeteksi objek yang terlalu jauh di depan. Sementara itu, motor getar ditempatkan pada bagian pegangan tongkat dan berfungsi memberikan umpan balik berupa getaran apabila sensor mendeteksi adanya objek pada jarak tertentu.

3.3.7 Desain Aplikasi



Gambar 3. 7 Desain Aplikasi Pengguna Tunanetra

1. Tampilan Aplikasi untuk Pengguna Tunanetra

Tampilan aplikasi untuk pengguna tunanetra terdiri dari halaman utama, area deteksi objek, peta navigasi, serta panel informasi sistem. Antarmuka ini digunakan untuk menampilkan hasil proses deteksi objek, navigasi, dan data dari perangkat keras yang terhubung.

a. Tampilan Deteksi Kamera

Area deteksi kamera digunakan untuk menampilkan hasil deteksi objek yang diperoleh dari kamera *smartphone* menggunakan model MobileNet SSD. Objek yang dapat dideteksi meliputi orang, mobil, motor, sepeda, bangku, pohon, dan lubang. Hasil deteksi ditampilkan dalam bentuk kotak pembatas (*bounding box*) dan label objek pada layar aplikasi.

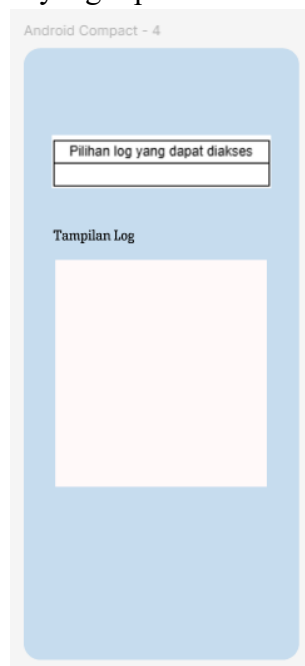
b. Peta Navigasi

Peta navigasi digunakan untuk menampilkan lokasi pengguna dan rute perjalanan menuju tujuan yang dipilih. Rute diperoleh melalui layanan OSRM (*Open Source Routing Machine*) berdasarkan koordinat GPS *smartphone*. Informasi navigasi selanjutnya disampaikan kepada pengguna melalui fitur *Text-to-Speech* (TTS). Lalu di bawah tampilan navigasi juga menampilkan jarak deteksi dari sensor lidar.

c. Pengaturan Sistem

Menu pengaturan sistem digunakan untuk melakukan konfigurasi dan pemantauan perangkat yang terhubung dengan aplikasi. Pada menu ini pengguna dapat menghubungkan aplikasi dengan ESP32 melalui Bluetooth, mengatur batas jarak deteksi sensor LiDAR VL53L1X, serta menyesuaikan volume keluaran suara (*Text-to-Speech*). Selain itu, menu ini juga menampilkan informasi status koneksi perangkat selama sistem beroperasi.

Secara keseluruhan, aplikasi berfungsi sebagai pusat pengolahan data yang mengintegrasikan fitur deteksi objek, navigasi, dan pemantauan perangkat. Informasi yang dihasilkan sistem disampaikan kepada pengguna melalui keluaran suara (*Text-to-Speech*) berdasarkan hasil deteksi objek, data jarak dari sensor LiDAR, dan petunjuk navigasi yang diperoleh dari layanan OSRM.



Gambar 3. 8 Desain Aplikasi Monitoring Tunanetra

1. Tampilan Aplikasi Monitoring Keluarga

Antarmuka aplikasi monitoring keluarga dirancang untuk memungkinkan anggota keluarga atau pendamping memantau aktivitas dan kondisi pengguna tunanetra secara jarak jauh. Tampilan aplikasi terdiri dari daftar log aktivitas dan area tampilan detail log yang tersimpan pada basis data Firebase.

a. Pilihan Log yang Dapat Diakses

Bagian ini menampilkan daftar log aktivitas yang tersimpan pada Firebase Realtime Database. Setiap log berisi informasi yang dikirimkan oleh aplikasi pengguna, seperti waktu (*timestamp*), lokasi pengguna, objek yang terdeteksi serta jaraknya. Daftar log dapat dipilih untuk melihat informasi lebih rinci terkait aktivitas pengguna pada waktu tertentu.

b. Tampilan Log Aktivitas

Bagian ini digunakan untuk menampilkan isi dari log yang dipilih. Informasi yang ditampilkan meliputi data lokasi pengguna, waktu pencatatan, dan aktivitas yang tersimpan pada sistem. Data log diperbarui secara otomatis setiap 3 menit dan disimpan ke dalam Firebase. Untuk memudahkan pengelolaan data, log dikelompokkan berdasarkan waktu penyimpanan sehingga tersusun dalam folder per jam. Selain itu, sistem secara otomatis mengganti atau menghapus log lama dari aplikasi setiap berganti hari, sehingga data yang ditampilkan selalu merupakan data terbaru. Tampilan ini memudahkan keluarga dalam memantau riwayat aktivitas dan lokasi pengguna berdasarkan data yang tersimpan pada sistem.

3.4. Alat dan Bahan

Tabel 3. 2 Alat dan Bahan

No	Nama Alat/Bahan	Jumlah	Fungsi
1	Sensor LiDAR VL53L1X	1 Unit	Mengukur jarak hambatan di depan pengguna menggunakan metode <i>Time of Flight</i> (ToF).

2	ESP32 DevKit V1	1 Unit	Mengolah data sensor LiDAR, mengendalikan motor getar, dan mengirim data ke aplikasi Android melalui Bluetooth.
3	Motor Getar (<i>Vibration Motor</i>)	1 Unit	Memberikan peringatan getar kepada pengguna berdasarkan jarak hambatan yang terdeteksi.
4	<i>Battery Shield V3 Dual Slot 18650</i>	1 Unit	Mengatur dan menyalurkan daya dari baterai ke seluruh perangkat keras.
5	Baterai Li-ion 18650 2500 mAh	2 Buah	Sumber daya utama sistem perangkat keras.
6	Saklar ON/OFF	1 Buah	Menghubungkan dan memutus aliran daya pada sistem.
7	Tongkat Tunanetra	1 Buah	Media pemasangan seluruh komponen perangkat keras.
8	<i>Smartphone</i> Android	1 Unit	Menjalankan aplikasi deteksi objek, navigasi, TTS, dan monitoring lokasi.
9	Android Studio	1 Software	Digunakan untuk mengembangkan aplikasi Android.
10	Visual Studio Code (Python 3.9)	1 Software	Digunakan untuk proses pelatihan dan pengujian model MobileNet SSD.
11	TensorFlow Object Detection API	1 <i>Framework</i>	Digunakan untuk melatih dan mengevaluasi model deteksi objek.
12	TensorFlow Lite	1 <i>Framework</i>	Digunakan untuk menjalankan model deteksi objek pada perangkat Android.
13	MobileNet SSD V2	1 Model	Model kecerdasan buatan untuk mendeteksi objek di sekitar pengguna.

14	Roboflow	1 Platform	Digunakan untuk anotasi, augmentasi, dan pengelolaan dataset.
15	Firebase Realtime Database	1 Layanan	Menyimpan dan menyinkronkan data lokasi pengguna secara <i>real-time</i> .
16	API OSRM (<i>Open Source Routing Machine</i>)	1 API	Menyediakan layanan navigasi dan penentuan rute perjalanan.
17	Kabel Jumper dan Konektor	Secukupnya	Menghubungkan antar komponen elektronik.
18	PCB/Perfboard	1 Buah	Media pemasangan rangkaian elektronik secara permanen.
19	Kotak Casing (Box Elektronik)	1 Buah	Melindungi komponen elektronik dari kerusakan fisik.
20	Double Tape, Lem Tembak, dan Cable Tie	Secukupnya	Digunakan untuk pemasangan dan perapian komponen pada tongkat.

3.5. Alur Penggunaan Sistem

Alur penggunaan sistem dimulai dengan menyalakan perangkat keras dan aplikasi Android yang akan terhubung melalui Bluetooth. Setelah koneksi berhasil dilakukan, pengguna dapat menggunakan tongkat pintar untuk mendeteksi hambatan, mengenali objek di sekitar, serta memperoleh bantuan navigasi menuju lokasi tujuan. Seluruh proses berlangsung secara real-time sehingga pengguna dapat memperoleh informasi mengenai kondisi lingkungan selama berjalan.

Langkah-langkah penggunaan sistem adalah sebagai berikut :

1. Menyalakan perangkat

Pengguna (bisa dari penggunan tunanetra mapupun dibantu keluarga) menyalakan perangkat dari tongkat dengan mengaktifkan saklar nya menjadi *on* sehingga ESP32, sensor LiDAR VL53L1X, dan motor getar dalam posisi aktif.

2. Melakukan konfigurasi dengan aplikasi

Selanjutnya pengguna menyalakan aplikasi *smart-cane* (bisa dibantu oleh keluarga) selanjutnya menyambungkan koneksi Aplikasi *smart-cane* dengan ESP32 melalui Bluetooth. Setelah koneksi berhasil, data hasil pembacaan sensor LiDAR dapat diterima oleh aplikasi Android secara *real-time*. Selanjutnya mengatur volume suara aplikasi dan batas pembacaan sensor lidar. Semua pengaturan ini di lakukan di halaman pengaturan aplikasi.

3. Mengaktifkan fitur deteksi objek

Pengguna (tunanetra) mengaktifkan kamera melalui aplikasi dengan menekan tombol volume atas 1 kali sehingga model SSD MobileNet V2 mulai melakukan proses deteksi objek secara *real-time*. Objek yang berhasil dikenali akan ditampilkan pada aplikasi dan diinformasikan kepada pengguna menggunakan fitur Text-to-Speech (TTS).

4. Menggunakan fitur navigasi

Pengguna (tunanetra) menentukan lokasi tujuan menggunakan fitur *Voice Command* sebagai masukkan perintah pencarian rute dengan cara menekan tombol volume bawah 1 kali. Selanjutnya aplikasi memperoleh rute dari layanan OSRM berdasarkan posisi GPS pengguna dan memberikan petunjuk arah menggunakan *Text-to-Speech*.

5. Proses berjalan

Selama pengguna (tunanetra) berjalan, sensor LiDAR terus mengukur jarak hambatan di depan pengguna. Apabila terdapat hambatan, ESP32 mengaktifkan motor getar sesuai jarak yang terdeteksi dan mengirimkan informasi jarak ke aplikasi Android. Pada saat yang sama, aplikasi melakukan deteksi objek menggunakan kamera dan memberikan informasi suara mengenai jenis objek, jarak hambatan, serta petunjuk navigasi.

6. Aplikasi monitoring

Selama sistem digunakan, koordinat GPS pengguna (tunanetra), dan hasil deteksi dikirimkan ke Firebase *Realtime Database* sehingga informasi pengguna (tunanetra) dapat dipantau secara *real-time* melalui aplikasi monitoring.

7. Mengakhiri penggunaan sistem

Setelah pengguna (tunanetra) mencapai tujuan, aplikasi memberikan notifikasi suara bahwa tujuan telah tercapai. Selanjutnya pengguna (tunanetra) dapat menutup fitur deteksi dengan menekan tombol volume atas 2 kali dan untuk mematikan fitur deteksi menekan tombol volume bawah 2 kali pada aplikasi dan mematikan perangkat tongkat dengan menekan saklar on menjadi off.

3.6. Parameter Pengujian

Dalam penelitian ini, terdapat tiga parameter utama yang digunakan untuk menguji kinerja dan keandalan sistem tongkat pintar, yaitu pengujian fungsionalitas perangkat keras, fungsionalitas aplikasi, dan konektivitas IoT.

1. Pengujian Fungsionalitas Perangkat Hardware

Pengujian ini bertujuan untuk memastikan bahwa seluruh komponen perangkat keras berfungsi sebagaimana mestinya. Komponen yang diuji antara lain sensor LiDAR VL53L1X, motor getar, serta mikrokontroler ESP32. Fokus pengujian meliputi kemampuan sensor dalam mendeteksi hambatan pada berbagai jarak, serta respon sistem dalam memberikan peringatan melalui suara dan getaran.

2. Pengujian Fungsionalitas Aplikasi

Pengujian ini dilakukan pada aplikasi Android yang dikembangkan untuk mendeteksi objek menggunakan kamera *smartphone* dengan algoritma MobileNet SSD, serta memberikan navigasi dan peringatan suara. Pengujian mencakup akurasi deteksi objek, pengaturan *Confidence Threshold*, responsivitas antarmuka pengguna (*user interface*), serta kejelasan dan ketepatan informasi suara yang disampaikan melalui fitur *Text-to-Speech* (TTS).

3. Pengujian IoT Connectivity

Pengujian ini dilakukan untuk menilai kestabilan dan kecepatan konektivitas antara perangkat keras (ESP32) dan aplikasi Android melalui platform Firebase. Aspek yang diuji meliputi sinkronisasi data secara *real-time* dalam pengiriman dan penerimaan data, serta keandalan sistem dalam mempertahankan koneksi selama perangkat digunakan.

3.7 Perancangan Program ESP32

Program pada ESP32 berfungsi sebagai pengendali utama perangkat *Smart-Cane*. Program ini bertugas membaca data dari sensor LiDAR VL53L1X, mengendalikan motor getar sebagai peringatan haptic, serta mengirimkan data jarak ke aplikasi Android melalui komunikasi Bluetooth *Low Energy* (BLE). Sensor LiDAR VL53L1X digunakan untuk mengukur jarak antara tongkat dengan objek di depannya. Komunikasi antara ESP32 dan sensor menggunakan antarmuka I2C melalui pin SDA dan SCL. Sensor dikonfigurasi menggunakan *mode Long Distance* sehingga mampu melakukan pengukuran hingga sekitar 4 meter.

a. Program Pembacaan Sensor LiDAR

Pembacaan sensor dilakukan secara periodik setiap 300 ms menggunakan fungsi `millis()` sehingga program dapat berjalan secara *non-blocking* tanpa menggunakan fungsi `delay()`. Data hasil pembacaan kemudian dilakukan proses validasi berdasarkan status sensor dan batas maksimum jarak pengukuran. Selain itu diterapkan metode *Moving Average Filter* dengan lima sampel untuk mengurangi fluktuasi pembacaan sensor akibat gangguan cahaya maupun noise.

Potongan program pembacaan sensor ditunjukkan pada Gambar 3.10

```
void bacaLidar() {  
    if (!lox.dataReady()) {  
        lox.clearInterrupt();  
        return;  
    }  
  
    int16_t rawJarak = lox.distance();  
    lox.clearInterrupt();  
  
    if (bacaanValid) {  
        jarak = movingAverage(rawJarak);  
    }  
}
```

Gambar 3. 9 Gambar Potongan Program Pembacaan Sensor LiDAR

Program di atas merupakan fungsi `bacaLidar()` yang digunakan untuk membaca data jarak dari sensor LiDAR VL53L1X. Proses pembacaan diawali dengan pengecekan kondisi sensor menggunakan fungsi `lox.dataReady()`. Apabila data belum siap dibaca, sistem akan menjalankan fungsi `clearInterrupt()` untuk menghapus status interupsi pada sensor, kemudian fungsi dihentikan sementara menggunakan `return`. Langkah ini bertujuan agar ESP32 tidak membaca data yang belum valid.

Selanjutnya, apabila data telah tersedia, nilai jarak diambil menggunakan fungsi `lox.distance()` dan disimpan ke dalam variabel `rawJarak`. Setelah pembacaan selesai, fungsi `clearInterrupt()` dipanggil kembali untuk menghapus status pembacaan sehingga sensor dapat melakukan pengukuran berikutnya. Data jarak yang diperoleh kemudian divalidasi melalui variabel `bacaanValid`. Jika data dinyatakan valid, nilai jarak akan diproses menggunakan metode *Moving Average Filter* melalui fungsi `movingAverage()`. Metode ini menghitung rata-rata dari lima data pembacaan terakhir sehingga fluktuasi pengukuran akibat noise atau gangguan cahaya dapat dikurangi. Hasil filtering selanjutnya digunakan sebagai nilai jarak yang dikirimkan ke aplikasi Android melalui Bluetooth Low Energy (BLE) dan sebagai acuan dalam menentukan pola getaran motor.

b. Program Motor Getar

Motor getar digunakan sebagai media peringatan haptic kepada pengguna berdasarkan jarak objek yang dideteksi sensor LiDAR. Sistem menggunakan tiga tingkat peringatan yang dibedakan berdasarkan jarak objek.

```
// Logika pemicu getaran berdasarkan jarak halangan
unsigned long now = millis();
if (now - lastVibrateMillis >= VIBRATE_COOLDOWN && !vib.active) {
    if (jarak <= JARAK_BAHAYA) {
        Serial.println("[DANGER] Objek sangat dekat! Menjalankan getar 2x panjang.");
        startVibrate(2, 300);
        lastVibrateMillis = now;
    } else if (jarak <= JARAK_PERINGATAN) {
        Serial.println("[WARNING] Objek mendekat! Menjalankan getar 1x sedang.");
        startVibrate(1, 200);
        lastVibrateMillis = now;
    } else if (jarak <= JARAK_WASPADA) {
        Serial.println("[CAUTION] Ada objek di depan. Menjalankan getar 1x pendek.");
        startVibrate(1, 100);
        lastVibrateMillis = now;
    }
}
```

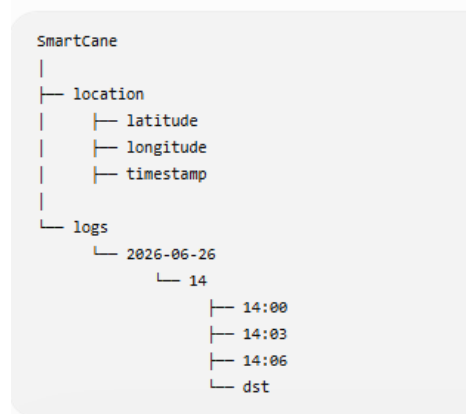
Gambar 3. 10 Program Motor Getar

Potongan program pada Gambar 3.11 menunjukkan proses pengendalian motor getar berdasarkan jarak objek yang dideteksi oleh sensor LiDAR VL53L1X. Sebelum motor getar diaktifkan, sistem memanfaatkan fungsi `millis()` untuk memberikan jeda (*cooldown*) antar getaran sehingga motor tidak aktif secara terus-menerus. Selain itu, kondisi `!vib.active` digunakan untuk memastikan motor tidak menerima perintah getaran baru ketika proses getaran sebelumnya masih berlangsung.

Selanjutnya, sistem membandingkan nilai jarak objek dengan tiga batas jarak yang telah ditentukan, yaitu `JARAK_BAHAYA` (≤ 50 cm), `JARAK_PERINGATAN` (≤ 200 cm), dan `JARAK_WASPADA` (≥ 200 cm). Apabila objek berada pada jarak kurang dari atau sama dengan 50 cm, motor akan menghasilkan dua kali getaran dengan durasi 300 ms sebagai peringatan bahaya. Jika objek berada pada jarak 50–200 cm, motor akan menghasilkan satu kali getaran dengan durasi 200 ms sebagai peringatan sedang. Sementara itu, apabila objek berada pada jarak lebih dari 200 cm, motor akan menghasilkan satu kali getaran dengan durasi 100 ms sebagai peringatan awal. Pengaturan pola getaran ini bertujuan agar pengguna dapat membedakan tingkat kedekatan hambatan hanya melalui umpan balik getaran tanpa perlu melihat layar *smartphone*.

3.8 Perancangan Database Firebase

Pada penelitian ini, Firebase Realtime Database digunakan sebagai media penyimpanan data berbasis *cloud* untuk mendukung proses pemantauan pengguna tunanetra secara *real-time*. Data yang dikirim berasal dari aplikasi Android pengguna, kemudian disimpan pada Firebase sehingga dapat diakses oleh aplikasi monitoring keluarga melalui jaringan internet.



Gambar 3. 11 Struktur penyimpanan data pada Firebase.

Firebase dipilih karena memiliki kemampuan sinkronisasi data secara *real-time*, sehingga setiap perubahan data lokasi maupun riwayat aktivitas pengguna dapat langsung diperbarui pada aplikasi monitoring tanpa memerlukan proses *refresh* secara manual.

3.9 Perancangan AI Deteksi Objek

Rancangan Instalasi *Library*

Pada tahap awal pengembangan model deteksi objek dilakukan proses instalasi berbagai *library* yang dibutuhkan untuk proses pelatihan (*training*) dan implementasi model. Lingkungan pengembangan menggunakan Python 3.9, Visual Studio Code, serta TensorFlow Object Detection API sebagai *Framework* utama.

Library yang diinstal meliputi TensorFlow, TensorFlow Object Detection API, Roboflow, Protobuf, Pillow, Matplotlib, Pandas, dan Pycocotools. *Library* tersebut digunakan untuk proses pengolahan dataset, pelatihan model MobileNet SSD, konversi model TensorFlow Lite, serta pengujian hasil deteksi objek.

```
pip install tensorflow-cpu==2.12.0
pip install protobuf==3.20.3
pip install pillow matplotlib pandas
pip install roboflow requests
pip install tf-slim lvis
pip install pycocotools-windows
```

Gambar 3. 12 *Library yang harus diinstal*

Berdasarkan Gambar 3.12 dilakukan instalasi beberapa *library* yang diperlukan untuk proses pengembangan model deteksi objek menggunakan Python. *Library* yang diinstal meliputi TensorFlow sebagai *Framework* utama untuk membangun dan menjalankan model *deep learning*, TensorFlow Object Detection API untuk proses pelatihan model MobileNet SSD, Roboflow untuk mengunduh dataset, Protobuf sebagai pendukung TensorFlow Object Detection API, Pillow untuk pengolahan citra, Matplotlib untuk visualisasi hasil pelatihan, Pandas untuk pengolahan data, tf-slim sebagai pendukung model TensorFlow, serta Pycocotools untuk membaca dataset COCO. Seluruh *library* berhasil diinstal sehingga lingkungan pengembangan siap digunakan untuk proses pelatihan model deteksi objek.

3.6.3 Konfigurasi Model SSD MobileNet V2

Konfigurasi model dilakukan menggunakan file *pipeline.config* pada TensorFlow Object Detection API. Konfigurasi ini digunakan untuk menentukan parameter pelatihan, model yang digunakan, dataset, serta output model sebelum proses *training* dijalankan.

Tabel 3. 3 Konfigurasi Model SSD MobileNetV2

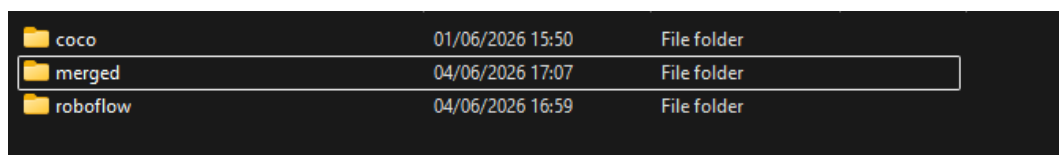
Parameter	Nilai
<i>Framework</i>	TensorFlow Object Detection API
Model	SSD MobileNet V2

Pre-trained Model	SSD MobileNet V2 320×320 COCO17 TPU-8
Ukuran Citra (Input)	300 × 300 piksel
Jumlah Kelas	8 kelas
Dataset	COCO + Roboflow (Merge Dataset)
Batch Size	4
Optimizer	Momentum Optimizer
Learning rate	Cosine Decay (Base Learning rate = 0,02)
Warmup Learning rate	0,005
Warmup Steps	500
Jumlah Training Steps	150.000
Data Augmentation	Random Horizontal Flip, SSD Random Crop, Random Brightness, Random Contrast
Checkpoint Awal	SSD MobileNet V2 COCO17 TPU-8
Output Model	TensorFlow Lite (.tflite)

Berdasarkan konfigurasi tersebut, model selanjutnya dilatih menggunakan dataset hasil penggabungan COCO dan Roboflow hingga diperoleh model TensorFlow Lite yang digunakan pada aplikasi *Smart-Cane*.

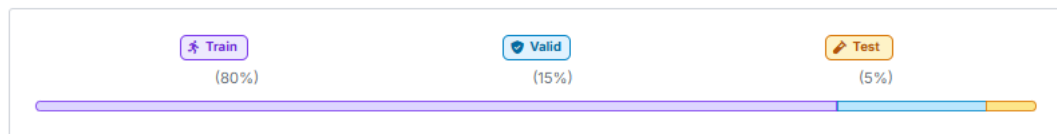
3.10 Perancangan *Training*

Pada penelitian ini, proses pelatihan (*training*) model deteksi objek dilakukan menggunakan metode transfer learning dengan model dasar SSD MobileNet. Dataset yang digunakan berasal dari dua sumber, yaitu dataset COCO dan dataset Roboflow. Dataset COCO digunakan untuk memperoleh data objek umum seperti orang, mobil, motor, sepeda, dan bangku, sedangkan dataset Roboflow digunakan untuk menambahkan objek yang lebih spesifik sesuai kebutuhan penelitian, yaitu lubang, tembok, dan pohon.



Gambar 3. 13 Folder Datasets

Berdasarkan Gambar 3.13, proses pelatihan diawali dengan mengumpulkan dataset dari dua sumber, yaitu COCO dan Roboflow. Selanjutnya kedua dataset digabungkan (*merge*) menjadi satu dataset agar seluruh objek yang dibutuhkan tersedia dalam satu folder. Setelah proses *merge* selesai, dilakukan penyamaan nama kelas (*class name*) ke dalam Bahasa Indonesia sehingga label objek yang digunakan pada seluruh dataset menjadi seragam dan sesuai dengan bahasa yang digunakan pada aplikasi *Smart-Cane*



Gambar 3. 14 Pembagian Dataset

Setelah proses penggabungan dataset selesai, data kemudian dibagi menjadi tiga bagian, yaitu 80% *training*, 15% *validation*, dan *testing* sebanyak (5%). Data digunakan untuk melatih model, data *validation* digunakan untuk mengevaluasi performa model selama proses pelatihan, sedangkan data *testing* digunakan untuk menguji kemampuan model terhadap data yang belum pernah dikenali sebelumnya.

```
python -c "  
import tensorflow as tf  
converter = tf.lite.TFLiteConverter.from_saved_model('exported_model/saved_model')  
converter.optimizations = [tf.lite.Optimize.DEFAULT]  
tflite_model = converter.convert()  
open('model.tflite', 'wb').write(tflite_model)
```

Gambar 3. 15 Program Ekspor ke Format TensorFlow Lite

Setelah proses pelatihan menghasilkan model dengan performa yang sesuai, model kemudian diekspor ke format TensorFlow Lite (.tflite). Proses konversi dilakukan menggunakan TensorFlow Lite *Converter* sehingga model dapat dijalankan pada perangkat Android dengan kebutuhan memori yang lebih kecil dan waktu inferensi yang lebih cepat. File model.tflite yang dihasilkan selanjutnya diintegrasikan ke dalam aplikasi *Smart-Cane* melalui modul *ObjectDetector* untuk melakukan deteksi objek secara *real-time*.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Tempat dan Waktu Penelitian

Pengujian sistem tongkat pintar dilakukan di beberapa lokasi di sekitar Politeknik Negeri Malang dan lingkungan tempat tinggal penulis yang berlokasi di Jalan Taman Bunga Merak, Kota Malang. Pemilihan lokasi tersebut bertujuan untuk menguji kinerja sistem pada kondisi lingkungan yang umum ditemui oleh penyandang tunanetra, seperti area pejalan kaki, halaman, dan jalan sekitar. Pengujian dilakukan dengan berbagai variasi objek sesuai kelas yang ada. untuk mengevaluasi kemampuan sistem dalam mendeteksi objek, mengukur jarak rintangan, serta memberikan informasi navigasi kepada pengguna.

Seluruh rangkaian pengujian dilaksanakan selama periode Februari 2026 hingga Juli 2026. Selama periode tersebut dilakukan pengujian terhadap model deteksi objek berbasis MobileNet SSD, sensor LiDAR VL53L1X, fitur navigasi berbasis OSRM, serta integrasi keseluruhan sistem tongkat pintar. Hasil pengujian kemudian dianalisis untuk mengetahui tingkat keberhasilan sistem dalam membantu proses navigasi dan deteksi rintangan bagi penyandang tunanetra.

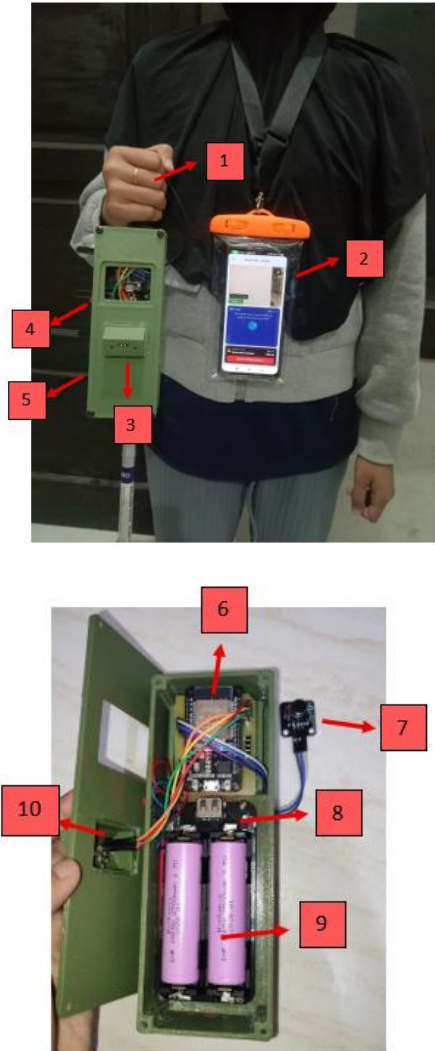
4.2 Hasil Penelitian

Bab ini menyajikan hasil penelitian dan pembahasan yang diperoleh selama proses perancangan, implementasi, dan pengujian sistem tongkat pintar deteksi objek dan navigasi bagi penyandang tunanetra. Hasil penelitian disusun secara sistematis untuk menjawab rumusan masalah serta mengevaluasi pencapaian tujuan penelitian yang telah ditetapkan. Pembahasan pada bab ini dibagi menjadi beberapa bagian, yaitu hasil perancangan dan implementasi sistem, hasil pelatihan model deteksi objek MobileNet SSD, hasil pengujian deteksi objek, hasil pengujian sensor LiDAR VL53L1X, hasil pengujian fitur navigasi dan perintah suara, serta hasil pengujian integrasi keseluruhan sistem. Selanjutnya, setiap hasil pengujian dianalisis untuk mengetahui kinerja sistem dalam mendukung deteksi rintangan dan navigasi bagi penyandang tunanetra.

4.2.1 Hasil Perencanaan Desain Alat

Tahap perencanaan desain alat dilakukan sebagai langkah awal dalam pengembangan sistem tongkat pintar untuk penyandang tunanetra. Pada tahap ini dirancang struktur perangkat keras yang mengintegrasikan sensor LiDAR VL53L1X, mikrokontroler ESP32, modul getar, sumber daya baterai 18650, serta *smartphone* Android sebagai media pemrosesan deteksi objek dan navigasi. Gambar dan tabel berikut menunjukkan hasil perancangan sistem tongkat pintar yang telah dibuat.

Tabel 4. 1 Hasil Perencanaan Desain Alat

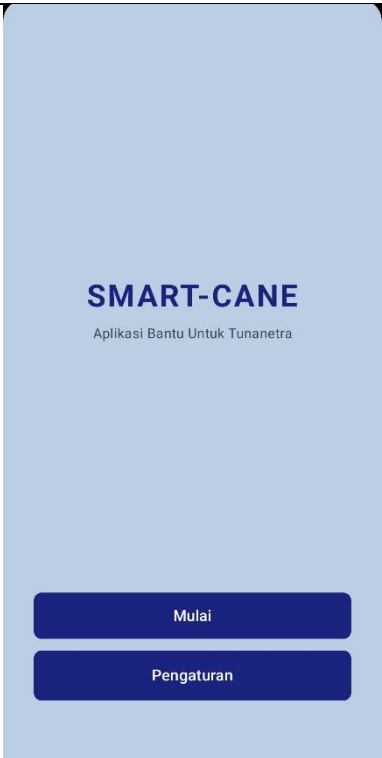
Hasil Desain	Keterangan
	1. Penempatan Modul Getar 2. Penempatan Aplikasi Tunanetra 3. Penempatan Sensor Lidar 4. Penempatan Saklar <i>On/Off</i> 5. Penempatan Lubang <i>Charger</i> 6. ESP 32 7. Modul Getar 8. <i>Shield Battery</i> 18650 V3 9. Battery 18650 10. Sensor Lidar

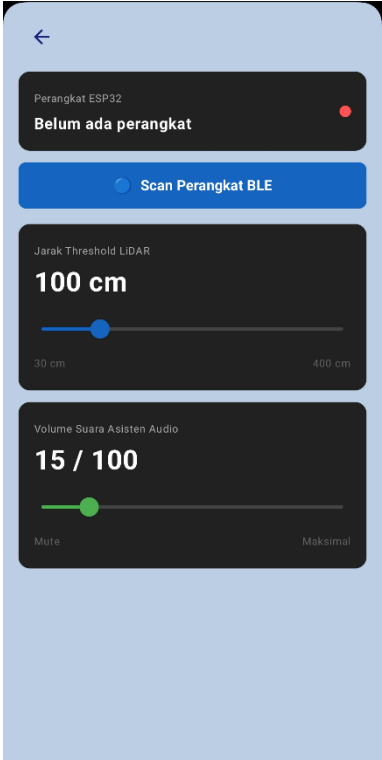
Pada tahap perencanaan desain alat, dilakukan perancangan perangkat keras sistem tongkat pintar secara menyeluruh dengan mengintegrasikan sensor LiDAR VL53L1X, mikrokontroler ESP32, modul getar, baterai, dan *smartphone* Android sebagai pendukung deteksi objek serta navigasi. Perancangan dilakukan untuk memastikan seluruh komponen dapat bekerja sesuai fungsinya dan terpasang dengan baik pada tongkat sehingga nyaman digunakan oleh pengguna. Hasil perancangan tersebut kemudian direalisasikan menjadi prototipe tongkat pintar yang dapat digunakan sebagai alat bantu navigasi bagi penyandang tunanetra.

4.2.2 Hasil Perencanaan Aplikasi

Selain perancangan perangkat keras, penelitian ini juga merancang aplikasi Android sebagai antarmuka utama sistem. Aplikasi yang dikembangkan terdiri dari dua bagian, yaitu aplikasi untuk pengguna tunanetra dan aplikasi untuk keluarga atau pengawas. Aplikasi tunanetra berfungsi untuk mendukung deteksi objek, navigasi, perintah suara, serta pemberian informasi melalui suara. Sementara itu, aplikasi keluarga digunakan untuk melakukan pemantauan lokasi pengguna secara *real-time* serta melihat riwayat perjalanan yang tersimpan pada sistem. Perencanaan fitur pada kedua aplikasi ditunjukkan pada Tabel

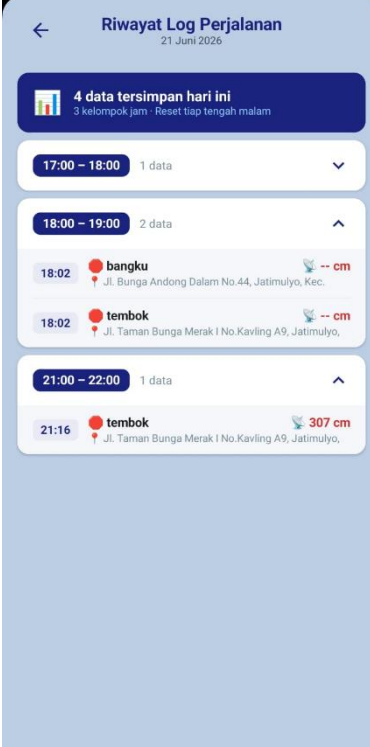
Tabel 4. 2 Hasil Perancangan Aplikasi untuk Tunanetra

Hasil Desain	Keterangan
	- Pengguna menekan tombol Mulai untuk masuk ke halaman deteksi dan navigasi. - Tombol Pengaturan digunakan untuk masuk ke menu konfigurasi sistem.
	- Untuk mengaktifkan deteksi objek, pengguna menekan tombol volume atas satu kali. Kamera <i>smartphone</i> akan aktif dan sistem mulai mendeteksi objek di sekitar pengguna. Untuk menonaktifkan deteksi objek, pengguna menekan tombol volume atas dua kali. - Untuk memulai navigasi, pengguna menekan tombol volume bawah satu kali, kemudian mengucapkan nama lokasi tujuan. Sistem akan memproses tujuan dan memberikan petunjuk arah melalui suara. Untuk

	menghentikan navigasi, pengguna menekan tombol volume bawah dua kali. c. Nilai jarak yang ditampilkan merupakan hasil pembacaan sensor LiDAR VL53L1X .
	a. Pengguna menekan tombol Scan Perangkat BLE untuk mencari perangkat ESP32 yang tersedia. Setelah perangkat terhubung indikator yang semula merah menjadi berwarna hijau tanda sudah terhubung. b. Pengguna dapat menggeser <i>slider</i> untuk menentukan batas jarak deteksi yang diinginkan. c. Pengguna dapat menggeser <i>slider</i> volume untuk mengatur tingkat keras suara keluaran <i>Text-to-Speech</i> (TTS) sesuai kebutuhan.

Tabel 4. 3 Hasil Perancangan Aplikasi untuk Monitoring Tunanetra

Hasil Desain	Keterangan
 The image shows a login screen for the 'SMART-CANE' application. At the top, there is an illustration of a white cane with a red tip. Below it, the text 'SMART-CANE' is displayed in a bold, dark blue font. Underneath, in a smaller font, it says 'Aplikasi Monitoring Keluarga' and 'Menghubungkan & Memantau Real-Time'. At the bottom, there is a dark blue button with the white text 'MASUK KE MONITORING'.	Pengguna (keluarga) menekan tombol Masuk ke Monitoring untuk masuk ke halaman utama pemantauan.
 The image shows the dashboard of the 'SMART-CANE' application. At the top, it says 'Halo, Keluarga' with a hand icon and 'Monitoring Smart-Cane'. Below this is a dark blue banner with a cane icon and the text 'Periode Aktif Perangkat Terhubung Terkini' and 'Sinkronisasi Cloud Real-Time'. There are two white cards: one with a target icon labeled 'Objek Terakhir lubang' and another with a clock icon labeled 'Waktu Pembaruan 21:17'. Below these is a white card with a location pin icon labeled 'Lokasi Jalan Terakhir Jl. Taman Bunga Merak I' and a 'Lihat >' link. At the bottom, there are two dark blue buttons: 'Riwayat Log' and 'Peta Lokasi'.	<i>Dashboard</i> menampilkan informasi terbaru mengenai kondisi pengguna, meliputi status perangkat, objek terakhir yang terdeteksi, waktu pembaruan data, serta lokasi terakhir pengguna. Pada halaman ini juga tersedia menu Riwayat Log dan Peta Lokasi.

	Halaman ini menampilkan posisi pengguna tunanetra secara <i>real-time</i> pada peta. Selain lokasi, ditampilkan pula informasi alamat, waktu pembaruan terakhir, objek yang terdeteksi, dan jarak hambatan yang dibaca oleh sensor LiDAR.
	Halaman ini digunakan untuk menampilkan riwayat aktivitas yang tersimpan pada Firebase. Data log dikelompokkan berdasarkan waktu pencatatan dan memuat informasi objek yang terdeteksi, lokasi pengguna, serta jarak hambatan saat data direkam.

Berdasarkan Tabel 4.2 dan Tabel 4.3, aplikasi tunanetra dirancang sebagai pusat interaksi pengguna dengan sistem tongkat pintar. Aplikasi ini

mengintegrasikan fitur deteksi objek menggunakan kamera *smartphone*, navigasi pejalan kaki, perintah suara, notifikasi suara (*Text-to-Speech*), serta komunikasi dengan perangkat ESP32 melalui Bluetooth. Di sisi lain, aplikasi keluarga dirancang untuk mendukung pemantauan pengguna melalui tampilan peta lokasi secara *real-time* dan penyimpanan log perjalanan. Dengan adanya dua aplikasi yang saling terhubung, sistem tidak hanya membantu pengguna dalam mengenali lingkungan sekitar, tetapi juga memungkinkan keluarga untuk memantau keberadaan pengguna ketika diperlukan.

4.2.3 Hasil Pengujian Sensor LiDAR VL53L1X

Pengujian sensor LiDAR VL53L1X dilakukan untuk mengetahui tingkat akurasi pembacaan jarak yang dihasilkan sensor terhadap jarak sebenarnya. Pengujian dilakukan dengan membandingkan hasil pembacaan sensor yang ditampilkan pada terminal arduino IDE dengan hasil pengukuran menggunakan meteran sebagai nilai acuan.

Pengambilan data dilakukan pada beberapa variasi jarak mulai dari 10 cm hingga 400 cm. Pada setiap jarak dilakukan pengukuran dengan menempatkan objek di depan sensor LiDAR VL53L1X, kemudian hasil pembacaan sensor dicatat dan dibandingkan dengan jarak sebenarnya yang diukur menggunakan meteran.



Gambar 4. 1 Pengujian Sensor Lidar

Langkah-langkah pengujian yang dilakukan adalah sebagai berikut:

1. Menyiapkan sensor LiDAR VL53L1X, ESP32, *smartphone* Android, meteran sebagai alat ukur dan benda sebagai halangan uji.
2. Menghubungkan sensor LiDAR VL53L1X dengan ESP32 melalui komunikasi I2C.

3. Menempatkan objek pada jarak tertentu mulai dari 10 cm hingga 400 cm.
4. Mencatat hasil pembacaan jarak yang ditampilkan oleh sistem.
5. Membandingkan hasil pembacaan sensor dengan hasil pengukuran menggunakan meteran.
6. Menghitung error dan akurasi pembacaan sensor.

Untuk mengetahui tingkat kesalahan pengukuran sensor digunakan persamaan sebagai berikut:

$$Error (\%) = \frac{[jarak\ terukur - jarak\ pengukuran]}{jarak\ pengukuran} \times 100\%$$

Keterangan:

- Jarak Sensor = hasil pembacaan jarak oleh sensor LiDAR VL53L1X
- Jarak Referensi = hasil pengukuran menggunakan meteran

Selanjutnya nilai akurasi dihitung menggunakan persamaan:

$$Akurasi (\%) = 100\% - Error\%$$

Hasil pengujian sensor LiDAR VL53L1X ditunjukkan pada Tabel 4.4

Tabel 4. 4 Hasil pengujian sensor LiDAR VL53L1X

No	Jarak Asli	Pembacaan Lidar Lewat Terminal Monitor Arduino IDE	Error(%)
1	10 cm	10 cm	0%
2	20 cm	20 cm	0%
3	30 cm	27,6 cm	8%
4	40 cm	38 cm	5%
5	50 cm	48,8 cm	2,4%
6	60 cm	57,7 cm	3,83%
7	70 cm	67,5 cm	3,5%
8	80 cm	77,6 cm	3%
9	90 cm	87,7 cm	2,5%
10	100 cm	96,7 cm	3,3%

11	110 cm	106,4 cm	3,27%
12	120 cm	120,2 cm	0.16%
13	130 cm	128,8 cm	0.92%
14	140 cm	139,3 cm	0.5%
15	150 cm	149,3 cm	0.46%
16	160 cm	158,1 cm	1.18%
17	170 cm	164, 9 cm	3%
18	180 cm	178,8 cm	0.6%
19	190 cm	191,7 cm	0.89%
20	200 cm	209,6 cm	4.8%
21	210 cm	214,6 cm	2,19%
22	220 cm	221,9 cm	0.86%
23	230 cm	228 cm	0,86%
24	240 cm	237 cm	1,25%
25	250 cm	247 cm	1,2%
26	260 cm	258,8 cm	0,46%
27	270 cm	269,2 cm	0,296%
28	280 cm	284,5 cm	1,61%
29	290 cm	291,1 cm	0.37%
30	300 cm	308,4 cm	2,8%
31	310 cm	312,3 cm	0,74%
32	320 cm	317,9 cm	0,65%
33	330 cm	330,1 cm	0,03%
34	340 cm	336,2 cm	1,12%
35	350 cm	350,5 cm	0,14%
36	360 cm	359,1 cm	0,25%
37	370 cm	363,8 cm	1,67%
38	380 cm	374,8 cm	1,37%
39	390 cm	396,7 cm	1,71%
40	400 cm	405,5 cm	1,37%
Total rata-rata error			1,73%

1. Rata-rata Error

$$\begin{aligned} \text{Rata - rata Error} &= \frac{\Sigma \text{Error}}{n} \\ \text{Rata - rata Error} &= \frac{69,036}{40} = 1,73\% \end{aligned}$$

2. Rata-rata Akurasi

$$\begin{aligned} \text{Akurasi} &= 100\% - \text{Error} \\ \text{Akurasi} &= 100\% - 1,73\% \\ \text{Akurasi} &= 98,27\% \end{aligned}$$

3. Perkiraan Selisih Error dalam cm

$$\begin{aligned} \text{Jarak rata - rata} &= \frac{\text{jarak minimum} - \text{jarak maksimum}}{2} \\ &= \frac{10 + 400}{2} = 205 \text{ cm} \end{aligned}$$

Maka:

$$\text{Error (cm)} = \frac{1,73}{100} \times 205 = 3,55 \text{ cm}$$

$$\text{Rata - rata selisih pengukuran} \approx 3,55 \text{ cm}$$

Berdasarkan hasil pengujian sebanyak 40 kali pada rentang jarak 10 cm hingga 400 cm, diperoleh rata-rata persentase error sebesar 1,73%. Berdasarkan nilai tersebut, tingkat akurasi sensor LiDAR VL53L1X yang digunakan pada sistem mencapai 98,27%. Jika dikonversikan terhadap rata-rata jarak pengujian, selisih pengukuran yang dihasilkan sensor berada pada kisaran 3,55 cm. Hasil ini menunjukkan bahwa sensor LiDAR VL53L1X memiliki tingkat akurasi yang cukup baik sebagai pendeteksi.

4.2.4 Hasil Pengujian Modul Getar

```
Message (Enter to send message to 'ESP32 Dev Module' on 'COM5')

[DANGER] Objek sangat dekat! Menjalankan getar 2x panjang.
[LiDAR] Jarak Terbaca: 328 mm (32.8 cm)
[LiDAR] Jarak Terbaca: 256 mm (25.6 cm)
[LiDAR] Jarak Terbaca: 228 mm (22.8 cm)
[LiDAR] Jarak Terbaca: 230 mm (23.0 cm)
[DANGER] Objek sangat dekat! Menjalankan getar 2x panjang.
[LiDAR] Jarak Terbaca: 283 mm (28.3 cm)
[LiDAR] Jarak Terbaca: 366 mm (36.6 cm)
[LiDAR] Jarak Terbaca: 457 mm (45.7 cm)

[WARNING] Objek mendekat! Menjalankan getar 1x sedang.
[LiDAR] Jarak Terbaca: 985 mm (98.5 cm)
[LiDAR] Jarak Terbaca: 1446 mm (144.6 cm)
[LiDAR] Jarak Terbaca: 1904 mm (190.4 cm)
[CAUTION] Ada objek di depan. Menjalankan getar 1x pendek.
[LiDAR] Jarak Terbaca: 2348 mm (234.8 cm)
```

Gambar 4. 2 Hasil Pengujian Modul Getar Pada Serial Monitor

Pengujian modul getar dilakukan untuk mengetahui respons motor getar terhadap perubahan jarak yang dibaca oleh sensor LiDAR VL53L1X. Pengujian dilakukan dengan mendekatkan objek pada beberapa rentang jarak yang telah ditentukan, kemudian mengamati pola getaran yang dihasilkan oleh sistem. Hasil pengujian ditampilkan melalui Serial Monitor Arduino IDE seperti yang ditunjukkan pada Gambar 4.2.

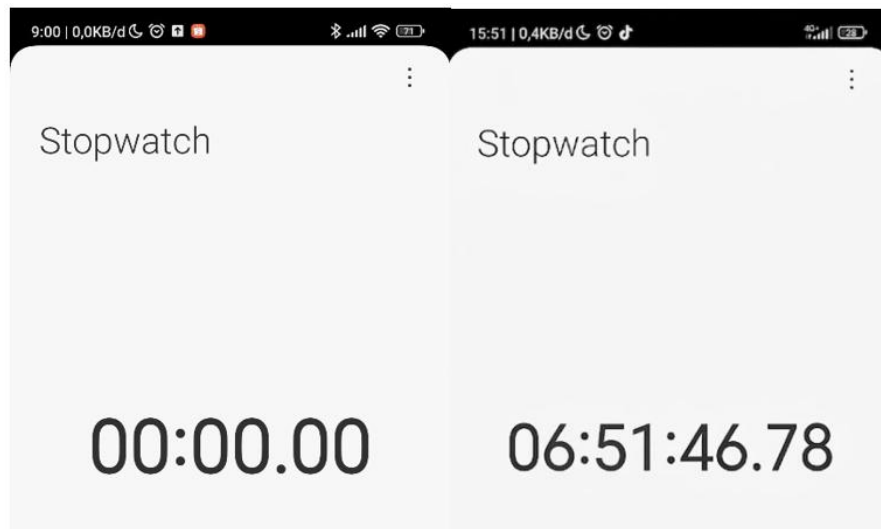
Berdasarkan hasil pengujian, motor getar mampu memberikan peringatan yang berbeda sesuai dengan jarak hambatan yang terdeteksi. Pada jarak objek lebih dari 2 meter, sistem menjalankan satu getaran dengan interval pendek pada motor getar. Ketika objek berada pada rentang 1–2 meter, sistem memberikan peringatan berupa satu getaran dengan interval sedang sebagai tanda adanya hambatan di depan pengguna. Sementara itu, saat objek berada pada jarak kurang dari 50 cm, motor getar menghasilkan dua getaran berturut-turut dengan interval yang lebih cepat sebagai peringatan bahwa hambatan berada sangat dekat dengan pengguna.

Hasil pengujian menunjukkan bahwa modul getar dapat bekerja sesuai dengan logika yang telah dirancang dan mampu memberikan umpan balik taktil berdasarkan tingkat kedekatan objek.

4.2.5 Hasil Pengujian Daya Tahan Batrai Hardware

Pengujian daya tahan baterai hardware dilakukan untuk mengetahui lama waktu operasional perangkat Smart-Cane ketika digunakan secara normal. Pengujian dimulai dari kondisi baterai terisi penuh (100%) hingga baterai habis dan perangkat tidak dapat beroperasi. Pengujian dilakukan dengan mengaktifkan seluruh komponen utama, yaitu ESP32, sensor LiDAR VL53L1X, modul Bluetooth, dan motor getar yang terhubung dengan aplikasi Android.

Baterai yang digunakan terdiri dari dua buah baterai Li-Ion 18650 berkapasitas 2500 mAh yang dipasang secara paralel pada Battery Shield V3, sehingga menghasilkan tegangan nominal tetap sebesar 3,7 V dengan kapasitas total sekitar 5000 mAh. Tegangan keluaran dari Battery Shield V3 kemudian dinaikkan menjadi 5 V melalui rangkaian boost converter untuk mensuplai ESP32 melalui pin VIN. Selama pengujian, sistem dijalankan pada kondisi normal, yaitu sensor LiDAR melakukan pembacaan jarak secara kontinu, komunikasi Bluetooth aktif, dan motor getar bekerja ketika mendeteksi hambatan.



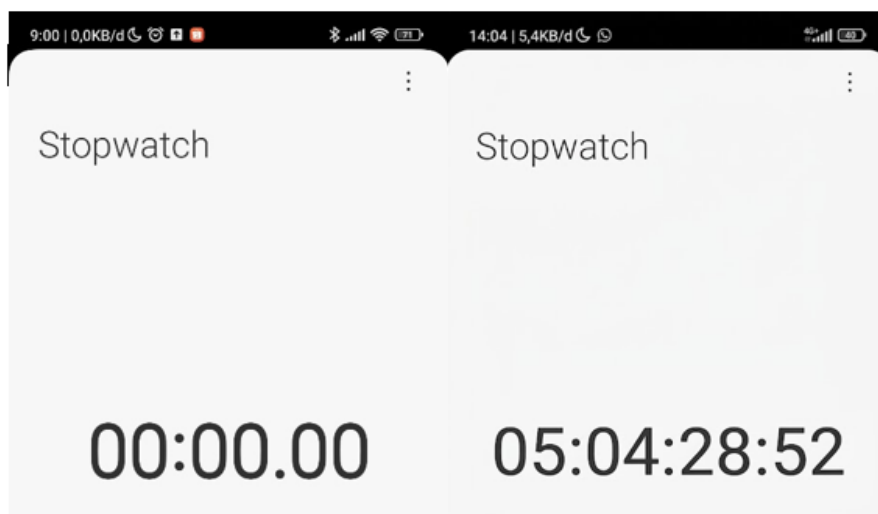
Gambar 4. 3 Lama Daya Tahan Batrai

Berdasarkan hasil pengujian yang ditunjukkan pada Gambar 4.3 Pengujian dilakukan dari jam 9.00 sampai jam 15.51 didapatkan bahwa sistem mampu beroperasi selama 6 jam 51 menit dengan menggunakan dua baterai Li-Ion 18650 berkapasitas total 5000 mAh. Selama pengujian, sensor LiDAR, ESP32,

Bluetooth, dan motor getar berada dalam kondisi aktif sehingga hasil yang diperoleh merepresentasikan penggunaan sistem secara normal. Pengukuran waktu operasional dilakukan menggunakan aplikasi stopwatch yang dijalankan sejak baterai berada pada kondisi penuh hingga baterai habis. Kondisi baterai penuh ditandai dengan menyala seluruh empat lampu indikator LED pada *Battery Shield V3* yang menunjukkan kapasitas baterai telah mencapai 100%. Sementara itu, kondisi baterai habis ditandai dengan padamnya seluruh lampu indikator LED pada *Battery Shield V3* sehingga sistem tidak lagi mampu menyuplai daya secara normal. Dengan metode tersebut, diperoleh waktu operasional perangkat selama 6 jam 51 menit dalam satu kali siklus penggunaan baterai.

4.2.6 Hasil Pengujian Daya Tahan Batrai Sistem Aplikasi

Pengujian daya tahan baterai sistem aplikasi dilakukan untuk mengetahui lama waktu penggunaan aplikasi Smart-Cane pada smartphone Android. Pengujian dilakukan dengan mengaktifkan seluruh fitur utama aplikasi, meliputi deteksi objek menggunakan kamera dan model MobileNet SSD, komunikasi Bluetooth dengan ESP32, layanan GPS, navigasi berbasis OSRM, serta fitur Text-to-Speech (TTS). Pengujian menggunakan smartphone Redmi Note 10 Pro yang memiliki baterai berkapasitas 5.020 mAh. Pengujian dimulai saat baterai smartphone berada pada kondisi penuh (100%) dan dihentikan ketika kapasitas baterai mencapai batas minimum penggunaan.



Gambar 4. 4 Lama Daya Tahan Batrai Aplikasi

Berdasarkan hasil pengujian yang ditunjukkan pada Gambar 4.4, aplikasi *Smart-Cane* mampu beroperasi selama 5 jam 4 menit 28 detik. Waktu tersebut diperoleh menggunakan aplikasi stopwatch yang dijalankan bersamaan dengan pengoperasian aplikasi *Smart-Cane*.

Hasil pengujian menunjukkan bahwa konsumsi daya terbesar berasal dari penggunaan kamera secara *real-time* untuk proses deteksi objek serta layanan GPS yang aktif secara berkelanjutan. Selain itu, pencahayaan layar yang full, komunikasi Bluetooth dengan ESP32 dan penggunaan *Text-to-Speech* juga turut menambah konsumsi daya *smartphone*. Meskipun demikian, aplikasi masih dapat digunakan selama lima jam, sehingga dinilai cukup untuk mendukung aktivitas mobilitas pengguna dalam penggunaan sehari-hari.

4.2.7 Hasil Persiapan Dataset

Dataset yang digunakan dalam penelitian ini merupakan gabungan antara dataset *Common Objects in Context* (COCO) dan dataset kustom yang dibuat menggunakan platform Roboflow. Penggabungan kedua dataset dilakukan untuk memenuhi kebutuhan delapan kelas objek yang digunakan dalam sistem tingkat pintar. Dataset COCO dipilih karena menyediakan berbagai objek umum yang sering dijumpai di lingkungan sehari-hari, sedangkan dataset kustom digunakan untuk menambahkan kelas objek yang tidak tersedia pada COCO, seperti lubang, tembok, dan pohon. Penambahan kelas tersebut dilakukan karena objek-objek tersebut berpotensi menjadi rintangan yang perlu dikenali oleh sistem untuk membantu navigasi penyandang tunanetra.

Setelah proses pengumpulan dan anotasi data selesai, seluruh dataset dibagi menjadi tiga subset dengan rasio 80% data pelatihan (*train*), 15% data validasi (*validation*), dan 5% data pengujian (*test*). Pembagian ini bertujuan untuk memastikan setiap subset memiliki distribusi data yang representatif sehingga model dapat dilatih, divalidasi, dan diuji secara optimal. Total dataset yang digunakan terdiri dari 16.045 gambar dengan 30.947 anotasi objek, yang terbagi menjadi 15.003 gambar pelatihan dengan 28.456 anotasi dan 1.042 gambar validasi dengan 2.491 anotasi.

Tabel 4. 5 Jumlah Dataset

Subset Dataset	Jumlah Gambar	Jumlah Anotasi
<i>Train</i>	15.003	28.456
<i>Validation</i>	1.042	2.491
Total	16.045	30.947

Tiga kelas kustom (lubang, tembok, dan pohon) sepenuhnya bersumber dari dataset Roboflow yang dianotasi secara manual, sementara 5 kelas lainnya (orang, mobil, motor, sepeda, bangku) merupakan gabungan dari dataset COCO dan dataset Roboflow tambahan. Proses anotasi pada Roboflow menggunakan format Pascal VOC yang kemudian dikonversi ke format COCO JSON untuk proses penggabungan dataset. Augmentasi data sebesar 3x diterapkan pada dataset Roboflow, meliputi flip horizontal, rotasi $\pm 15^\circ$, penyesuaian brightness, dan cropping, menghasilkan total 12.889 gambar dari 4.957 gambar asli.

Tabel 4. 6 Distribusi Jumlah Anotasi pada Setiap Kelas Objek

No	Kelas Objek	Data <i>Train</i>	Data Validasi
1	Bangku	3.356	267
2	Lubang	3.771	204
3	Mobil	3.655	376
4	Motor	3.340	297
5	Orang	3.556	591
6	Pohon	3.714	199
7	Sepeda	3.303	317
8	Tembok	3.761	240
Total		28.456	2.491

Tabel di atas menunjukkan distribusi jumlah anotasi pada masing-masing kelas objek yang digunakan dalam penelitian. Dataset terdiri dari delapan kelas objek, yaitu bangku, lubang, mobil, motor, orang, pohon, sepeda, dan tembok. Jumlah anotasi pada data pelatihan sebanyak 28.456 anotasi, sedangkan data

validasi sebanyak 2.491 anotasi. Distribusi data yang relatif seimbang pada setiap kelas dibuat agar model MobileNet SSD dapat mempelajari karakteristik objek secara lebih optimal selama proses pelatihan.

4.2.8 Hasil *Training* MobileNet SSD

Proses pelatihan model deteksi objek dilakukan menggunakan arsitektur MobileNet SSD V2 dengan ukuran masukan citra sebesar 300×300 piksel. Pelatihan dilaksanakan menggunakan TensorFlow Object Detection API versi 2.10.0 pada perangkat dengan prosesor Intel Core i5 Generasi ke-11 tanpa menggunakan akselerasi GPU. Model dilatih secara bertahap hingga mencapai 150.000 *steps* dengan *batch* size 4.

4.2.8.1 Konfigurasi Model MobileNet SSD

Sebelum proses pelatihan dilakukan, model deteksi objek terlebih dahulu dikonfigurasi menggunakan file `pipeline.config` yang disediakan oleh TensorFlow Object Detection API. File konfigurasi ini berfungsi untuk mengatur arsitektur model, parameter pelatihan (*training*), proses evaluasi, teknik augmentasi data, serta lokasi dataset yang digunakan selama proses pelatihan.

```
model {
  ssd {
    num_classes: 8
    image_resizer {
      fixed_shape_resizer {
        height: 300
        width: 300
      }
    }
  }
}
```

Gambar 4. 5 Konfigurasi Model Mobilenet SSD V2

Berdasarkan konfigurasi tersebut, model menggunakan SSD MobileNet V2 dengan jumlah kelas (*num_classes*) sebanyak 8 kelas, yaitu orang, mobil, motor, sepeda, bangku, lubang, tembok, dan pohon. Seluruh citra masukan

(*input image*) diubah menjadi ukuran 300×300 piksel sebelum diproses oleh model sehingga ukuran input menjadi seragam selama proses pelatihan maupun inferensi.

Selanjutnya dilakukan konfigurasi parameter pelatihan yang meliputi ukuran *batch*, jumlah langkah pelatihan (*training steps*), *optimizer*, serta model awal (*pre-trained model*) yang digunakan.

```
r'batch_size: \d+' : 'batch_size: 4',
r'num_steps: \d+' : 'num_steps: 150000',
r'fine_tune_checkpoint: ".*?"' : 'fine_tune_checkpoint: '
'"ssd_mobilenet_v2_320x320_coco17_tpu-8/checkpoint/ckpt-0"',
```

Gambar 4. 6 Konfigurasi *Training*

Berdasarkan konfigurasi tersebut, proses pelatihan menggunakan *batch size* sebesar 4 dengan total 150.000 langkah pelatihan (*training steps*). Model juga menerapkan metode transfer learning dengan memanfaatkan model awal (*pre-trained model*) SSD MobileNet V2 COCO17 TPU-8, sehingga proses pelatihan tidak dimulai dari nol, tetapi melanjutkan pembelajaran dari bobot model yang telah dilatih sebelumnya

Selain itu, digunakan *Momentum Optimizer* dengan metode *Cosine Decay Learning rate* sebagai algoritma optimasi selama proses pelatihan.

```
optimizer {
  momentum_optimizer {
    learning_rate {
      cosine_decay_learning_rate {
        learning_rate_base: 0.02
```

Gambar 4. 7 Konfigurasi *Optimizer* dengan *Cosine Decay*

Penggunaan metode *Cosine Decay Learning rate* bertujuan untuk menurunkan nilai *Learning rate* secara bertahap selama proses pelatihan sehingga model dapat mencapai kondisi *konvergen* dengan lebih stabil dan mengurangi kemungkinan terjadinya *overfitting*.

Untuk meningkatkan kemampuan model dalam mengenali objek diterapkan beberapa teknik data augmentation selama proses pelatihan.

```
data_augmentation_options {
  random_horizontal_flip {
  }
}
data_augmentation_options {
  ssd_random_crop {
  }
}
data_augmentation_options {
  random_adjust_brightness {
    max_delta: 0.2
  }
}
data_augmentation_options {
  random_adjust_contrast {
    min_delta: 0.8
    max_delta: 1.25
  }
}
```

Gambar 4. 8 Augmentasi yang Digunakan

Teknik augmentasi yang digunakan meliputi *random horizontal flip*, *random crop*, *random brightness*, dan *random contrast*. Penerapan augmentasi ini bertujuan untuk menambah variasi data pelatihan sehingga model menjadi lebih *robust* terhadap perubahan orientasi objek, kondisi pencahayaan, serta variasi kontras gambar.

Selanjutnya dilakukan konfigurasi dataset pelatihan dan validasi dalam format TFRecord yang digunakan oleh TensorFlow Object Detection API.

```
train_input_reader {
  label_map_path: "label_map.pbtxt"
  tf_record_input_reader {
    input_path: "tfrecords/train.record"
```

Gambar 4. 9 Konfigurasi `train_input_reader` pada file `pipeline.config`.

Dataset yang digunakan merupakan hasil penggabungan (*merge*) antara dataset COCO dan Roboflow yang telah melalui proses penyamaan nama kelas (*class name*) menjadi Bahasa Indonesia. Dataset tersebut kemudian dikonversi ke dalam format TFRecord, dengan *train.record* digunakan sebagai data pelatihan, *valid.record* sebagai data validasi, serta *label_map.pbtxt* sebagai pemetaan kelas yang akan dikenali oleh model.

Dengan konfigurasi tersebut, model SSD MobileNet V2 siap digunakan untuk proses pelatihan hingga menghasilkan model deteksi objek yang selanjutnya diekspor ke format TensorFlow Lite (.tflite) agar dapat diimplementasikan pada aplikasi *Smart-Cane* berbasis Android.

4.2.8.2 Hasil *Loss* Pelatihan

Tabel menunjukkan perkembangan nilai *loss* selama proses pelatihan model.

Tabel 4. 7 Hasil *Loss* Pelatihan Model MobileNet SSD

<i>Steps</i>	<i>Localization Loss</i>	<i>Classification Loss</i>	<i>Regularization Loss</i>	<i>Total Loss</i>
50.000	0,259	0,467	0,135	0,861
80.000	0,250	0,441	0,133	0,824
100.000	0,235	0,445	0,132	0,812
150.000	0,236	0,439	0,129	0,803

Berdasarkan Tabel 4.7 nilai total *loss* mengalami penurunan dari 0,861 pada 50.000 *steps* menjadi 0,803 pada 150.000 *steps*. Penurunan ini menunjukkan bahwa model semakin mampu menyesuaikan parameter terhadap data pelatihan. Selain itu, nilai *Regularization loss* juga mengalami penurunan secara bertahap hingga akhir pelatihan, yang mengindikasikan bahwa kompleksitas model semakin terkendali dan kemampuan generalisasi model terhadap data baru menjadi lebih baik.

4.2.8.3 Hasil Evaluasi mAP

Evaluasi performa model dilakukan menggunakan metrik mean *Average Precision* (mAP) yang umum digunakan pada sistem deteksi objek.

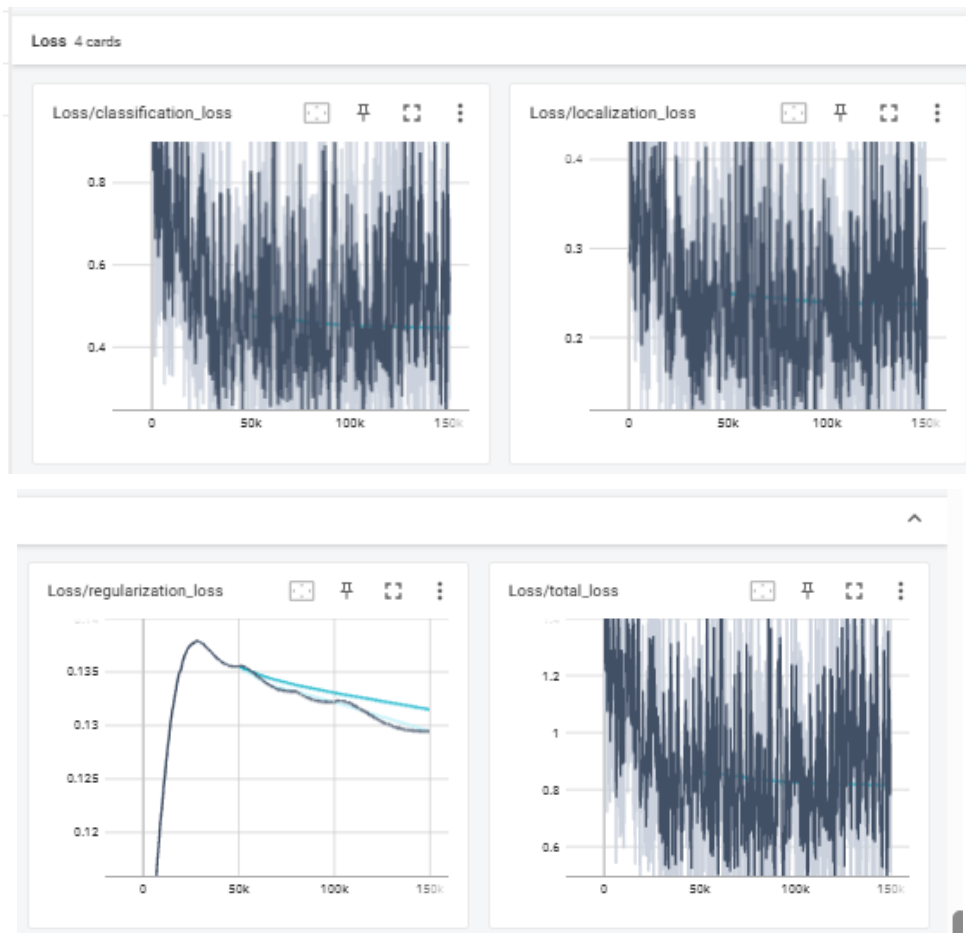
Tabel 4. 8 Hasil Evaluasi mAP Selama Pelatihan

<i>Steps</i>	mAP	mAP@50	mAP@75
50.000	29,2%	50,1%	31,2%
80.000	30,6%	51,3%	32,4%
100.000	31,1%	51,8%	33,4%
150.000	31,0%	52,5%	32,7%

Hasil evaluasi menunjukkan bahwa performa model mengalami peningkatan seiring bertambahnya jumlah langkah pelatihan. Pada model akhir yang digunakan dalam penelitian ini, diperoleh nilai mAP sebesar 31,0%, mAP@50 sebesar 52,5%, dan mAP@75 sebesar 32,7%. Dibandingkan dengan hasil pelatihan awal, nilai mAP meningkat sebesar 8,6 poin persentase, sedangkan nilai mAP@50 meningkat sebesar 12,9 poin persentase. Peningkatan tersebut menunjukkan bahwa proses pelatihan berhasil meningkatkan kemampuan model dalam mendeteksi dan mengklasifikasikan objek pada dataset yang digunakan.

Namun, setelah mencapai 100.000 *steps*, peningkatan performa model cenderung sangat kecil. Nilai mAP hanya berubah dari 31,1% pada 100.000 *steps* menjadi 31,0% pada 150.000 *steps*, sedangkan nilai mAP@50 meningkat dari 51,8% menjadi 52,5%. Hasil ini menunjukkan bahwa model telah mendekati kondisi *konvergen*, yaitu kondisi ketika penambahan jumlah langkah pelatihan tidak lagi memberikan peningkatan performa yang signifikan. Oleh karena itu, proses pelatihan tidak dilanjutkan melebihi 150.000 training steps karena diperkirakan hanya akan meningkatkan waktu komputasi tanpa memberikan peningkatan akurasi.

4.2.8.4 Grafik *Training*



Gambar 4. 10 Grafik Loss Evaluation Menggunakan Platform TensorBoard pada Proses Training 150k Steps.

Selama proses pelatihan, pemantauan performa model dilakukan menggunakan TensorBoard. Grafik yang diamati meliputi *Classification loss*, *Localization loss*, *Regularization loss*, *total loss*, dan *learning rate*.

Berdasarkan grafik TensorBoard, nilai *Classification loss* dan *Localization loss* menunjukkan pola fluktuatif pada setiap langkah pelatihan, namun memiliki kecenderungan menurun secara konsisten. Pola tersebut merupakan karakteristik yang umum terjadi pada pelatihan model deteksi objek dengan ukuran *batch* yang relatif kecil. Sementara itu, *Regularization loss* mengalami peningkatan pada tahap awal pelatihan sebelum kemudian menurun secara bertahap hingga akhir proses pelatihan. Kondisi ini menunjukkan bahwa model dapat mempelajari pola dari dataset sehingga kemampuan deteksinya semakin meningkat selama proses pelatihan.

4.2.9 Hasil Evaluasi Model

Tabel 4. 9 Hasil *Average Precision* (AP) Model MobileNet SSD per Kelas

No	Nama Kelas	AP (mAP)	AP@50	AP@75	AP <i>Small</i>	AP <i>Medium</i>	AP <i>Large</i>
1	Orang	4,0%	12,5%	1,9%	0,7%	6,7%	10,5%
2	Mobil	25,4%	45,3%	23,9%	4,9%	14,2%	52,6%
3	Motor	41,6%	68,0%	46,3%	0,1%	4,3%	54,6%
4	Sepeda	15,1%	32,6%	11,5%	0,6%	13,5%	34,2%
5	Bangku	47,8%	68,1%	54,9%	0,1%	12,6%	64,5%
6	Lubang	54,0%	83,8%	63,5%	9,3%	41,4%	64,9%
7	Tembok	26,3%	45,2%	26,6%	0,0%	3,3%	42,8%
8	Pohon	34,2%	64,4%	33,2%	0,0%	7,1%	50,9%
Rata-rata		31,0%	52,5%	32,7%	1,9%	12,9%	46,9%

Berdasarkan Tabel 4.9, model MobileNet SSD yang digunakan pada sistem *Smart-Cane* memperoleh nilai rata-rata mAP sebesar 31,0%, AP@50 sebesar 52,5%, dan AP@75 sebesar 32,7%. Hasil tersebut menunjukkan bahwa model telah mampu mendeteksi objek-objek yang digunakan dalam penelitian dengan tingkat akurasi yang cukup baik.

Kelas lubang memperoleh performa terbaik dengan nilai AP sebesar 54,0%, AP@50 sebesar 83,8%, dan AP@75 sebesar 63,5%. Hasil ini menunjukkan bahwa model mampu mengenali objek lubang dengan lebih baik dibandingkan kelas lainnya. Selain itu, kelas bangku dan motor juga menunjukkan performa yang baik dengan nilai AP@50 masing-masing sebesar 68,1% dan 68,0%.

Sebaliknya, kelas orang memiliki nilai AP terendah, yaitu sebesar 4,0%. Rendahnya performa pada kelas ini diduga disebabkan oleh tingginya variasi bentuk, ukuran, pose, dan kondisi pencahayaan pada objek manusia. Sementara itu, nilai rata-rata AP *Small* hanya mencapai 1,9%, yang menunjukkan bahwa model masih mengalami kesulitan dalam mendeteksi objek berukuran kecil. Kondisi ini dipengaruhi oleh penggunaan resolusi input 300×300 piksel pada arsitektur MobileNet SSD sehingga informasi visual objek kecil menjadi lebih sulit dikenali oleh model.

Tabel 4. 10 Hasil *Average Recall* (AR) Model MobileNet SSD per Kelas

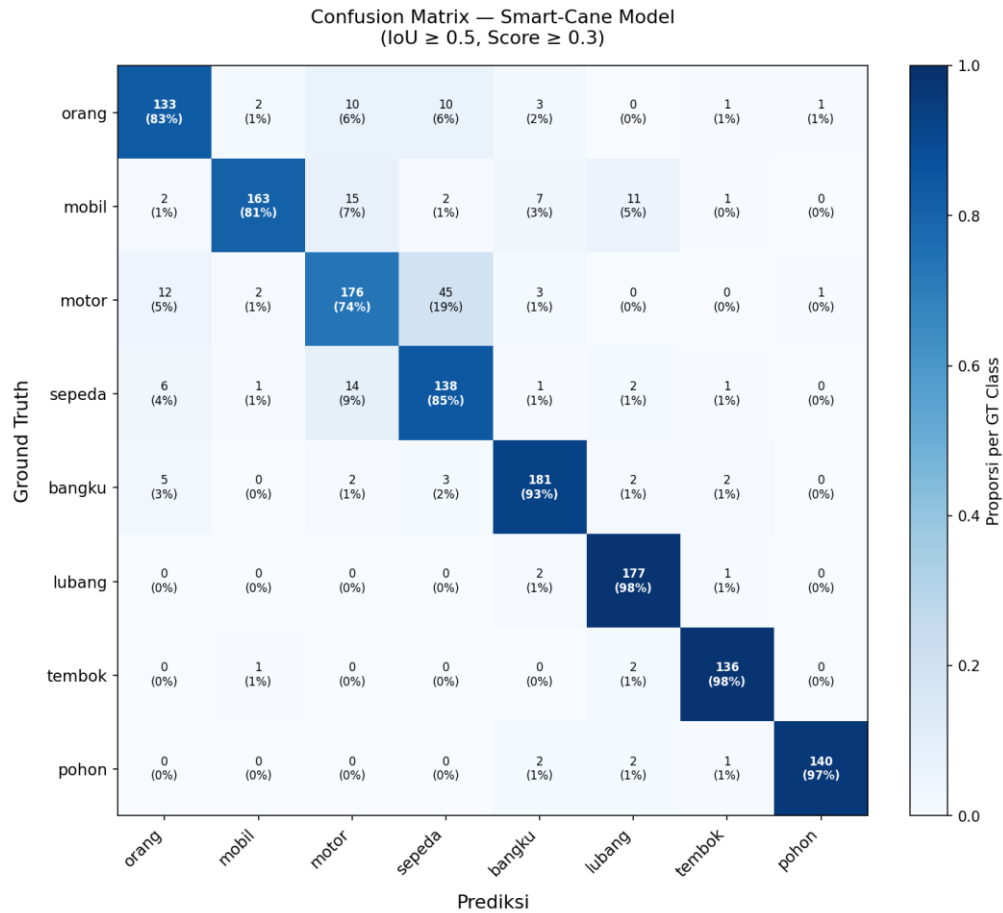
No	Nama Kelas	AR@1	AR@10	AR@100	AR <i>Small</i>	AR Medium	AR Large
1	Orang	4,2%	12,0%	16,7%	3,0%	23,8%	46,8%
2	Mobil	26,2%	33,4%	36,3%	6,4%	34,0%	66,6%
3	Motor	40,6%	52,8%	54,9%	1,2%	25,8%	67,6%
4	Sepeda	15,8%	27,1%	31,4%	4,2%	31,2%	59,1%
5	Bangku	47,6%	56,4%	57,9%	9,5%	25,5%	74,4%
6	Lubang	55,4%	64,6%	65,0%	12,0%	55,2%	75,0%
7	Tembok	15,4%	43,1%	48,6%	0,0%	26,6%	68,5%
8	Pohon	30,1%	45,6%	50,5%	0,0%	27,9%	65,9%
Rata-rata		29,4%	41,9%	45,2%	4,5%	31,3%	65,5%

Tabel 4.10 menunjukkan hasil evaluasi *Average Recall* (AR) yang digunakan untuk mengukur kemampuan model dalam menemukan objek yang terdapat pada data validasi. Secara keseluruhan, model memperoleh nilai rata-rata AR@100 sebesar 45,2%.

Kelas lubang memperoleh nilai AR@100 tertinggi sebesar 65,0%, diikuti oleh kelas bangku sebesar 57,9% dan motor sebesar 54,9%. Hasil tersebut menunjukkan bahwa model mampu menemukan sebagian besar objek pada kelas-kelas tersebut selama proses evaluasi. Selain itu, nilai rata-rata AR Large mencapai 65,5%, yang menunjukkan bahwa model memiliki kemampuan deteksi yang lebih baik pada objek berukuran besar dibandingkan objek berukuran kecil.

Sementara itu, nilai rata-rata AR *Small* hanya mencapai 4,5%, yang menunjukkan bahwa kemampuan model dalam menemukan objek berukuran kecil masih terbatas. Kelas orang memiliki nilai AR@100 terendah sebesar 16,7%, yang menunjukkan bahwa masih terdapat objek manusia yang tidak berhasil terdeteksi pada data validasi. Secara umum, hasil evaluasi *Recall* menunjukkan bahwa model telah mampu mendeteksi sebagian besar objek

utama yang digunakan dalam penelitian, terutama objek yang memiliki ukuran relatif besar dan karakteristik visual yang konsisten.



Gambar 4. 11 Confusion Matrix

Berdasarkan Gambar 4.11, *confusion matrix* digunakan untuk menganalisis kemampuan model dalam mengklasifikasikan delapan kelas objek, yaitu orang, mobil, motor, sepeda, bangku, lubang, tembok, dan pohon. Nilai pada diagonal utama menunjukkan jumlah objek yang berhasil dikenali dengan benar (*True Positive*), sedangkan nilai di luar diagonal menunjukkan kesalahan klasifikasi (*misClassification*) antar kelas.

Secara umum, model menunjukkan kemampuan deteksi yang cukup baik pada sebagian besar kelas. Hal ini terlihat dari dominasi nilai pada diagonal utama yang lebih besar dibandingkan nilai di luar diagonal. Kelas lubang dan tembok memiliki hasil deteksi terbaik dengan tingkat keberhasilan masing-masing sebesar 98%, sedangkan kelas bangku memiliki tingkat keberhasilan sebesar 93%.

Tabel 4. 11 Metrik *Precision* dan *Recall* Berdasarkan *Confusion matrix*

Kelas	TP	FP	FN	<i>Precision</i> (%)	<i>Recall</i> (%)	<i>Accuracy</i> (%)	<i>F1-Score</i> (%)
Orang	133	826	458	13,9	22,5	79,57	17,2
Mobil	163	593	213	21,6	43,4	87,55	28,8
Motor	176	407	121	30,2	59,3	91,60	40,0
Sepeda	138	1008	179	12,0	43,5	81,11	18,8
Bangku	181	349	86	34,2	67,8	93	45,4
Lubang	177	236	27	42,9	86,8	95, 82	57,4
Tembok	136	317	104	30,0	56,7	93,3	39,2
Pohon	140	392	59	26,3	70,4	93	38,3

Berdasarkan Tabel 4.11, nilai *Precision* , *Recall*, *Accuracy*, dan *F1-Score* digunakan untuk mengevaluasi performa model pada setiap kelas objek. Nilai *Precision* menunjukkan ketepatan model dalam memberikan prediksi terhadap suatu kelas, *Recall* menunjukkan kemampuan model dalam menemukan seluruh objek yang benar pada kelas tersebut, sedangkan *Accuracy* digunakan untuk mengetahui tingkat ketepatan prediksi model secara keseluruhan pada masing-masing kelas. *F1-Score* merupakan rata-rata harmonis antara *Precision* dan *Recall* yang digunakan untuk menggambarkan keseimbangan performa model dalam mendeteksi suatu kelas objek.

Dilakukan perhitungan pada tiap kelas berdasarkan data pada *confusion matrix* sebagai berikut :

Kelas orang

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 133 - 826 - 458$$

$$TN = 4868$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$Accuracy = \frac{133 + 4868}{133 + 4868 + 826 + 458} \times 100\%$$

$$= \frac{5001}{6285} \times 100\% = 79,57\%$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 79,57% untuk kelas orang

2. Perhitungan *Precision*

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

$$= \frac{133}{133 + 826} \times 100\% = 13,87 \%$$

Nilai *Precision* sebesar 13,87% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai orang, sebanyak 13,87% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{133}{133 + 458} = 22,5 \%$$

Nilai *Recall* sebesar 22,5% menunjukkan bahwa model mampu mendeteksi 22,5% dari seluruh objek orang yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 13,9 \times 22,5}{13,9 + 22,5} = 17,2 \%$$

Nilai *F1-Score* sebesar 17,2 % menunjukkan bahwa keseimbangan antara *Precision* dan *Recall* pada kelas orang masih rendah.

Kelas mobil

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 163 - 593 - 213$$

$$TN = 5506$$

1. Perhitungan *Accuracy*

$$\begin{aligned} Accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \times 100\% \\ Accuracy &= \frac{163 + 5506}{163 + 5506 + 593 + 213} \times 100\% \\ &= \frac{5669}{6285} \times 100\% = 87,55\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 87,55% untuk kelas mobil

2. Perhitungan *Precision*

$$\begin{aligned} Precision &= \frac{TP}{TP + FP} \times 100\% \\ &= \frac{163}{163 + 593} \times 100\% = 21,5\% \end{aligned}$$

Nilai *Precision* sebesar 21,56% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai mobil, sebanyak 21,56% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{133}{163 + 213} = 43,35\%$$

Nilai *Recall* sebesar 43,35% menunjukkan bahwa model mampu mendeteksi 43,35% dari seluruh objek mobil yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$\begin{aligned} F1 - Score &= \frac{2 \times Precision \times Recall}{Precision + Recall} \\ F1 - Score &= \frac{2 \times 21,6 \times 43,4}{21,6 + 43,4} = 28,8\% \end{aligned}$$

Nilai *F1-Score* sebesar 28,8% menunjukkan keseimbangan kemampuan model dalam mendeteksi objek mobil.

Kelas motor

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 176 - 407 - 121$$

$$TN = 5581$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{176 + 5581}{176 + 5581 + 407 + 121} \times 100\% \\ &= \frac{5757}{6285} \times 100\% = 91,60\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 91,60% untuk kelas motor

2. Perhitungan *Precision*

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

$$= \frac{176}{176 + 407} \times 100\% = 30,19 \%$$

Nilai *Precision* sebesar 30,19% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai motor, sebanyak 30,19% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{176}{176 + 121} = 59,2 \%$$

Nilai *Recall* sebesar 59,2% menunjukkan bahwa model mampu mendeteksi 59,2% dari seluruh objek motor yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 30,2 \times 59,3}{30,2 + 59,3} = 40 \%$$

Nilai *F1-Score* sebesar 40 % menunjukkan kemampuan model cukup seimbang dalam mendeteksi objek motor.

Kelas sepeda

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 138 - 1008 - 179$$

$$TN = 4960$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{138 + 4960}{138 + 4960 + 1008 + 179} \times 100\% \\ &= \frac{5098}{6285} \times 100\% = 81,11\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 81,11% untuk kelas sepeda

2. Perhitungan *Precision*

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

$$= \frac{138}{138 + 1008} \times 100\% = 12,04 \%$$

Nilai *Precision* sebesar 12,04% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai sepeda, sebanyak 12,04% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{138}{138 + 179} = 43,53 \%$$

Nilai *Recall* sebesar 43,53% menunjukkan bahwa model mampu mendeteksi 43,53% dari seluruh objek sepeda yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 12 \times 43,5}{12 + 43,5} = 18,8 \%$$

Nilai *F1-Score* sebesar 18,8% menunjukkan keseimbangan *Precision* dan *Recall* pada kelas sepeda masih rendah.

Kelas Bangku

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 181 - 349 - 86$$

$$TN = 5669$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{181 + 5669}{181 + 5669 + 349 + 86} \times 100\% \\ &= \frac{5850}{6285} \times 100\% = 93\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 93% untuk kelas bangku

2. Perhitungan *Precision*

$$\begin{aligned} Precision &= \frac{TP}{TP + FP} \times 100\% \\ &= \frac{181}{181 + 349} \times 100\% = 34,15\% \end{aligned}$$

Nilai *Precision* sebesar 34,15% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai bangku, sebanyak 34,15% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{181}{181 + 86} = 67,8\%$$

Nilai *Recall* sebesar 67,8% menunjukkan bahwa model mampu mendeteksi 67,8% dari seluruh objek bangku yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 34,2 \times 67,8}{34,2 + 67,8} = 45,4\%$$

Nilai *F1-Score* sebesar 45,4 % menunjukkan model memiliki keseimbangan yang cukup baik dalam mendeteksi objek bangku.

Kelas tembok

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 136 - 317 - 104$$

$$TN = 5728$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{136 + 5728}{136 + 5728 + 317 + 104} \times 100\% \\ &= \frac{5001}{6285} \times 100\% = 93,3\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 93,3% untuk kelas tembok

2. Perhitungan *Precision*

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

$$= \frac{136}{136 + 317} \times 100\% = 30 \%$$

Nilai *Precision* sebesar 30% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai tembok, sebanyak 30% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{136}{136 + 104} = 56,7 \%$$

Nilai *Recall* sebesar 56,7% menunjukkan bahwa model mampu mendeteksi 56,7% dari seluruh objek tembok yang sebenarnya terdapat pada data uji.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 30 \times 56,7}{30 + 56,7} = 39,2 \%$$

Nilai *F1-Score* sebesar 39,2 % menunjukkan keseimbangan kemampuan model dalam mendeteksi objek tembok.

Kelas Lubang

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 177 - 236 - 27$$

$$TN = 5845$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{177 + 5845}{177 + 5845 + 236 + 27} \times 100\% \\ &= \frac{6022}{6285} \times 100\% = 95,82\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 95,82% untuk kelas lubang, yang berarti sebagian besar data berhasil diklasifikasikan dengan benar.

2. Perhitungan *Precision*

$$\begin{aligned} Precision &= \frac{TP}{TP + FP} \times 100\% \\ &= \frac{177}{177 + 236} \times 100\% = 42,9\% \end{aligned}$$

Nilai *Precision* sebesar 42,9% menunjukkan bahwa dari seluruh objek yang diprediksi sebagai lubang, sebanyak 42,9% merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{177}{177 + 27} = 86,8\%$$

Nilai *Recall* sebesar 86,8% menunjukkan bahwa model berhasil mendeteksi 86,8% dari seluruh objek lubang yang terdapat pada data pengujian.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 42,9 \times 86,8}{42,9 + 86,8} = 57,4\%$$

Nilai *F1-Score* sebesar 57,4% menunjukkan keseimbangan *Precision* dan *Recall* terbaik dibandingkan kelas lainnya.

Kelas pohon

$$TN = 6285 - TP - FP - FN$$

$$TN = 6285 - 140 - 392 - 59$$

$$TN = 5694$$

1. Perhitungan *Accuracy*

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%$$

$$\begin{aligned} Accuracy &= \frac{140 + 5694}{140 + 5694 + 392 + 59} \times 100\% \\ &= \frac{5834}{6285} \times 100\% = 93\% \end{aligned}$$

Hasil tersebut menunjukkan bahwa model memiliki tingkat ketepatan klasifikasi sebesar 93% untuk kelas pohon

2. Perhitungan *Precision*

$$Precision = \frac{TP}{TP + FP} \times 100\%$$

$$= \frac{140}{140 + 392} \times 100\% = 26,32 \%$$

Nilai *Precision* sebesar 26,32 % menunjukkan bahwa dari seluruh objek yang diprediksi sebagai pohon, sebanyak 26,32 % merupakan prediksi yang benar.

3. Perhitungan *Recall*

$$Recall = \frac{TP}{TP + FN} = \frac{140}{140 + 59} = 70,4 \%$$

Nilai *Recall* sebesar 70,4% menunjukkan bahwa model berhasil mendeteksi 70,4% dari seluruh objek pohon yang terdapat pada data pengujian.

4. Perhitungan *F1-Score*

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$F1 - Score = \frac{2 \times 26,3 \times 70,4}{26,3 + 70,4} = 38,3\%$$

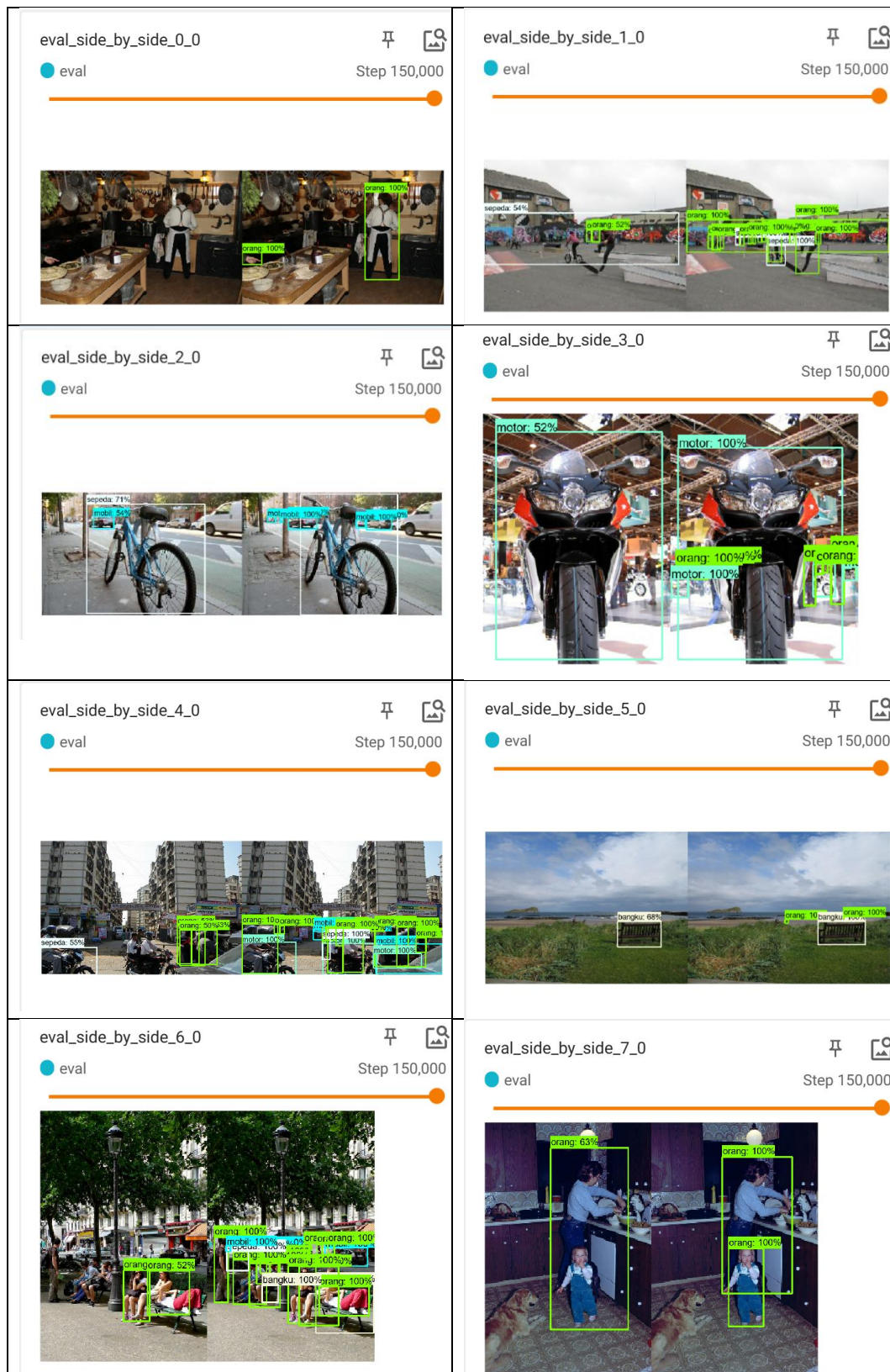
Nilai *F1-Score* sebesar 38,3% menunjukkan kemampuan model cukup seimbang dalam mendeteksi objek pohon.

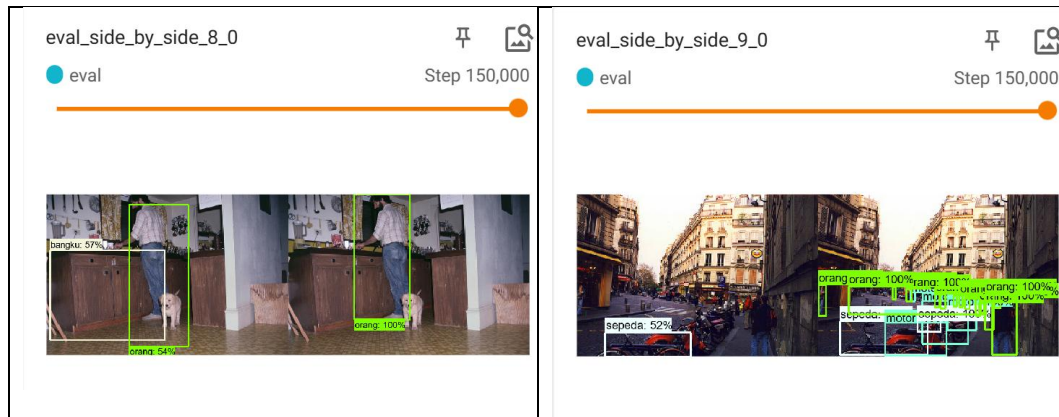
Hasil evaluasi menunjukkan bahwa kelas lubang memiliki performa terbaik dengan nilai *Precision* sebesar 42,9%, *Recall* sebesar 86,8%, *Accuracy* sebesar 95,82%, dan *F1-Score* sebesar 57,4%. Nilai tersebut menunjukkan bahwa model mampu mengenali objek lubang dengan tingkat ketepatan dan kemampuan deteksi yang paling baik dibandingkan kelas lainnya. Selain itu, kelas bangku, tembok, dan pohon juga memperoleh nilai *Accuracy* di atas 92% dengan nilai *F1-Score* masing-masing sebesar 45,4%, 39,2%, dan 38,3%, sehingga menunjukkan performa deteksi yang cukup baik.

Sebaliknya, kelas orang memperoleh nilai *Precision* sebesar 13,9%, *Recall* sebesar 22,5%, *Accuracy* sebesar 79,57%, dan *F1-Score* sebesar 17,2%, sedangkan kelas sepeda memperoleh *Accuracy* sebesar 81,11% dan *F1-Score* sebesar 18,8%. Nilai tersebut menunjukkan bahwa model masih mengalami kesalahan dalam mendeteksi kedua kelas tersebut. Kondisi ini dipengaruhi oleh kemiripan karakteristik visual antarobjek, variasi ukuran objek, perbedaan sudut pengambilan gambar, kondisi pencahayaan, serta adanya objek yang saling bertumpuk sehingga meningkatkan jumlah False Positive dan False Negative.

Secara keseluruhan, model SSD MobileNet V2 telah mampu mendeteksi seluruh delapan kelas objek yang digunakan pada penelitian, yaitu orang, mobil, motor, sepeda, bangku, lubang, tembok, dan pohon. Meskipun masih terdapat kesalahan klasifikasi pada beberapa kelas, hasil evaluasi berdasarkan nilai *Precision*, *Recall*, *Accuracy*, dan *F1-Score* menunjukkan bahwa model memiliki performa yang cukup baik dalam mendeteksi objek pada dataset yang digunakan.

Tabel 4. 12 Hasil Validasi Visual Evaluasi Model *Train* 150.000 *steps*





Selain evaluasi kuantitatif menggunakan metrik *Average Precision* (AP) dan *Average Recall* (AR), dilakukan pula analisis kualitatif terhadap hasil deteksi objek menggunakan fitur *eval_side_by_side* pada TensorBoard. Analisis ini bertujuan untuk mengamati kemampuan model dalam mengenali objek secara visual pada berbagai kondisi lingkungan. Pengamatan dilakukan pada beberapa sampel data validasi setelah model mencapai hasil pelatihan terbaik pada 150.000 *steps*.

Pada sampel *eval_side_by_side_0_0*, *eval_side_by_side_7_0*, dan *eval_side_by_side_8_0*, model berhasil mendeteksi objek orang dengan tingkat kepercayaan yang tinggi. Hasil ini menunjukkan bahwa model mampu mengenali karakteristik visual manusia pada berbagai kondisi dan posisi objek. Kemampuan tersebut penting karena keberadaan pejalan kaki merupakan salah satu objek yang perlu dikenali oleh sistem saat digunakan di lingkungan sekitar pengguna.

Pada sampel *eval_side_by_side_4_0* dan *eval_side_by_side_9_0*, model diuji pada kondisi lingkungan yang mengandung beberapa objek sekaligus, seperti orang, motor, mobil, dan sepeda. Hasil deteksi menunjukkan bahwa model mampu mengenali lebih dari satu objek dalam satu citra meskipun terdapat sebagian objek yang saling berdekatan. Kemampuan ini menunjukkan bahwa model dapat digunakan pada kondisi lingkungan luar ruangan yang memiliki kepadatan objek relatif tinggi.

3. Deteksi Objek Berukuran Besar dan Jarak Dekat

Pada sampel *eval_side_by_side_3_0*, model berhasil mendeteksi objek motor yang berada pada jarak relatif dekat dengan tingkat kepercayaan yang tinggi. Selain itu, objek orang yang berada di latar belakang juga masih dapat dikenali oleh model. Hasil ini sejalan dengan nilai AR Large yang lebih tinggi dibandingkan AR *Small*, yang menunjukkan bahwa model cenderung lebih baik dalam mendeteksi objek dengan ukuran tampilan yang besar pada citra.

4. Deteksi Objek Bangku

Pada sampel *eval_side_by_side_5_0*, model mampu mendeteksi objek bangku yang berada di area terbuka dengan baik. Hasil ini menunjukkan bahwa model dapat mengenali objek statis yang umum dijumpai pada area pejalan kaki. Kemampuan tersebut mendukung fungsi sistem dalam membantu pengguna mengenali berbagai jenis rintangan maupun fasilitas yang berada di lingkungan sekitar.

5. Keterbatasan Deteksi dan *Bounding box* Bertumpuk

Berdasarkan beberapa sampel evaluasi, masih ditemukan kondisi di mana satu objek terdeteksi oleh lebih dari satu *bounding box*. Fenomena ini terlihat pada sampel *eval_side_by_side_3_0* dan *eval_side_by_side_6_0*, terutama pada objek orang yang berada dalam kelompok atau berdekatan dengan objek lain. Kondisi tersebut merupakan karakteristik yang umum ditemukan pada arsitektur Single Shot Multibox Detector (SSD).

Untuk mengurangi terjadinya *Bounding box* yang bertumpuk, pada implementasi aplikasi Android diterapkan metode *Non-Maximum Suppression* (NMS) dengan nilai ambang *Intersection over Union* (IoU) sebesar 0,3. Metode ini digunakan untuk memilih prediksi dengan tingkat kepercayaan tertinggi dan menghilangkan prediksi lain yang memiliki lokasi serupa, sehingga hasil deteksi menjadi lebih jelas dan mengurangi kemungkinan terjadinya duplikasi informasi saat sistem memberikan umpan balik suara kepada pengguna.

4.2.10 Hasil Pengujian Parameter Deteksi Objek

Pengujian parameter deteksi objek dilakukan untuk menentukan nilai *Confidence Threshold* yang paling sesuai pada model MobileNet SSD yang digunakan dalam aplikasi *Smart-Cane*. *Confidence Threshold* merupakan nilai batas minimum tingkat kepercayaan model dalam mengenali suatu objek. Deteksi dengan nilai *Confidence* di bawah *Threshold* akan diabaikan oleh sistem, sedangkan deteksi yang memenuhi nilai *Threshold* akan ditampilkan dan diinformasikan kepada pengguna melalui fitur *Text-to-Speech* (TTS).

Pada tahap pengujian, dilakukan beberapa percobaan menggunakan nilai *Threshold* yang berbeda untuk mengetahui pengaruhnya terhadap hasil deteksi objek. Nilai *Threshold* yang terlalu rendah menyebabkan sistem menghasilkan banyak deteksi yang tidak sesuai (*false positive*), sedangkan nilai *Threshold* yang terlalu tinggi menyebabkan beberapa objek gagal terdeteksi (*false negative*). Oleh karena itu, diperlukan nilai *Threshold* yang mampu memberikan keseimbangan antara jumlah objek yang terdeteksi dan tingkat akurasi deteksi.

Tabel 4. 13 Hasil Pengujian *Confidence Threshold*

No	<i>Confidence Threshold</i>	Hasil Pengamatan
1	0,30	Banyak menghasilkan deteksi yang tidak sesuai (<i>false positive</i>)
2	0,40	Deteksi mulai stabil namun masih terdapat beberapa kesalahan deteksi
3	0,50	Keseimbangan terbaik antara jumlah deteksi dan akurasi
4	0,60	Akurasi meningkat tetapi beberapa objek mulai tidak terdeteksi
5	0,70	Banyak objek yang tidak terdeteksi (<i>false negative</i>)

Berdasarkan hasil pengujian tersebut, sistem tidak menggunakan satu nilai *Threshold* yang sama untuk seluruh kelas objek. Setiap kelas objek diberikan nilai *Threshold* yang berbeda sesuai dengan performa model dan tingkat kepentingan objek terhadap keselamatan pengguna tunanetra.

4.2.11 Nilai *Confidence Threshold* yang Digunakan pada Sistem

```
// Threshold akurasi per kelas
private val classThreshold = mapOf(
    "orang" to 0.45f,
    "mobil" to 0.60f,
    "motor" to 0.60f,
    "sepeda" to 0.60f,
    "bangku" to 0.60f,
    "lubang" to 0.60f,
    "tembok" to 0.60f,
    "pohon" to 0.40f
)
```

Gambar 4. 12 Classthreshold Pada File Objectdetector.Kt

Pengaturan nilai *Confidence Threshold* pada aplikasi Android dilakukan pada file ObjectDetector.kt di Android Studio. Pada bagian ini, setiap kelas objek diberikan nilai *Threshold* yang berbeda sesuai hasil pengujian dan evaluasi model. Sistem akan membandingkan nilai *Confidence* score hasil prediksi dengan nilai *Threshold* yang telah ditentukan. Apabila nilai *Confidence* lebih rendah dari *Threshold*, maka objek tidak akan ditampilkan maupun diinformasikan melalui fitur TTS. Sebaliknya, apabila nilai *Confidence* memenuhi atau melebihi *Threshold* yang ditetapkan, maka objek akan dianggap valid dan diproses lebih lanjut oleh sistem.

Tabel 4. 14 Nilai *Confidence Threshold* yang Digunakan pada Sistem

No	Kelas Objek	AP (%)	AP@50 (%)	<i>Threshold</i>
1	Orang	4,0	12,5	0,45
2	Mobil	25,4	45,3	0,60
3	Motor	41,6	68,0	0,60
4	Sepeda	15,1	32,6	0,60
5	Bangku	47,8	68,1	0,60
6	Lubang	54,0	83,8	0,60
7	Tembok	26,3	45,2	0,60
8	Pohon	34,2	64,4	0,40

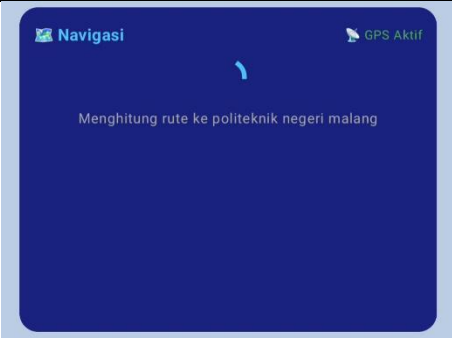
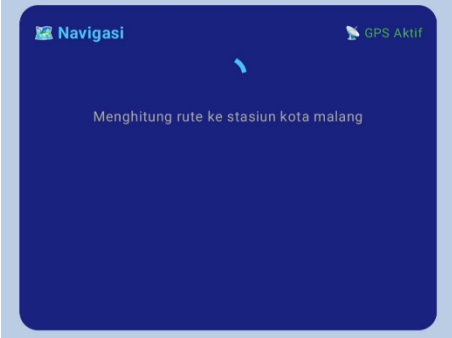
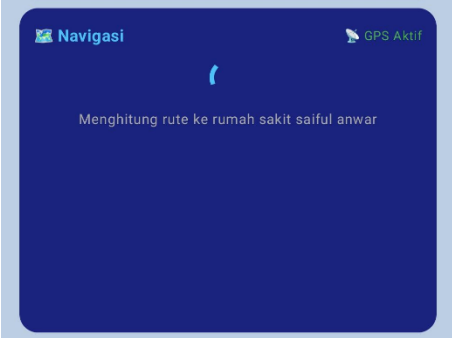
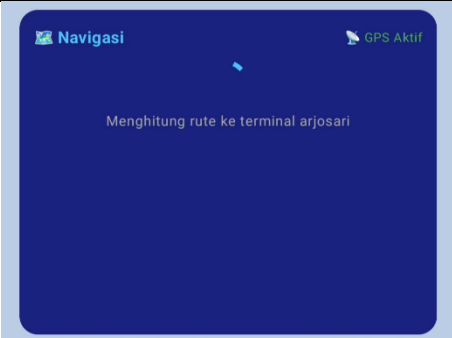
Ditunjukkan oleh nilai *Average Precision* (AP) serta tingkat risiko objek terhadap keselamatan pengguna. Objek orang diberikan *Threshold* sebesar 0,45 disebabkan oleh nilai AP yang rendah karena kegagalan mendeteksi manusia dapat membahayakan pengguna saat berjalan. Objek pohon menggunakan *Threshold* sebesar 0,40 karena pada beberapa kondisi pencahayaan model cenderung mengalami *false negative*. Sementara itu, objek mobil, motor, sepeda, bangku, lubang, dan tembok menggunakan *Threshold* sebesar 0,60 karena model telah mampu mengenali objek tersebut dengan tingkat kepercayaan yang cukup baik sehingga dapat mengurangi terjadinya *false positive*.

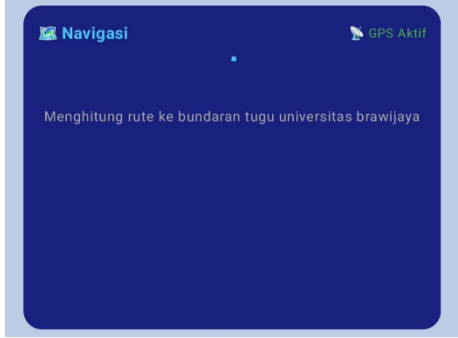
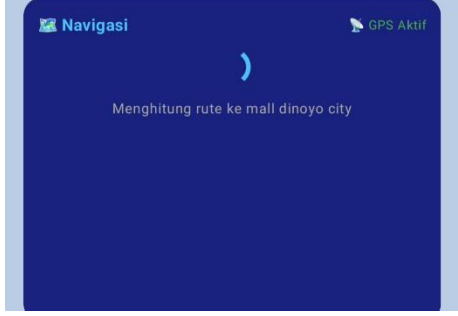
Penerapan *Threshold* yang berbeda untuk setiap kelas objek menghasilkan performa deteksi yang lebih stabil dibandingkan penggunaan satu *Threshold* yang sama untuk seluruh objek. Pengaturan ini memungkinkan sistem tetap sensitif terhadap objek yang berpotensi membahayakan pengguna sekaligus mengurangi jumlah deteksi yang tidak sesuai, sehingga informasi yang disampaikan melalui fitur TTS menjadi lebih akurat dan tidak mengganggu proses navigasi pengguna tunanetra.

4.2.12 Hasil Pengujian *Voice Command*

Pengujian *Voice Command* dilakukan untuk mengetahui kemampuan sistem dalam mengenali perintah suara yang diucapkan pengguna dan mengubahnya menjadi tujuan navigasi. Pengujian dilakukan menggunakan fitur *Speech Recognizer* pada *smartphone* Android dengan beberapa lokasi tujuan yang berbeda. Setelah tombol volume bawah ditekan satu kali, sistem akan mengaktifkan mode perekaman suara dan menunggu pengguna mengucapkan lokasi tujuan. Hasil pengenalan suara kemudian diproses untuk mencari koordinat tujuan dan menghasilkan rute navigasi menggunakan layanan OSRM.

Tabel 4. 15 Hasil Pengujian *Voice Command*

No	Perintah Suara	Hasil Pengenalan	Status
1	Politeknik Negeri Malang		Berhasil
2	Stasiun Malang		Berhasil
3	Rumah Sakit Saiful Anwar		Berhasil
4	Terminal Arjosari		Berhasil

5	Bundaran Tugu Universitas Brawijaya		Berhasil
6	RS Islam UNISMA		Berhasil
7	Mall Dinoyo City		Berhasil
8	Graha Polinema		Berhasil

Berdasarkan hasil pengujian, sistem mampu mengenali beberapa tujuan yang diucapkan pengguna dengan baik, seperti "Politeknik Negeri Malang", "Stasiun Malang", dan "Terminal Arjosari". Setelah tujuan dikenali, aplikasi berhasil menghitung rute dan memberikan petunjuk arah melalui *Text-to-Speech* (TTS). Namun, pada beberapa kondisi terdapat kesalahan pengenalan ketika suara pengguna kurang jelas atau lingkungan sekitar memiliki tingkat kebisingan yang tinggi.

4.2.13 Hasil Pengujian *Text-to-Speech* (TTS)

Pengujian *Text-to-Speech* (TTS) dilakukan untuk memastikan aplikasi mampu menyampaikan informasi kepada pengguna tunanetra dalam bentuk suara sesuai dengan kondisi yang terjadi pada sistem. Pengujian dilakukan pada tiga fungsi utama, yaitu penyampaian informasi hasil deteksi objek, informasi jarak dari sensor LiDAR VL53L1X, dan instruksi navigasi menuju lokasi tujuan.

Pada pengujian deteksi objek dan jarak, sistem berhasil mengubah hasil deteksi objek dari model MobileNet SSD menjadi informasi suara yang dapat dipahami pengguna. Sebagai contoh, ketika objek orang terdeteksi pada jarak 150 cm, aplikasi mengeluarkan suara "Di depan ada orang, dari lidar terdeteksi objek 150 centimeter". Pengujian serupa dilakukan pada objek motor, pohon, bangku, dan lubang, serta seluruh objek dapat diinformasikan dengan baik oleh sistem. Selain itu, ketika sensor LiDAR mendeteksi adanya hambatan tetapi objek tidak berhasil dikenali oleh model deteksi, sistem tetap memberikan peringatan suara berupa "Ada halangan, dari lidar terdeteksi objek xxx centimeter". Hasil ini menunjukkan bahwa integrasi antara deteksi objek dan sensor LiDAR dapat memberikan informasi yang lebih lengkap kepada pengguna.

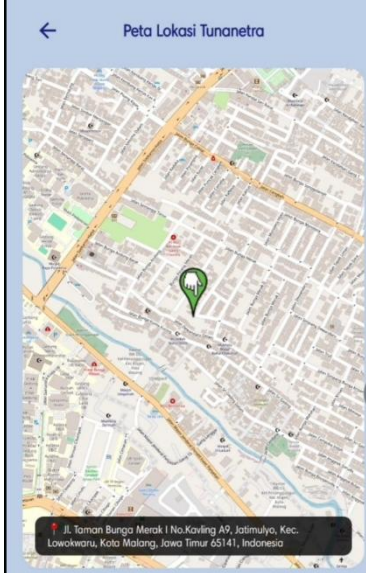
Pengujian juga dilakukan pada fitur navigasi dengan tujuan menuju Politeknik Negeri Malang. Setelah pengguna menentukan tujuan, aplikasi berhasil memperoleh rute perjalanan dan memberikan instruksi suara secara bertahap sesuai kondisi perjalanan. Instruksi yang diberikan meliputi perintah berjalan lurus, belok kiri, belok kanan, hingga pemberitahuan saat tujuan telah tercapai. Sebagai contoh, "mulai berjalan lurus sejauh 72 meter" saat berbelok "belok kiri, lalu jalan lurus 81 meter", kemudian dilanjutkan dengan instruksi berikutnya sesuai rute yang diperoleh. Saat pengguna telah mencapai lokasi tujuan, aplikasi memberikan notifikasi suara "Anda telah sampai di tujuan".

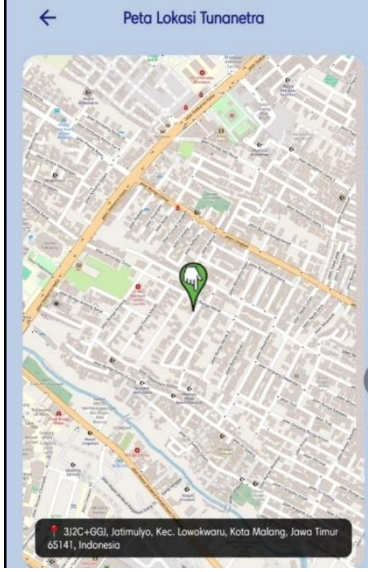
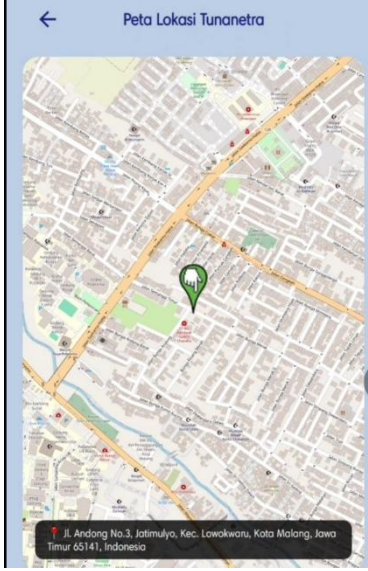
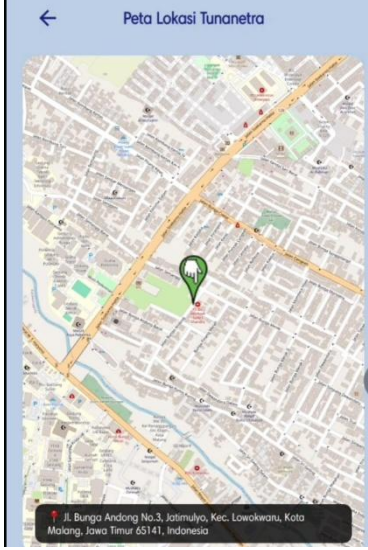
Berdasarkan hasil pengujian yang telah dilakukan, fitur *Text-to-Speech* (TTS) mampu berfungsi dengan baik dalam menyampaikan informasi hasil deteksi objek, jarak hambatan, dan petunjuk navigasi.

4.2.14 Hasil Pengujian Navigasi OSRM

Pengujian navigasi dilakukan untuk mengetahui kemampuan sistem dalam menentukan rute perjalanan dan memberikan petunjuk arah kepada pengguna tunanetra secara *real-time*. Pengujian dilakukan dengan skenario perjalanan dari Jalan Taman Bunga Merak menuju Rumah Sakit Ibu dan Anak Galeri Candra menggunakan fitur *Voice Command* dan layanan navigasi OSRM. Pengguna terlebih dahulu mengaktifkan fitur navigasi menggunakan tombol volume bawah, kemudian mengucapkan tujuan perjalanan yaitu "Rumah Sakit Ibu dan Anak Galeri Candra". Sistem selanjutnya memproses perintah suara menggunakan *Speech Recognizer* dan mengirimkan lokasi tujuan ke layanan OSRM untuk memperoleh rute perjalanan.

Tabel 4. 16 Hasil Pengujian Navigasi OSRM

Aplikasi Tunanetra	Aplikasi Monitoring	Keterangan
		Sistem berhasil menerima dan mengenali tujuan navigasi melalui <i>Voice Command</i> .
		Sistem berhasil mengirim tujuan ke layanan OSRM dan memperoleh rute navigasi.
		Sistem berhasil memberikan instruksi navigasi awal sesuai rute yang dihasilkan.

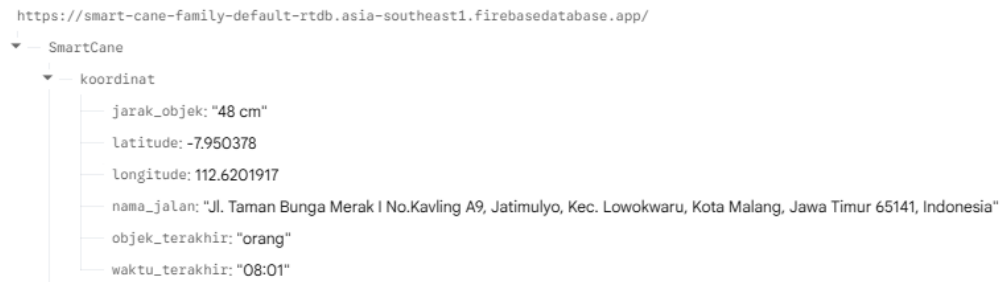
		Sistem berhasil memperbarui petunjuk arah berdasarkan posisi pengguna saat berjalan
		
		Sistem berhasil mengarahkan pengguna hingga mencapai lokasi tujuan.

Berdasarkan hasil pengujian navigasi dari Jl. Taman Bunga Merak menuju RS Ibu dan Anak Galeri Candra, sistem berhasil menerima tujuan melalui *Voice Command*, memperoleh rute dari layanan OSRM, serta memberikan petunjuk arah melalui fitur *Text-to-Speech* (TTS). Selama proses navigasi berlangsung, aplikasi monitoring juga berhasil menampilkan dan memperbarui posisi pengguna pada peta secara berkala. Hasil pengujian menunjukkan bahwa integrasi antara *Voice Command*, GPS, API OSRM, dan aplikasi monitoring dapat berjalan dengan baik hingga pengguna mencapai lokasi tujuan.

Untuk meningkatkan ketepatan navigasi, sistem memberikan peringatan dalam dua tahap sebelum pengguna mencapai titik belok. Peringatan pertama diberikan pada jarak sekitar 10 meter sebelum tikungan sebagai pemberitahuan persiapan belok. Selanjutnya, perintah belok diberikan pada jarak sekitar 5 meter sebelum tikungan. Pengaturan ini dilakukan karena proses pembacaan instruksi oleh TTS memerlukan waktu sekitar 3 detik. Dengan asumsi kecepatan berjalan pengguna sekitar 1 m/s, pengguna dapat berpindah sejauh kurang lebih 3 meter selama instruksi dibacakan. Oleh karena itu, setelah instruksi selesai diucapkan, pengguna masih berada sekitar 2 meter sebelum titik tikungan sehingga memiliki waktu yang cukup untuk melakukan manuver belok sesuai arahan yang diberikan sistem. Hasil pengujian menunjukkan bahwa metode ini mampu membantu pengguna mengikuti rute navigasi dengan lebih tepat dan mengurangi risiko terlambat berbelok.

4.2.15 Hasil Pengujian Database Firebase

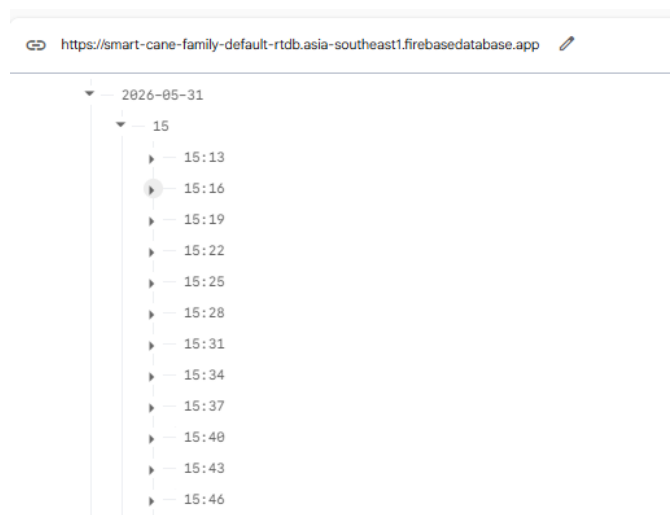
Pengujian database Firebase dilakukan untuk memastikan proses penyimpanan, pembaruan, dan pengambilan data dapat berjalan dengan baik pada aplikasi *Smart-Cane* dan aplikasi monitoring keluarga. Data yang diuji meliputi koordinat lokasi pengguna, objek terakhir yang terdeteksi, jarak objek, nama jalan, serta waktu pencatatan data.



Gambar 4. 13 Pengujian Pembaruan Data Real-Time Database Firebase

Pada Gambar 4.13 terlihat bahwa Firebase berhasil menerima dan memperbarui data lokasi pengguna secara berkala yang terdiri dari informasi latitude, longitude, nama jalan, objek terakhir yang terdeteksi, jarak objek, dan waktu pembaruan. Data tersebut digunakan sebagai sumber informasi pada *Dashboard* dan peta lokasi di aplikasi monitoring keluarga.

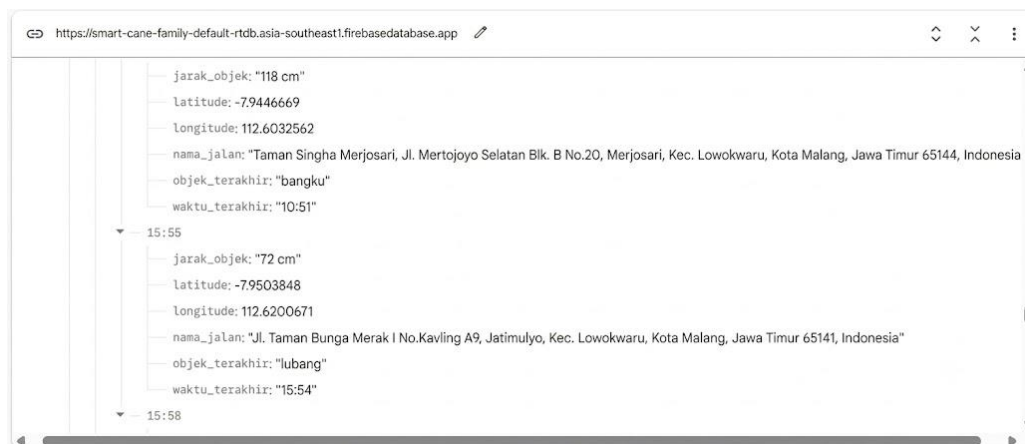
Meskipun proses sinkronisasi data berjalan dengan baik, hasil pengujian menunjukkan adanya waktu tunda (*delay*) pembaruan sekitar 10–15 detik. Karena aplikasi mengirimkan data setiap 10 detik, maka data yang tampil pada aplikasi monitoring umumnya mengalami pembaruan sekitar 20 detik setelah kondisi sebenarnya terjadi. Akibatnya, posisi penanda (*marker*) pada peta monitoring tidak berubah secara instan dan mengalami sedikit keterlambatan dibandingkan posisi pengguna sebenarnya.



Gambar 4. 14 Penyimpanan Log Data Pada Firebase

Berdasarkan hasil pengujian pada Gambar 4.14, sistem berhasil menyimpan data log secara otomatis setiap 3 menit. Data tersebut disusun dalam struktur hierarki berdasarkan tanggal, jam, dan waktu pencatatan sehingga memudahkan proses pencarian data pada aplikasi monitoring.

Selain itu, sistem juga berhasil melakukan pergantian data log secara otomatis setiap hari. Data yang ditampilkan pada aplikasi monitoring akan menyesuaikan dengan tanggal aktif sehingga riwayat lama tidak ditampilkan pada halaman utama dan digantikan oleh data terbaru.



Gambar 4. 15 Isi pada Log Data

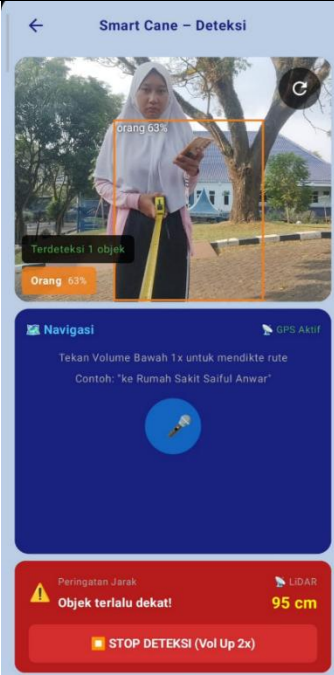
Hasil pengujian pada Gambar 4.15 menunjukkan bahwa aplikasi monitoring keluarga berhasil mengambil data log yang tersimpan pada Firebase dan menampilkannya sesuai kondisi yang terjadi pada pengguna. Informasi yang berhasil ditampilkan meliputi nama objek yang terdeteksi, jarak objek, lokasi pengguna, dan waktu kejadian.

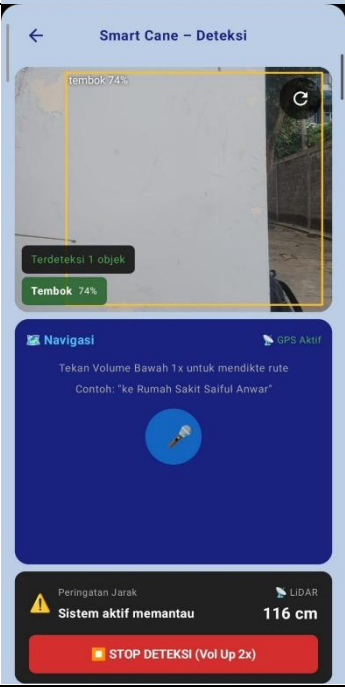
Data yang tersimpan pada Firebase dapat diakses kembali oleh aplikasi monitoring sehingga keluarga dapat melihat riwayat aktivitas pengguna tunanetra berdasarkan waktu dan lokasi kejadian. Hal ini menunjukkan bahwa proses integrasi antara Firebase dan aplikasi monitoring telah berjalan dengan baik.

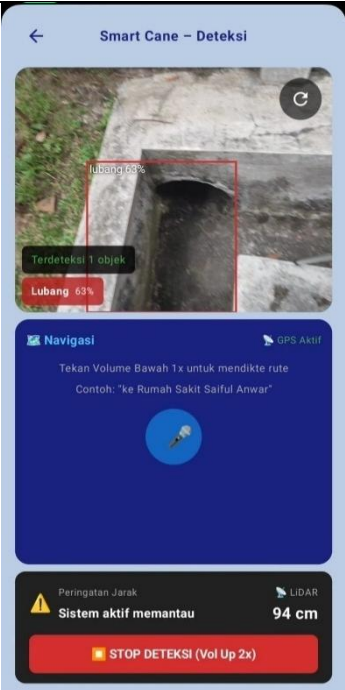
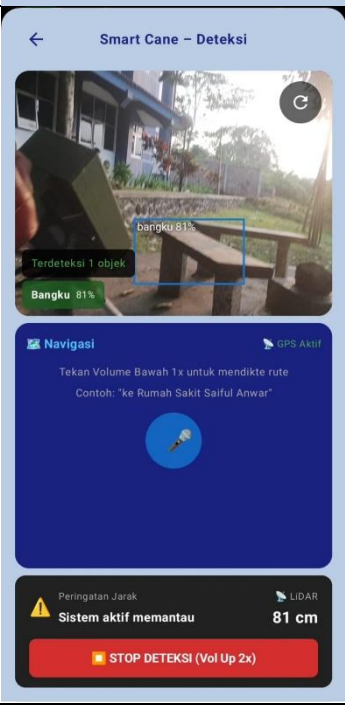
4.2.16 Pengujian Deteksi Objek dan LiDAR Secara Bersamaan

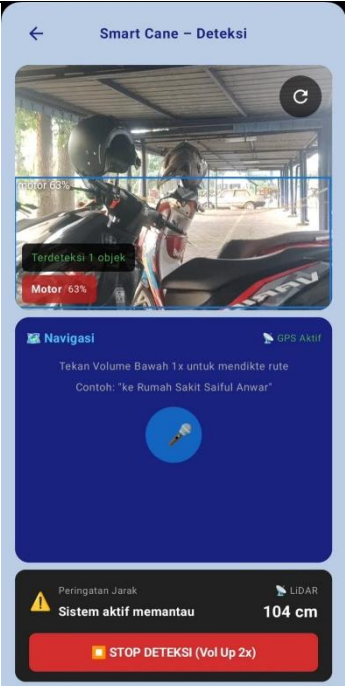
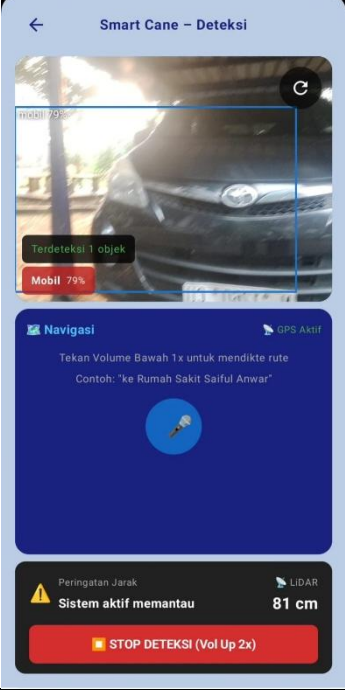
Pengujian ini dilakukan untuk mengevaluasi kemampuan sistem *Smart Cane* dalam mengintegrasikan kamera *smartphone Android* dengan model SSD MobileNet V2 untuk mendeteksi objek secara *real-time*, sekaligus mengukur jarak objek menggunakan sensor LiDAR VL53L1X. Pengujian dilakukan terhadap delapan kelas objek yang digunakan pada proses pelatihan model, yaitu orang, mobil, motor, sepeda, bangku, pohon, tembok, dan lubang. Setiap objek ditempatkan pada jarak sebenarnya sebesar 100 cm dari sensor LiDAR, kemudian kamera *smartphone* menangkap citra objek untuk diproses oleh model SSD MobileNet V2 berbasis TensorFlow Lite, sementara sensor LiDAR secara bersamaan melakukan pengukuran jarak. Hasil deteksi objek yang diperoleh dari kamera dan informasi jarak yang dikirimkan oleh ESP32 melalui Bluetooth Low Energy (BLE) ditampilkan secara bersamaan pada aplikasi *Android* sebagai bentuk integrasi antara sistem pengenalan objek dan pengukuran jarak.

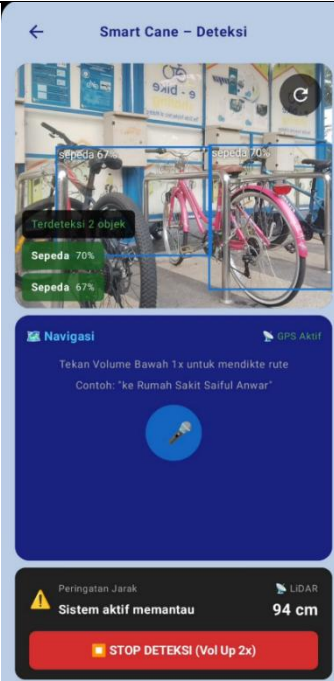
Tabel 4. 17 Pengujian Deteksi Objek dan LiDAR Secara Bersamaan

Objek	Gambar	Jarak Asli	Hasil Deteksi
Orang		100 cm	

Pohon		100 cm	
Tembok		100 cm	

Lubang		100 cm	
Bangku		100 cm	

Motor		100 cm	 Smart Cane - Deteksi Motor 63% Terdeteksi 1 objek Motor 63% Navigasi Tekan Volume Bawah 1x untuk mendikte rute Contoh: "ke Rumah Sakit Saiful Anwar" Peringatan Jarak Sistem aktif memantau 104 cm STOP DETEKSI (Vol Up 2x)
Mobil		100 cm	 Smart Cane - Deteksi Mobil 79% Terdeteksi 1 objek Mobil 79% Navigasi Tekan Volume Bawah 1x untuk mendikte rute Contoh: "ke Rumah Sakit Saiful Anwar" Peringatan Jarak Sistem aktif memantau 81 cm STOP DETEKSI (Vol Up 2x)

Sepeda		100 cm	
--------	---	--------	--

Berdasarkan hasil pengujian pada Tabel 4.17, sistem berhasil mendeteksi seluruh objek sesuai dengan kelas yang telah dilatih menggunakan model SSD MobileNet V2. Untuk mengevaluasi lebih lanjut hasil deteksi tersebut, dilakukan analisis terhadap kemampuan kamera smartphone yang telah diintegrasikan dengan model SSD MobileNet V2 berdasarkan kesesuaian kelas objek dan nilai *confidence* yang dihasilkan pada setiap proses deteksi. Hasil pengujian tersebut disajikan pada Tabel 4.18.

Tabel 4. 18 Hasil Pengujian Deteksi Kamera dengan Objek

Objek Uji	Objek Terdeteksi	Confidence (%)	Kecocokan	Status
Orang	Orang	63	✓	Berhasil
Pohon	Pohon	84	✓	Berhasil
Bangku	Bangku	81	✓	Berhasil
Motor	Motor	63	✓	Berhasil
Mobil	Mobil	79	✓	Berhasil
Sepeda	Sepeda	70	✓	Berhasil
Tembok	Tembok	74	✓	Berhasil
Lubang	Lubang	63	✓	Berhasil

Berdasarkan hasil pengujian pada Tabel 4.18, seluruh objek uji berhasil dideteksi sesuai dengan kelas yang telah dilatih menggunakan model SSD MobileNet V2, sehingga tingkat kecocokan deteksi terhadap kelas objek mencapai 100%. Hal ini menunjukkan bahwa integrasi kamera *smartphone* dengan model SSD MobileNet V2 berbasis TensorFlow Lite telah berjalan dengan baik dan mampu melakukan deteksi objek secara *real-time*. Nilai *confidence* yang dihasilkan berada pada rentang 63% hingga 84%. Kelas pohon memperoleh nilai *confidence* tertinggi sebesar 84%, sedangkan kelas orang, motor, dan lubang memperoleh nilai *confidence* sebesar 63%. Meskipun terdapat perbedaan nilai *confidence* pada setiap kelas, seluruh objek tetap berhasil dikenali sesuai dengan kelasnya. Perbedaan nilai *confidence* dipengaruhi oleh karakteristik visual setiap objek, kondisi pencahayaan, sudut pengambilan gambar, ukuran objek di dalam citra, serta kompleksitas latar belakang saat proses deteksi. Hasil pengujian ini menunjukkan bahwa model SSD MobileNet V2 mampu diimplementasikan pada *smartphone Android* dan bekerja dengan baik sebagai sistem deteksi objek pada *Smart Cane* untuk membantu pengguna tunanetra mengenali objek-objek di lingkungan sekitar secara *real-time*.

Setelah objek berhasil dikenali oleh kamera *smartphone*, sistem selanjutnya melakukan pengukuran jarak terhadap objek menggunakan sensor LiDAR VL53L1X secara *real-time*. Informasi jenis objek hasil deteksi dan data jarak kemudian diintegrasikan sehingga pengguna memperoleh informasi objek beserta jaraknya secara bersamaan. Hasil pengujian menunjukkan bahwa sensor LiDAR mampu mengukur jarak objek dengan baik, meskipun masih terdapat perbedaan antara jarak hasil pengukuran dan jarak sebenarnya. Perbedaan tersebut dipengaruhi oleh karakteristik sensor LiDAR VL53L1X yang menggunakan metode *Time of Flight* (ToF), sehingga performanya dipengaruhi oleh intensitas cahaya lingkungan, sudut pantulan sinar laser terhadap permukaan objek, serta karakteristik material objek yang dideteksi. Untuk mengetahui tingkat ketelitian sensor LiDAR, dilakukan perhitungan nilai error dan akurasi menggunakan persamaan berikut :

$$Error (\%) = \frac{[jarak \text{ terukur} - jarak \text{ pengukuran}]}{jarak \text{ pengukuran}} \times 100\%$$

$$Akurasi (\%) = 100\% - Error\%$$

Tabel 4. 19 Tabel Perhitungan Hasil Pengujian Jarak Lidar

Objek	Jarak Sebenarnya	Hasil LiDAR	Error (%)	Akurasi (%)
Orang	100 cm	95 cm	5	95
Pohon	100 cm	96 cm	4	96
Bangku	100 cm	81 cm	19	81
Motor	100 cm	104 cm	4	96
Mobil	100 cm	81 cm	19	81
Sepeda	100 cm	94 cm	6	94
Tembok	100 cm	116 cm	16	84
Lubang	100 cm	94 cm	6	94
Total rata-rata			9, 88	90, 13

Berdasarkan hasil pengujian terhadap delapan objek, sensor LiDAR VL53L1X memperoleh rata-rata error sebesar 9,88% dan akurasi sebesar 90,13%. Perbedaan hasil pengukuran pada beberapa objek dipengaruhi oleh intensitas cahaya matahari, sudut pantulan sinar laser, serta karakteristik permukaan objek. Meskipun demikian, integrasi model SSD MobileNet V2 dengan sensor LiDAR VL53L1X mampu bekerja secara bersamaan untuk mendeteksi jenis objek dan mengukur jaraknya secara *real-time* .

4.2.17 Hasil Pengujian Sistem pada Lingkungan Indoor dan Outdoor

Pengujian dilakukan untuk mengetahui kinerja sistem pada lingkungan dalam ruangan (indoor) dan luar ruangan (outdoor). Pengujian dilakukan pada kondisi pencahayaan normal sesuai dengan batasan penelitian. Aspek yang diamati meliputi kemampuan model SSD MobileNet V2 dalam mendeteksi objek, kinerja sensor LiDAR VL53L1X dalam mengukur jarak objek, serta fungsi *Text-to-Speech* (TTS), Bluetooth, dan navigasi

Tabel 4. 20 Hasil Pengujian Sistem pada Lingkungan Indoor dan Outdoor

Pengujian	Indoor	Outdoor
Deteksi objek	Beberapa objek mengalami salah deteksi	Deteksi lebih stabil dan sesuai kelas
Pengukuran jarak	Respon cepat dan akurasi lebih tinggi	Respon sedikit lebih lambat, akurasi menurun
Bluetooth	Berfungsi	Berfungsi
TTS	Berfungsi	Berfungsi
Navigasi	Berfungsi	Berfungsi

Berdasarkan hasil pengujian, seluruh fitur dapat berfungsi dengan baik pada lingkungan indoor maupun outdoor. Model SSD MobileNet V2 menunjukkan performa yang lebih baik pada lingkungan outdoor karena dataset pelatihan yang digunakan merupakan gabungan dataset COCO dan Roboflow, di mana sebagian besar citra pada dataset berasal dari lingkungan luar ruangan. Sebaliknya, pada lingkungan indoor masih ditemukan beberapa kesalahan klasifikasi pada objek yang memiliki karakteristik visual yang mirip.

Berbeda dengan model deteksi objek, sensor LiDAR VL53L1X menunjukkan performa yang lebih baik pada lingkungan indoor. Pada lingkungan outdoor, sensor mengalami peningkatan waktu respon (*delay*) sekitar 3 detik serta penurunan akurasi pengukuran jarak akibat pengaruh intensitas cahaya matahari. Sensor LiDAR VL53L1X menggunakan teknologi *Time of Flight* (ToF) dengan laser inframerah berpanjang gelombang 940 nm. Cahaya matahari yang juga mengandung spektrum inframerah dapat mengganggu proses pemancaran dan penerimaan sinyal laser, sehingga hasil

pengukuran menjadi kurang stabil dibandingkan saat digunakan di dalam ruangan.

Kondisi tersebut terlihat dari hasil pengujian sensor LiDAR pada Subbab 4.2.3, di mana rata-rata akurasi pengukuran mencapai 98,27% pada kondisi pengujian normal. Namun, pada pengujian di lingkungan outdoor yang disajikan pada Subbab 4.2.16, rata-rata akurasi menurun menjadi 90,13%. Penurunan akurasi ini menunjukkan bahwa kondisi lingkungan, khususnya paparan sinar matahari, memengaruhi performa sensor LiDAR dalam mengukur jarak objek.

4.2.18 Dokumentasi pengujian sistem oleh penyandang tunanetra.

Pengujian sistem dilakukan untuk mengetahui kinerja tongkat pintar ketika digunakan secara langsung oleh penyandang tunanetra. Pengujian dilaksanakan pada Hari Minggu, 5 Juli 2026 di kawasan *Car Free Day* (CFD) Jalan Ijen, Kota Malang. Pengujian melibatkan satu responden, yaitu Ibu Sunarti, berusia 55 tahun, yang bekerja sebagai pengamen jalanan di kawasan CFD Jalan Ijen.

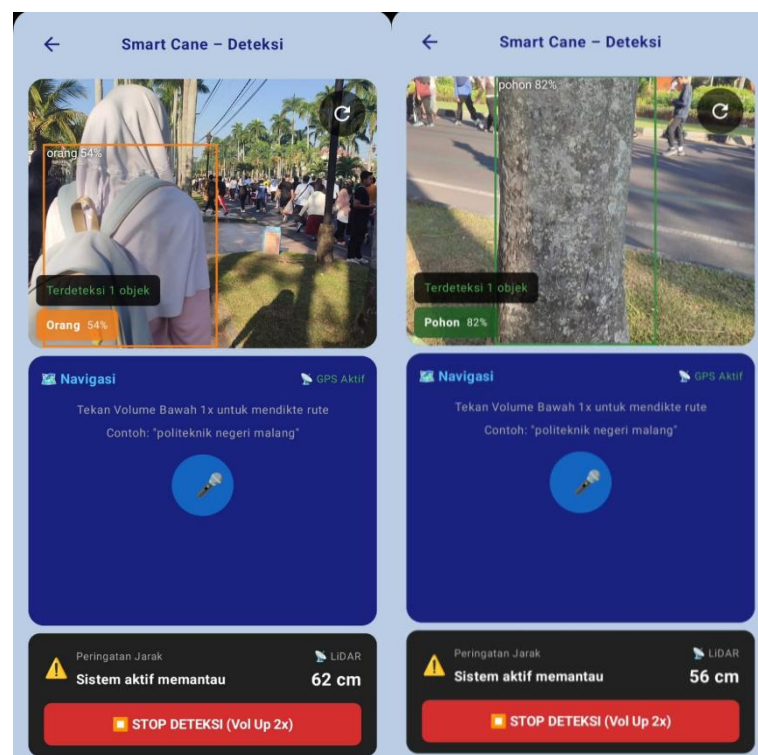


Gambar 4. 16 Dokumentasi Pengujian Tongkat Pintar Oleh Penyandang Tunanetra



Gambar 4. 17 Dokumentasi Pengujian Tongkat Pintar Oleh Penyandang Tunanetra

Selama pengujian, responden menggunakan tongkat pintar pada area trotoar di kawasan *Car Free Day* (CFD) Jalan Ijen. Berdasarkan hasil pengujian, aplikasi dapat mendeteksi beberapa objek yang berada di sekitar pengguna, seperti orang dan pohon. Informasi hasil deteksi ditampilkan pada aplikasi *Android* disertai dengan informasi jarak dari sensor LiDAR.



Gambar 4. 18 Hasil Deteksi Objek Pada Aplikasi Android.

Berdasarkan Gambar 4.18, aplikasi mampu mendeteksi objek yang berada di sekitar pengguna selama pengujian berlangsung. Pada pengujian pertama, sistem mendeteksi objek orang dengan nilai *Confidence* sebesar 54% dan memberikan informasi jarak 62 cm. Pada pengujian kedua, sistem mendeteksi objek pohon dengan nilai *Confidence* sebesar 82% dan memberikan informasi jarak 56 cm. Hasil tersebut menunjukkan bahwa fitur deteksi objek dan pengukuran jarak dapat bekerja secara bersamaan selama proses pengujian di lapangan.

Berdasarkan hasil wawancara setelah pengujian, responden menyatakan bahwa tongkat pintar dapat membantu memberikan informasi mengenai keberadaan objek di depan pengguna melalui notifikasi suara dan informasi jarak. Menurut responden, informasi tersebut cukup membantu selama proses berjalan. Namun demikian, responden juga menyampaikan bahwa sistem belum dapat memberikan informasi posisi objek berdasarkan arah, seperti objek yang berada di sisi kiri atau sisi kanan pengguna. Fitur tersebut diharapkan dapat dikembangkan pada penelitian selanjutnya agar informasi yang diterima pengguna menjadi lebih lengkap

BAB V

PENUTUP

5.1 Kesimpulan

- 1 Berhasil dirancang sistem tongkat pintar yang mengintegrasikan sensor LiDAR VL53L1X, ESP32, motor getar, dan aplikasi Android. Sensor LiDAR mampu mendeteksi hambatan secara real-time dengan rata-rata akurasi 98,27% pada pengujian normal dan 90,13% pada pengujian outdoor. Informasi jarak disampaikan kepada pengguna melalui Text-to-Speech (TTS) dan pola getaran sesuai jarak objek.
- 2 Berhasil diintegrasikan kamera smartphone dengan model SSD MobileNet V2 untuk mendeteksi delapan kelas objek, yaitu orang, mobil, motor, sepeda, bangku, lubang, tembok, dan pohon. Hasil evaluasi menunjukkan model memperoleh mAP sebesar 31,0% dan mAP@50 sebesar 52,5% dan nilai *confidence* yang dihasilkan berada pada rentang 63% hingga 84% .
- 3 Berhasil disinkronkan data sensor dengan sistem navigasi berbasis aplikasi Android menggunakan API OSRM. Sistem dapat memberikan petunjuk arah berdasarkan lokasi pengguna melalui GPS serta menyampaikan instruksi navigasi menggunakan Text-to-Speech (TTS).
- 4 Berhasil diimplementasikan fitur Voice Command menggunakan microphone pada smartphone untuk menerima perintah tujuan navigasi. Hasil pengujian menunjukkan seluruh perintah suara yang diuji dapat dikenali dengan baik sehingga pengguna dapat mengoperasikan sistem tanpa melakukan interaksi langsung dengan layar aplikasi.

5.2 Saran

1. Fitur deteksi objek dan pengukuran jarak diharapkan dapat diintegrasikan menjadi satu proses deteksi, sehingga setiap objek yang berhasil dikenali dapat langsung disertai informasi jaraknya tanpa bergantung pada proses terpisah antara model deteksi objek dan sensor LiDAR. Dengan demikian, sistem dapat memberikan informasi yang lebih cepat dan mudah dipahami oleh pengguna.

2. Performa model deteksi objek dapat ditingkatkan dengan menambah jumlah dan variasi dataset pelatihan serta memperluas jumlah kelas objek yang dideteksi. Dengan demikian, sistem diharapkan mampu meningkatkan akurasi deteksi dan dapat digunakan secara lebih optimal pada lingkungan indoor maupun outdoor.
3. Penelitian selanjutnya juga disarankan untuk mengembangkan sistem agar mampu mendeteksi posisi objek berdasarkan arah keberadaannya, seperti di sisi kanan, kiri, maupun depan pengguna. Informasi arah tersebut dapat disampaikan melalui suara atau pola getaran yang berbeda sehingga penyandang tunanetra lebih mudah mengetahui posisi hambatan dan mengambil keputusan untuk menghindari objek dengan lebih aman.

DAFTAR PUSTAKA

- [1] D. Aryani, A. Irfan Gunawan, K. Nisa, A. Zubair, J. Teknik Elektro, and P. Negeri Ujung Pandang, “Alat Bantu Jalan Tunanetra Menggunakan Sensor Light Detection And Ranging (LiDAR) berbasis Arduino,” *J. Teknol. Elekterika*, vol. 20, no. 2, 2023.
- [2] W. H. Organization, “World Report on Vision. Geneva.” Accessed: Jun. 18, 2025. [Online]. Available: <https://www.who.int/publications-detail/world-report-on-vision>
- [3] A. N. Al Ansori, “RI Duduki Peringkat Ketiga Dunia dalam Kasus Kebutaan,” *Liputan6.com*. Accessed: Oct. 06, 2025. [Online]. Available: <https://www.liputan6.com/disabilitas/read/5194116/ri-duduki-peringkat-ketiga-dunia-dalam-kasus-kebutaan?page=4>.
- [4] A. M. R. and S. Gatra, “Fasilitas Akses Penyandang Tuna Netra atas Objek Hak Cipta Berdasar Marrakesh Treaty,” *Kompas.com*. Accessed: Oct. 06, 2025. [Online]. Available: <https://nasional.kompas.com/read/2023/01/15/16070311/fasilitas-akses-penyandang-tuna-netra-atas-objek-hak-cipta-berdasar?page=all>.
- [5] M. Imran, “Peningkatan Pemberdayaan Penyandang Tunanetra melalui Perancangan Social Media Newsletter di Yayasan Sosial Tunanetra,” *J. Komunitas J. Pengabd. Kpd. Masy.*, vol. 6, no. 2, pp. 229–239, 2024, [Online]. Available: <http://ojs.stiami.ac.id>
- [6] M. T. D. Putra, A. Adiwilaga, A. Clarissa, A. A. P. Gustiansyah, A. D. Kurniadi, and Z. Mumtaz, “Alat Bantu Tuna Netra Berbasis Arduino Uno dan Artificial Intelligence dengan metode YOLO v7,” *J. Ilm. FIFO*, vol. 15, no. 2, p. 159, May 2024, doi: 10.22441/fifo.2023.v15i2.007.
- [7] A. P. Utomo, A. Sucipto, S. Ayu Wulandari, A. F. Rosyady, M. E. Lazuardi, and D. Dyiono, “Implementasi desain Smart Stick untuk anak tunanetra berbasis GPS terintegrasi dengan *smartphone*,” *J. ELTEK*, vol. 21, no. 1, pp.

- 10–19, Apr. 2023, doi: 10.33795/eltek.v21i1.369.
- [8] F. R. Hidayat, I. Komala Dewi Patwari, and T. Akbar, “Rancang Bangun Alat Bantu Jalan Tunanetra Berbasis Mikrokontroler dengan Output Suara,” *J. Print. J. Pengemb. Rekayasa Inform. dan Komput.*, vol. 2, no. 1, pp. 83–95, Jun. 2024, doi: 10.29408/jprinter.v2i1.23849.
 - [9] L. A. S. Fikri Gunawan Lubis, Indra Roza, “Tingkat Tunanetra Menggunakan Sensor Ultrasonik dan GPS Berbasis Mikrokontroler,” *J. Ilm. Teknol. Harapan*, vol. 13, no. 1, pp. 46–53, 2025.
 - [10] Espressif Systems, “Datasheet Esp32 DevKit v1,” 2023.
 - [11] G. Khekare and K. Solanki, “Real Time Object Detection with Speech Recognition using Tensorflow Lite,” *GIS Sci. J.*, 2022, [Online]. Available: <https://www.researchgate.net/publication/359393141>
 - [12] E. Cardillo, C. Li, and A. Caddemi, “Empowering Blind People Mobility: A Millimeter-Wave Radar Cane,” in *2020 IEEE International Workshop on Metrology for Industry 4.0 and IoT, MetroInd 4.0 and IoT 2020 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., Jun. 2020, pp. 213–217. doi: 10.1109/MetroInd4.0IoT48571.2020.9138239.
 - [13] S. Solikh, “Rancang Bangun Tingkat Bantu Tunanetra Berbasis Logika Fuzzy Terintegrasi IoT Menggunakan Sensor Lidar,” Politeknik Negeri Malang, 2023.
 - [14] STMicroelectronics, “VL53L1X Time-of-Flight Ranging Sensor Datasheet,” no. August, pp. 1–36, 2024.
 - [15] “*Vibration Motor* Module Mini Flat Vibrating DC Motor.”
 - [16] A.-D. V. GmbH, “Battery Expansion Shield 18650 V3 Datasheet. Nürnberg.” Accessed: Jul. 04, 2026. [Online]. Available: https://www.az-delivery.de/en/products/battery-expansion-shield-18650-v3-inkl-usb-kabel?utm_source=chatgpt.com
 - [17] A. Salunkhe, M. Raut, S. Santra, and S. Bhagwat, “Android-based object

- recognition application for visually impaired,” *ITM Web Conf.*, vol. 40, p. 03001, 2021, doi: 10.1051/itmconf/20214003001.
- [18] Google, “Android Developers Documentation.” Accessed: Jun. 18, 2026. [Online]. Available: <https://developer.android.com/>
 - [19] P. OSRM, “*Open Source Routing Machine* Documentation.” Accessed: Jun. 20, 2026. [Online]. Available: <https://project-osrm.org/>
 - [20] M. Sandler, M. Zhu, A. Zhmoginov, and C. V Mar, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” pp. 1–14, 2019.
 - [21] R. Padilla, S. L. Netto, and E. A. B. Silva, “A Survey on Performance Metrics for Object-Detection Algorithms,” 2020.
 - [22] T.-Y. Lin, G. P. MSR, and Y. Cui, “Microsoft COCO: Common Objects in Context.” [Online]. Available: <https://cocodataset.org>.
 - [23] Google, “Firebase Documentation.” Accessed: Jun. 18, 2026. [Online]. Available: <https://firebase.google.com/docs?hl=id>
 - [24] Roboflow, “Roboflow Documentation.” Accessed: Jun. 18, 2026. [Online]. Available: <https://docs.roboflow.com/>
 - [25] Microsoft, “Visual Studio Code Documentation.” Accessed: Jun. 18, 2026. [Online]. Available: <https://code.visualstudio.com/docs>

LAMPIRAN

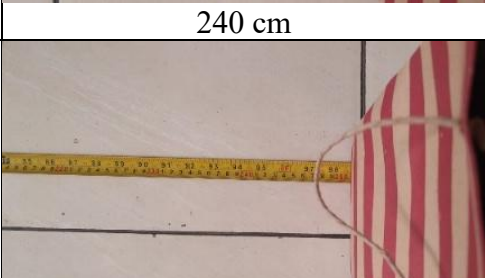
Lampiran 1. Dokumentasi Pengujian Sensor LiDAR VL53L1X

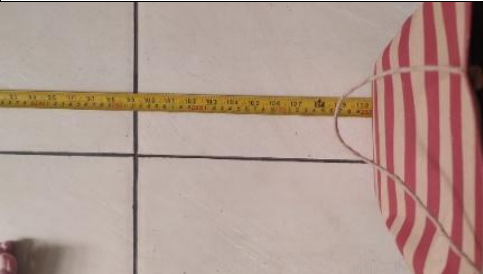
No	Jarak Asli	Pembacaan Lidar Lewat Terminal Monitor Arduino IDE
1	 10 cm	<pre>[LiDAR] Jarak terdeteksi: 100 mm (10.0 cm)</pre>
2	 20 cm	<pre>[LiDAR] Jarak terdeteksi: 200 mm (20.0 cm)</pre>
3	 30 cm	<pre>[LiDAR] Jarak terdeteksi: 276 mm (27.6 cm)</pre>
4	 40 cm	<pre>[LiDAR] Jarak terdeteksi: 380 mm (38.0 cm)</pre>
5	 50 cm	<pre>[LiDAR] Jarak terdeteksi: 488 mm (48.8 cm)</pre>

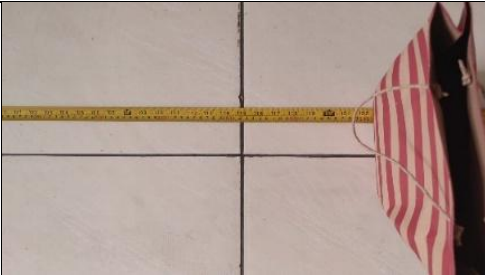
6	 60 cm	[LiDAR] Jarak terdeteksi: 577 mm (57.7 cm)
7	 70 cm	[LiDAR] Jarak terdeteksi: 675 mm (67.5 cm)
8	 80 cm	[LiDAR] Jarak terdeteksi: 776 mm (77.6 cm)
9	 90 cm	[LiDAR] Jarak terdeteksi: 877 mm (87.7 cm)
10	 100 cm	[LiDAR] Jarak terdeteksi: 967 mm (96.7 cm)

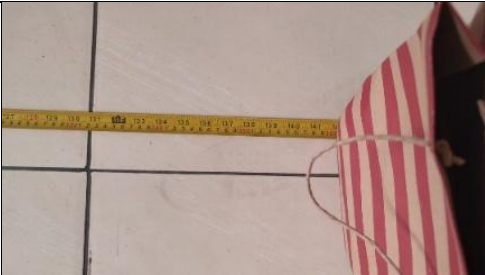
11	 110 cm	[LiDAR] Jarak terdeteksi: 1064 mm (106.4 cm)
12	 120 cm	[LiDAR] Jarak terdeteksi: 1202 mm (120.2 cm)
13	 130 cm	[LiDAR] Jarak terdeteksi: 1288 mm (128.8 cm)
14	 140 cm	[LiDAR] Jarak terdeteksi: 1393 mm (139.3 cm)
15	 150 cm	[LiDAR] Jarak terdeteksi: 1493 mm (149.3 cm)

16	 160 cm	[LiDAR] Jarak terdeteksi: 1581 mm (158.1 cm)
17	 170 cm	[LiDAR] Jarak terdeteksi: 1649 mm (164.9 cm)
18	 180 cm	[LiDAR] Jarak terdeteksi: 1788 mm (178.8 cm)
19	 190 cm	[LiDAR] Jarak terdeteksi: 1917 mm (191.7 cm)
20	 200 cm	[LiDAR] Jarak terdeteksi: 2096 mm (209.6 cm)

21	 210 cm	[LiDAR] Jarak terdeteksi: 2146 mm (214.6 cm)
22	 220 cm	[LiDAR] Jarak terdeteksi: 2219 mm (221.9 cm)
23	 230 cm	[LiDAR] Jarak terdeteksi: 2280 mm (228.0 cm)
24	 240 cm	[LiDAR] Jarak terdeteksi: 2370 mm (237.0 cm)
25	 250 cm	[LiDAR] Jarak terdeteksi: 2470 mm (247.0 cm)

26	 260 cm	[LiDAR] Jarak terdeteksi: 2588 mm (258.8 cm)
27	 270 cm	[LiDAR] Jarak terdeteksi: 2692 mm (269.2 cm)
28	 280 cm	[LiDAR] Jarak terdeteksi: 2845 mm (284.5 cm)
29	 290 cm	[LiDAR] Jarak terdeteksi: 2911 mm (291.1 cm)
30	 300 cm	[LiDAR] Jarak terdeteksi: 3084 mm (308.4 cm)

31	 310 cm	[LiDAR] Jarak terdeteksi: 3123 mm (312.3 cm)
32	 320 cm	[LiDAR] Jarak terdeteksi: 3179 mm (317.9 cm)
33	 330 cm	[LiDAR] Jarak terdeteksi: 3301 mm (330.1 cm)
34	 340 cm	[LiDAR] Jarak terdeteksi: 3362 mm (336.2 cm)
35	 350 cm	[LiDAR] Jarak terdeteksi: 3505 mm (350.5 cm)

36	 360 cm	[LiDAR] Jarak terdeteksi: 3591 mm (359.1 cm)
37	 370 cm	[LiDAR] Jarak terdeteksi: 3638 mm (363.8 cm)
38	 380 cm	[LiDAR] Jarak terdeteksi: 3748 mm (374.8 cm)
39	 390 cm	[LiDAR] Jarak terdeteksi: 3967 mm (396.7 cm)
40	 400 cm	[LiDAR] Jarak terdeteksi: 4055 mm (405.5 cm)

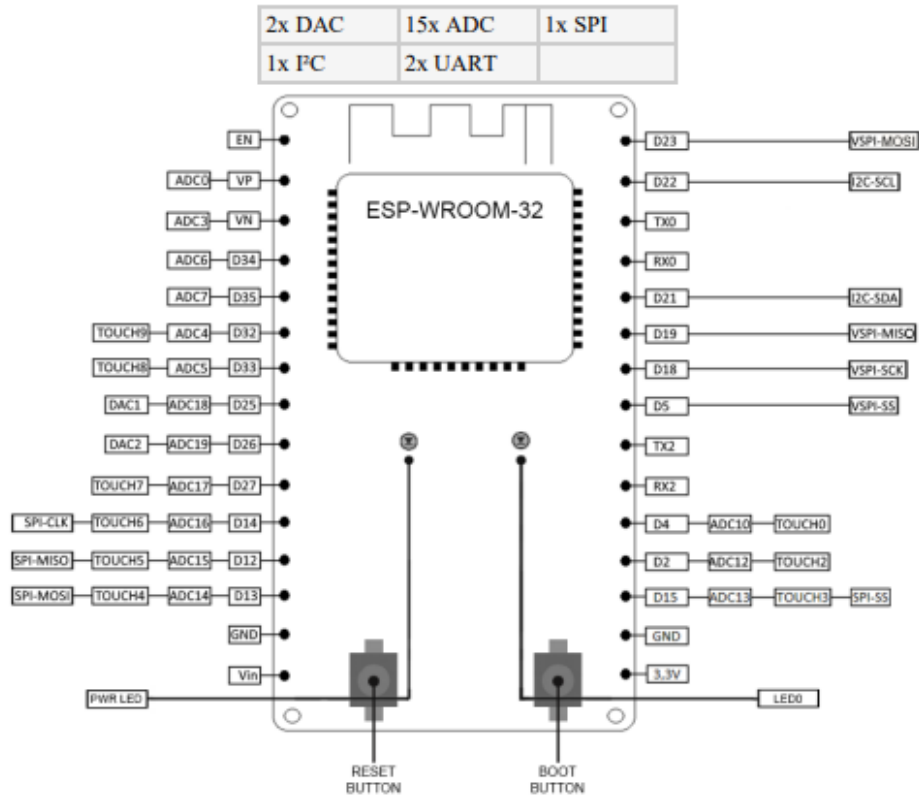
Lampiran 2. Datasheet ESP32 Dev Kit V1

2. OVERVIEW

The NodeMCU ESP32 module is a compact prototyping board and can be easily programmed via the Arduino IDE. It features 2.4 GHz dual-mode Wi-Fi and Bluetooth. Also integrated on the microcontroller development board are 512kb SRAM and 16MB memory.

PWM is enabled on every digital pin.

An overview of the available pins can be seen in the following figure:



Flash Layout

The internal flash of the ESP32 module is organized in a single flash area with pages of 4096 bytes each. The flash starts at address 0x00000, but many areas are reserved for Esp32 IDF SDK and Zerynth VM. There exist two different layouts based on the presence of BLE support.

In particular, for non-BLE VMs:

Start address	Size	Content
0x00009000	16Kb	Esp32 NVS area
0x0000D000	8Kb	Esp32 OTA data
0x0000F000	4Kb	Esp32 PHY data
0x00010000	1Mb	Zerynth VM
0x00110000	1Mb	Zerynth VM (FOTA)
0x00210000	512Kb	Zerynth Bytecode
0x00290000	512Kb	Zerynth Bytecode (FOTA)
0x00310000	512Kb	Free for user storage
0x00390000	448Kb	Reserved

For BLE VMs:

Start address	Size	Content
0x00009000	16Kb	Esp32 NVS area
0x0000D000	8Kb	Esp32 OTA data
0x0000F000	4Kb	Esp32 PHY data
0x00010000	1216Kb	Zerynth VM
0x00140000	1216Kb	Zerynth VM (FOTA)
0x00270000	320Kb	Zerynth Bytecode
0x002C0000	320Kb	Zerynth Bytecode (FOTA)
0x00310000	512Kb	Free for user storage
0x00390000	448Kb	Reserved

Device Summary

- Microcontroller: Tensilica 32-bit Single-/Dual-core CPU Xtensa LX6
- Operating Voltage: 3.3V
- Input Voltage: 7-12V
- Digital I/O Pins (DIO): 25
- Analog Input Pins (ADC): 6
- Analog Outputs Pins (DAC): 2
- UARTs: 3
- SPIs: 2
- I2Cs: 3
- Flash Memory: 4 MB
- SRAM: 520 KB
- Clock Speed: 240 Mhz
- Wi-Fi: IEEE 802.11 b/g/n/e/i:
 - Integrated TR switch, balun, LNA, power amplifier and matching network
 - WEP or WPA/WPA2 authentication, or open networks

Power

Power to the DOIT Esp32 DevKit v1 is supplied via the on-board USB Micro B connector or directly via the "VIN" pin. The power source is selected automatically.

The device can operate on an external supply of 6 to 20 volts. If using more than 12V, the voltage regulator may overheat and damage the device. The recommended range is 7 to 12 volts.

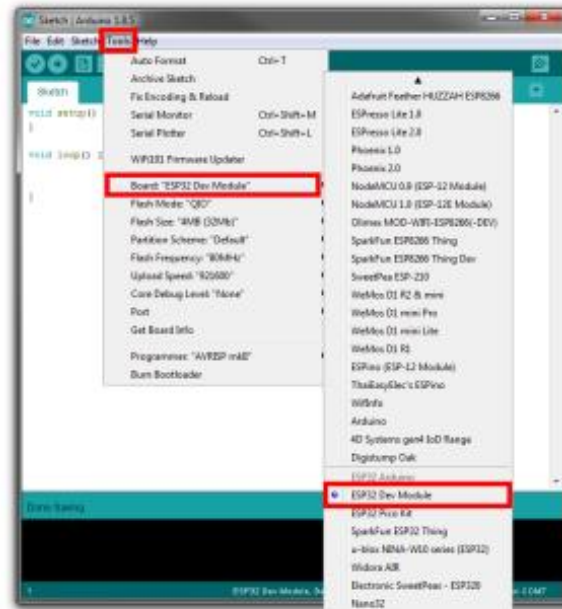
Connect, Register, Virtualize and Program

The DOIT Esp32 DevKit v1 comes with a serial-to-usb chip on board that allows programming and opening the UART of the ESP32 module. Drivers may be needed depending on your system (Mac or Windows) and can be download from the official [Espressif documentation](#) page. In Linux systems, the DevKit v1 should work out of the box.

3. MODULE INSTALLATION

If you have not yet installed the [Arduino IDE](#) on your computer, download and install it first.

Now open the Arduino IDE and change the board setting to "ESP32 Dev Module":



Download and install the [updated CP210x USB-UART driver](#) for your operating system.

The next step is to download the [official ESP32 library](#).

Unpack the contents of the archive into the "hardware" folder of your Arduino folder ("C:\Users\[your username]\Documents\Arduino\hardware").

Now restart your Arduino IDE. The module installation is now complete.

4. USAGE

Your NodeMCU ESP32 is now ready to use. Simply connect it to your computer with a USB cable. The installed library already provides many examples to allow a quick insight into the module. You can find the examples in your Arduino IDE under "File -> Examples -> ESP32".

The fastest and easiest way to test your NodeMCU ESP32 is to get the device number. Either copy the code below or use the example "GetChipID":

```
uint64_t chipid;

void setup() {
  Serial.begin(115200);
}

void loop() {
  chipid=ESP.getEfuseMac();//The chip ID is essentially its MAC address
  (length: 6 bytes).
  Serial.printf("ESP32 Chip ID = %04X",(uint16_t)(chipid>>32));//print
  High 2 bytes
  Serial.printf("%08X\n",(uint32_t)chipid);//print Low 4bytes.

  delay(3000);
}
```

To upload, click the upload button from the Arduino IDE and hold down the "BOOT"-button on the SBC NodeMCU ESP32. The upload is completed until the writing has reached 100% and you will be asked to reboot (hard resetting via RTS pin ...) with the "EN" key.

You can show the output of the example in the serial monitor.

Lampiran 3. Datasheet Sensor LiDAR VL53L1X



VL53L1X

Datasheet

A new generation, long distance ranging Time-of-Flight sensor based on ST's FlightSense technology



Product status link
VL53L1X

Features

Fully integrated miniature module

- 940 nm invisible laser emitter (Class 1)
- SPAD (single photon avalanche diode) receiving array with integrated lens
- Low power microcontroller running advanced digital firmware
- 4.9 x 2.5 x 1.56 mm

Fast, accurate distance ranging

- Fast and accurate long distance ranging
 - Up to 400 cm distance measurement
 - Up to 50 Hz ranging frequency
- Pin-to-pin compatible with the VL53L0X FlightSense ranging sensor
- 27° typical full field of view (FoV)
- Programmable region of interest (ROI) size on the receiving array allows reduction of the sensor FoV
- A programmable ROI position on the receiving array, provides multizone operation control from the host

Eye safety

- Class 1 laser device compliant with latest standard IEC 60825-1:2014 - 3rd edition

Easy integration

- Software driver and code example for turnkey ranging
- Single reflowable component
- Single power supply 2v8
- Can be hidden behind many cover glass materials
- I²C interface (up to 400 kHz)
- Shutdown and interrupt pins

Application

- User detection (autonomous low-power mode) to power on/off and lock/unlock devices like personal computers/laptops and the Internet of Things (IoT)
- Service robots and vacuum cleaners (long distance and fast obstacle detection)
- Drones (landing assistance, hovering, ceiling detection)
- Smart shelves and vending machines (goods inventory monitoring)
- Sanitary (robust user detection whatever the target reflectance)
- Smart building and smart lighting (people detection, gesture control)
- 1 D gesture recognition
- Laser assisted autofocus (AF) enhances the camera AF system's speed and robustness, especially in difficult scenes such as low light and low contrast, and provides video focus tracking assistance

Description

The VL53L1X is a state-of-the-art, Time-of-Flight (ToF) laser-ranging sensor, enhancing the ST FlightSense product family. It is the fastest miniature ToF sensor on the market with accurate ranging up to 4 m and fast ranging frequency up to 50 Hz.

Housed in a miniature and reflowable package, it integrates a SPAD receiving array, a 940 nm invisible Class 1 laser emitter, physical infrared filters, and optics to achieve the best ranging performance in various ambient lighting conditions with a range of cover glass options.

Unlike conventional IR sensors, the VL53L1X uses ST's latest generation ToF technology, which allows absolute distance measurement whatever the target color and reflectance.

It is also possible to program the size of the ROI on the receiving array, allowing the sensor FoV to be reduced.

2 Product overview

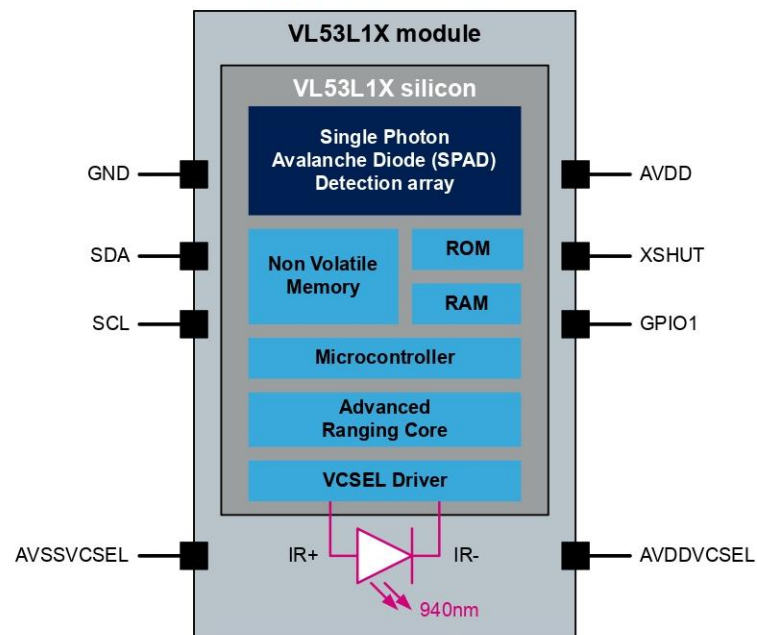
2.1 Technical specification

Table 2. Technical specification

Feature	Detail
Package	Optical LGA12
Size	4.4 x 2.5 x 1.56 mm
Operating voltage	2.6 to 3.5 V
Operating temperature	-20 to 85°C
Receiver FoV (diagonal FoV)	Programmable from 15 to 27 degrees
Infrared emitter	940 nm
I ² C	Up to 400 kHz (fast mode) serial bus Programmable address default is 0x52

2.2 System block diagram

Figure 1. VL53L1X block diagram



2.3 Device pinout

The following figure shows the pinout of the VL53L1X (see also [Section 8: Outline drawings](#)).

Figure 2. VL53L1X pinout (bottom view)

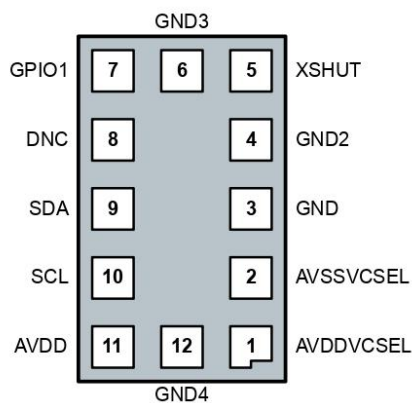


Table 3. VL53L1X pin description

Pin number	Signal name	Signal type	Signal description
1	AVDDVCSEL	Supply	VCSEL supply, connect to main supply
2	AVSSVCSEL	Ground	VCSEL ground, connect to main ground
3	GND		Connect to the main ground
4	GND2		
5	XSHUT	Digital input	Xshutdown pin, active low
6	GND3	Ground	Connect to the main ground
7	GPIO1	Digital output	Interrupt output. Open drain output
8	DNC	Digital input	Do not connect, must be left floating
9	SDA	Digital input/output	I ² C serial data
10	SCL	Digital input	I ² C serial clock input
11	AVDD	Supply	Supply, connect to the main supply
12	GND4	Ground	Connect to the main ground

Note: AVSSVCSEL and GND are ground pins and can be connected together in the application schematics.

Note: GND2, GND3, and GND4 are standard pins that we force to the ground domain in the application schematics to avoid possible instabilities if set to other states.

Lampiran 4. Datasheet *Vibration Motor Module*

Vibration Motor Module Mini Flat Vibrating DC Motor



Features:

- Rated Voltage: 5.0VDC
- Working Voltage: 3.0 - 5.3VDC
- Rated Rotate Speed: Min. 9000RPM
- Rated Current: Max. 60mA
- Starting Current: Max. 90mA
- Starting Voltage: DC 3.7V
- Insulation Resistance: 10Mohm
- Compatibility: Compatible with UNO R3 / Mega2560.

Lampiran 5. Source Code Program

Seluruh source code penelitian dapat diakses melalui repositori GitHub berikut:

<https://github.com/Rahmawati612/RANCANG-BANGUN-TONGKAT-PINTAR-DETEKSI-OBJEK-DENGAN-CNN-MOBILENET-DAN-NAVIGASI-BAGI-TUNANETRA.git>

DAFTAR RIWAYAT HIDUP



A. DATA PRIBADI

Nama Lengkap : Rahmawati Divana Putri
Tempat, Tanggal Lahir : Tulungagung, 12 Agustus 2003
Jenis Kelamin : Perempuan
Kewarganegaraan : Indonesia
Agama : Islam
Alamat : Ngunut, Tulungagung Jawa Timur
No. Telepon : 085694836522
Email : rahmawatidivanap@gmail.com

B. RIWAYAT PENDIDIKAN

1. TK Dharma Wanita Persatuan Ngunut	2009-2010
2. SDN 5 Ngunut	2011-2016
3. SMPN 1 Ngunut	2016-2019
4. SMA Negeri 1 Ngunut	2019-2022
5. Politeknik Negeri Malang	2022-2026

C. PENGALAMAN KERJA

PT. Telekomunikasi Seluler Indonesia

1 Juli – 24 Oktober 2025

Bagian Radio Transport & Power Engineering (RTPE) Department

- ☐ Melakukan simulasi cakupan sinyal dan analisis *link budget* menggunakan perangkat lunak Atoll dan QGIS
- ☐ Menganalisis parameter transmisi kompleks pada 35 *cellular sites* guna menjaga stabilitas layanan dan meminimalkan gangguan frekuensi radio (*RF impairments*).
- ☐ Berkolaborasi dengan tim *engineering* senior dalam mengevaluasi performa jaringan dan menyusun laporan teknis.